

DRAFT · PRE-MAINNET

REINEIRA LABS LIMITED

ReineiraOS Whitepaper

Specification of the Reineira Settlement Protocol, the RSS Standard, and the ReineiraOS Builder Stack

Draft — June 2026 · v1.0 at chaos-net mainnet (Q4 2026)

grimnote
madschristensen99
0xkkkn

Reineira Labs Limited · RAK DAO Free Zone · UAE
github.com/ReineiraOS · engineering@reineira.xyz

Abstract

Encrypted-state conditional settlement on programmable chains lacks a primitive surface that composes pluggable verification with pluggable underwriting under a shared operator set. ReineiraOS specifies four composable primitives (confidential escrow, pluggable gate, FHE-priced recourse, slashing-secured operator set) over a TFHE coprocessor substrate, formalises them as the Reineira Settlement Standard (RSS), and ships them with an AI-native builder stack — Reineira Atlas and Reineira Code. The protocol launches under a Public-Infrastructure regime: every contract is an immutable singleton with no upgrade authority; the protocol charges no fees; there is no token; settlement is non-custodial; operators stake cUSDC and earn their own relay fees. The Reineira Settlement Protocol is published by Reineira Labs Limited (RAK DAO Free Zone) as a Software Vendor. Arbitrum L2 is the canonical deployment; Ethereum L1 is planned; OP-Stack and Atlas-deployed third-party L3s extend the host inventory. Chaos-net deploys on Arbitrum L2 in July 2026.

Contents

§0 — Version Manifest	3
0.1 Versioning Matrix	4
0.2 License Summary	4
§1 — Abstract & Introduction	4
1.0 Abstract	4
1.1 The ReineiraOS stack	4
1.2 Background	5
1.3 Contributions	6
1.4 Non-goals	6
1.5 Status and confidentiality posture	6
1.6 Reading order	7
1.7 Notation and conventions	7
§2 — Background & Related Work	7
2.1 Encrypted state on EVM chains	7
2.2 Conditional and streaming settlement	8
2.3 Decentralized insurance and coverage	8
2.4 Operator and restaking networks	8
2.5 Cross-chain settlement transports	9
2.6 Account abstraction and meta-transactions	9
2.7 Settlement layers and multi-chain hosting	10
2.8 Synthesis — the gap	10
§3 — System Architecture	10
3.1 Design Goals	10
3.2 The Five-Layer Stack	10
3.3 Package Topology	11
3.4 Storage and Immutability Model	12
3.5 Meta-Transaction Model	13
3.6 Dual-Mode Operation — Public and Encrypted	13
3.7 The Four Primitives — At a Glance	14
3.8 Deployment Substrate	14
3.9 Conformance vs Reference Implementation	14
§4 — Confidential Settlement Primitives	14
4.1 Cryptographic substrate	14
4.2 Encrypted types	15
4.3 Homomorphic operations	15
4.4 Permission model	16
4.5 The silent-failure pattern	16
4.6 Observable vs confidential state	17
4.7 Encrypted-state invariants	18
4.8 Limits and open questions	18
§5 — Reineira Settlement Standard (RSS)	19
5.1 Standard versus Reference Implementation	19
5.2 Scope	19
5.3 Versioning	20
5.4 Conformance Criteria	20
5.5 Core Interfaces	20
5.6 Composability via Plugins	23
5.7 Conformance Test Surface	24
5.8 ERC and EIP Alignment	24
5.9 Protocol-level consequences of the standards-fork pattern	25

§6 — Escrow Engine	25
6.1 Role in the four-primitive system	25
6.2 State Shape	25
6.3 Lifecycle	26
6.4 Canonical Functions	27
6.5 Condition Resolver Pluggability	29
6.6 Virtual Escrows	29
6.7 Reentrancy and Economic Safety	29
6.8 Reference Plugins (Condition Resolvers)	30
6.9 Known Limitations and Open Problems	30
§7 — Recourse	31
7.1 Role in the four-primitive system	31
7.2 Role separation	31
7.3 Contract surface	33
7.4 Encrypted coverage purchase	33
7.5 Underwriter policy interface	34
7.6 Dispute flow	35
7.7 LP capital and economic invariants	35
7.8 Risk-score attestation	36
7.9 Reference plugins	36
7.10 Specification-vs-implementation gaps	36
7.11 Open problems	37
§8 — Operator Set & Decentralization	37
8.1 Role in the four-primitive system	37
8.2 Why a separate operator set	37
8.3 Contract surface	38
8.4 Registration, stake, and admission	39
8.5 Task routing and execution windows	40
8.6 Quorum and dispute	41
8.7 Slashing	43
8.8 Operator-fee routing and the protocol’s zero-charge posture	43
8.9 Specification-vs-implementation gaps	44
8.10 Decentralization roadmap	45
8.11 Off-chain operator services	46
§9 — Cross-Chain Settlement & Multi-Chain Hosting	47
Part A — Cross-Chain Settlement	47
Part B — Multi-Chain Hosting Roadmap	49
§10 — Security Model & Threat Analysis	52
10.1 Threat model	52
10.2 Trust assumptions	54
10.3 Formal properties	55
10.4 Observable vs confidential boundary	56
10.5 Access control	56
10.6 Known audit posture	56
10.7 Non-custodial software posture	57
10.8 Limits	57
§11 — Governance Posture	57
11.1 Immutable contracts, no upgrade authority	57
11.2 Non-custodial protocol, Software Vendor, no token, no fees	58
§12 — Economics	58
12.1 No token	58
12.2 The protocol charges nothing	59
12.3 Earning roles	59
12.4 Reineira Labs	60

§13 — Licensing & Public-Goods Posture	60
13.1 Licensing Summary	60
13.2 Layer-Asymmetric License Vector	60
13.3 What BUSL-1.1 Permits and Prohibits	61
13.4 Five-Trigger Relicense Schedule	61
13.5 Boundary Conditions	62
§14 — Conclusion & Open Problems	62
14.1 What this paper specified	62
14.2 What this paper deferred	63
14.3 Open Problems	63
14.4 Implementation Notes — spec-vs-code deltas	64
14.5 Closing statement	65
Appendix A — References	65
A.1 Cryptography and FHE	65
A.2 Settlement, Escrow, and Insurance Primitives	66
A.3 Restaking and Operator Networks	67
A.4 Cross-Chain Transports	67
A.5 L2 / L3 Hosting Stacks	67
A.6 Standards Bodies and Regulation	67
A.7 Public Standards (EIPs / ERCs / RFCs)	67
Appendix B — ERC / EIP Compliance Matrices	68
B.1 Per-contract compliance matrix	68
B.2 ERC-165 interface detection	69
B.3 ERC-7201 namespaced storage	70
B.4 ERC-2771 meta-transaction support	70
B.5 Deployment posture: immutable singletons (v1.0 mainnet)	70
B.6 Standards not used (and why)	71
Appendix C — Formal Specification Stubs	71
Notation	71
C.1 IConditionResolver	71
C.2 IUnderwriterPolicy / IConfidentialUnderwriterPolicy	72
C.3 IConfidentialEscrow	72
C.4 IExecution / IAgenticJob (Implementation Note)	72
C.5 Operator surfaces	72
C.6 Bridge handlers (CCTPHandler)	73
C.7 Open invariants for mechanised verification	73

§0 — Version Manifest

Draft — June 2026 · v1.0 at chaos-net mainnet (Q4 2026)

This document is a **pre-mainnet draft** of the ReineiraOS Whitepaper, dated June 2026. The whitepaper is mutable through the chaos-net hardening window and stabilises to edition **v1.0** only at chaos-net mainnet (Q4 2026), in lockstep with contract v1.0 and the first external smart-contract audit (§10.6). Whitepaper editions are date-versioned; semantic versioning is reserved for the protocol artifacts (the Reineira Settlement Standard and the on-chain contracts) defined in §5 and §6. Subsequent editions supersede earlier ones in full and carry a new month-year stamp.

Authors

[grimnote](#) — Founder & Protocol Lead [madschristensen99](#) — Protocol Developer & Web3 Lead [0xkkkn](#) — Head of Engineering

Reineira Labs Limited · RAK DAO Free Zone · UAE github.com/ReineiraOS · contact: engineering@reineira.xyz

Implementation Note (Publishing Entity). Published by Reineira Labs Limited (RAK DAO Free Zone) as a Software Vendor (§11.2).

0.1 Versioning Matrix

Layer	Versioning	Current
Whitepaper	date-versioned	Draft June 2026 → v1.0 at chaos-net mainnet (Q4 2026)
RSS (Reineira Settlement Standard)	semver	v0.1 testnet Jun 2026 → v0.2 chaos-net Q4 2026 → v1.0 at mainnet activation
Contracts	semver per deployment	v0.x testnet → v1.0 chaos-net mainnet (Q4 2026) → multi-chain expansion thereafter; all contracts immutable per §11.1
Builder stack	semver, platform-locked	Reineira Atlas v0.1.0 · Reineira Code v0.1.0 · platform: "0.1"
Lightpaper	date-stamped only	Draft June 2026 → v1.0 at chaos-net mainnet (Q4 2026)

Chaos-net (public-mode mainnet on Arbitrum L2) launches in July 2026; **the protocol charges no fees**. RSS v0.2 ships in Q4 2026 alongside the first five RIPS. The contract pragma is **^0.8.24**, and contracts are compiled with solc 0.8.25 (cancun EVM, optimizer 200 runs). Every protocol contract is deployed as an immutable singleton; no entity holds upgrade authority (§11.1).

0.2 License Summary

Source code: BUSL-1.1 with an irrevocable Change Date of 2029-06-01 and Apache 2.0 as the Change License; **Licensors: Reineira Labs Limited** (RAK DAO Free Zone). See §13 for the full clause table, layer-asymmetric vector, and trigger schedule.

§1 — Abstract & Introduction

1.0 Abstract

Encrypted-state conditional settlement on programmable chains lacks a primitive surface that composes pluggable verification with pluggable underwriting under a shared Operator set. ReineiraOS specifies four composable primitives (confidential Escrow, pluggable Gate, FHE-priced Recourse, and a slashing-secured Operator set) over a TFHE coprocessor substrate, formalises them as the Reineira Settlement Standard (RSS), and ships them with an AI-native builder stack — Reineira Atlas and Reineira Code. The protocol launches under a Public-Infrastructure regime: every contract is an immutable singleton with no upgrade authority (§11.1); the protocol charges no fees; there is no token; settlement is non-custodial; Operators stake cUSDC and earn relay fees (§8.8). The Reineira Settlement Protocol is published by Reineira Labs Limited (RAK DAO Free Zone) as a Software Vendor (§11.2). Arbitrum L2 is the canonical deployment; Ethereum L1 is planned; OP-Stack and Atlas-deployed third-party L3s extend the host inventory (§9). Chaos-net deploys on Arbitrum L2 in July 2026.

1.1 The ReineiraOS stack

ReineiraOS is the union of three artifacts published together: (i) the **Reineira Settlement Protocol**, a set of on-chain contracts implementing confidential Escrow, pluggable underwriting, and Operator-secured cross-chain settlement (§3–§8); (ii) the **Reineira Settlement Standard (RSS)**, a conformance specification with semver-versioned releases and a Reineira Improvement Proposal process that any deployer of the protocol can target (§5); and (iii) a **builder stack** — Reineira Atlas and Reineira Code — that scaffolds ventures and authors plugins against (i) and (ii). The three artifacts move in lockstep: the protocol defines the interfaces, RSS freezes them as a public standard, and the builder stack mechanises their use. Fig. 1.1 shows the layering.

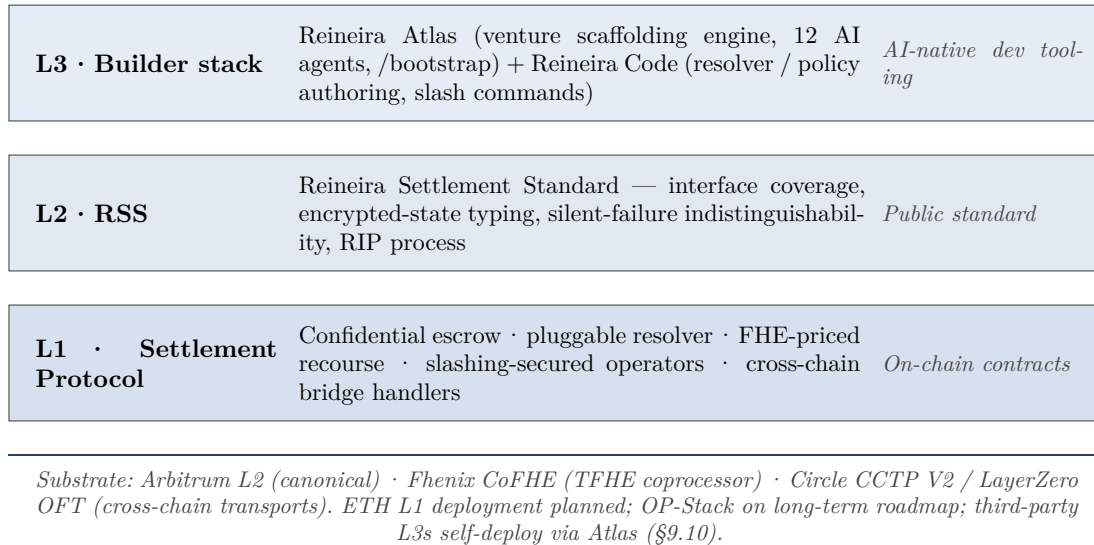


Figure 1: **Fig. 1.1 — The ReineiraOS stack.** Three layers move in lockstep: the on-chain Settlement Protocol provides the primitives; RSS standardises the surface; the builder stack (Reineira Atlas + Reineira Code) scaffolds ventures and plugins. Atlas templates a venture from a one-page brief in roughly an hour; Reineira Code generates and tests RSS-conformant resolvers and underwriter policies.

Reineira Atlas (`reineira-atlas`) is the venture scaffolding engine: a Claude Code repository with twelve AI agents organised across strategy, product, growth, execution, intelligence, and legal domains. A builder completes `brief.md`, runs the `/bootstrap` command, and Atlas generates a populated docs tree, sprint log, action items, and a working application skeleton from `platform-modules` (TypeScript backend + React 19 app with ZeroDev smart accounts). Atlas is platform-locked to a contract-interface version (`platform: "0.1"` in `reineira.json`): a breaking interface change in the protocol bumps the platform version, and Atlas advertises the incompatibility.

Reineira Code (`reineira-code`) is the AI-assisted plugin authoring environment: a Hardhat project with slash commands (`/new-resolver`, `/new-policy`, `/deploy`, `/test`, `/audit`, `/integrate`) that generate RSS-conformant Solidity from a natural-language description, run the `coffejs` FHE test suite, and wire the result into the Atlas-generated application via the SDK. The two repositories together compress the path from idea to deployed product to roughly one hour for the scaffold and one to two weeks for a custom resolver and audit-ready test suite.

The whitepaper specifies (i) and (ii) in full; the builder stack is described where it is load-bearing for the protocol’s open-economy thesis (§5.6, §9.10, §14.2) and is not treated as a separate normative surface.

1.2 Background

Two prior research programmes have matured in parallel without converging. Encrypted-state EVMs (Aztec, Penumbra, Zama fhEVM, Inco, Fhenix CoFHE) make confidential computation tractable at contract granularity (§2.1). Conditional-release patterns (HTLCs, OpenZeppelin escrows, Sablier streams, parametric coverage at Nexus and Sherlock) separate the release predicate from the value held (§2.2, §2.3). No prior

system composes encrypted conditional release with pluggable verification, pluggable underwriting, and a single operator set under cross-graph slashing as one primitive surface; that gap is the protocol's subject.

1.3 Contributions

This document specifies six contributions, each anchored to the chapter that develops it.

1. **The Reineira Settlement Standard (RSS).** A conformance specification (interface coverage, encrypted-state typing, silent-failure indistinguishability, namespaced upgrade-safe storage, canonical events) separated from the reference implementation, with semver-versioned releases and a Reineira Improvement Proposal process (§5).
2. **A four-primitive composable surface.** Confidential escrow (§6), pluggable `IConditionResolver` Gates (§5.5.1, §6.5), FHE-priced recourse with four-role separation (§7), and a permissionless operator set under stake-weighted task authorization (§8) — interoperable through `@reineira-os/shared` interfaces without core-contract modification.
3. **The silent-failure pattern.** A control-flow construction in which the success and failure transcripts of an encrypted-condition redemption are computationally indistinguishable to any party without decryption authority, formalised as Proposition 4.3 under TFHE IND-CPA.
4. **FHE-priced recourse.** Premiums computed via homomorphic arithmetic on encrypted risk scores returned by an `IConfidentialUnderwriterPolicy` plugin, with per-buyer premiums and per-buyer payouts held as `euInt64` ciphertexts (§7.4). The reference implementation ships `ConfidentialCoverageManager`, `ConfidentialRecoursePool`, `ConfidentialPolicyRegistry`, and `ConfidentialPoolFactory`; the `QuorumAttestedPolicy`, `RiskScoreAttestor`, and full `PoolFactory` surface are specified and tracked toward chaos-net v1.0 per §7.10.
5. **A multi-chain immutable reference implementation with a bridge-handler abstraction.** Arbitrum L2 is the canonical host; Ethereum L1 is planned as a second host; OP-Stack hosting is on the long-term roadmap. **All contracts are immutable singletons** (§11.1); functional evolution ships as parallel deployments at new addresses with opt-in user migration. Cross-chain funding via a forwarder–receiver pair funnels into `ConfidentialEscrow.fundFrom`. The shipped surface covers Circle CCTP V2 for USDC and LayerZero OFT (USDT0) for USDT; cUSDC and cUSDT both ship at v1.0 as immutable ERC-7984 wrappers, with the operator bond denominated in cUSDC only (§8.1). Third-party rollups self-deploy ReineiraOS via Reineira Atlas, with operator-routed cross-chain liquidity flowing over native rollup bridges (§9.10). Deployment and maintenance are operated by Reineira Labs Limited (RAK DAO Free Zone) as Software Vendor, which never custodies user funds or admin keys (§11.2).
6. **A layer-asymmetric license vector with a pre-announced trigger schedule.** Core protocol BUSL-1.1 → Apache 2.0 on the irrevocable Change Date 2029-06-01, with five publication-time accelerators (§13.4) that may shorten but cannot extend the conversion path. Licensors: Reineira Labs Limited.

1.4 Non-goals

This whitepaper specifies only the protocol-layer surfaces and references the builder stack (Reineira Atlas, Reineira Code) where it is load-bearing for the open-economy thesis (§5.6, §9.10, §14.2). Off-chain Operator service internals, builder-platform business logic above the SDK, and Reineira Labs Limited commercial products are out of scope. The protocol does not compete on pure permissionlessness at the Recourse pool layer (which would create role-collapsed owner-managed pools), on agentic micropayments at the under-ten-dollar tier, or on L2 substrate performance — the substrate is consumed, not built. The protocol charges no fees; settlement is non-custodial; and no entity holds upgrade authority over any deployed contract (§11.1).

1.5 Status and confidentiality posture

Implementation status. The reference implementation runs on Arbitrum testnet at publication, published by Reineira Labs Limited (RAK DAO Free Zone) as a Software Vendor. Settlement is non-custodial; there is no protocol token.

Chaos-net mainnet launches July 2026 on Arbitrum L2 in public mode; the protocol charges no fees; mainnet v1.0 hardening completes in Q4 2026 on Arbitrum L2 (the canonical host); the encrypted-mode deployment activates as a separate immutable contract set once Fhenix CoFHE mainnet lands. Ethereum L1 hosting is planned, with OP-Stack hosting on the long-term roadmap (§9.7–§9.9). Third-party rollups self-deploy ReineiraOS via Reineira Atlas under the L3-owner maintenance model (§9.10). Several specification items in this document diverge from current code and are surfaced as Implementation Notes under the convention introduced in §3 and collated in §14; this is the document’s primary spec-versus-implementation reconciliation surface.

In encrypted mode, sensitive settlement state (escrow owner, caller, amount, paid amount, redemption flag; coverage holder, coverage amount, risk score, premium, payout; per-pool liquidity and premium accumulators) is held as TFHE ciphertexts and is recoverable only by addresses on the substrate-side access list (§4.4). The protocol exposes a deliberate *public skeleton* (Definition 4.4) of plaintext invariants: escrow existence, event topics, operator identity for slashing attribution, and bridge-boundary token flow. This skeleton supports indexing, gas accounting, and audit without revealing per-actor activity, and the silent-failure pattern (§4.5) extends the same boundary to the redemption decision itself.

1.6 Reading order

Protocol researchers should read §3 (architecture) → §4 (cryptographic substrate) → §5 (the standard). Security auditors should read §4 (primitives and invariants) → §6/§7/§8 (engines and Operator set) → §10 (threat model). Governance and licensing readers should read §11 (governance) → §13 (licensing trajectory) → Appendix B (ERC compliance matrix). The full sequence is §0 manifest, §1 introduction, §2 background, §3 architecture, §4 confidential primitives, §5 RSS, §6 Escrow, §7 Recourse, §8 Operators, §9 cross-chain and multi-chain hosting, §10 security, §11 governance, §12 economics, §13 licensing, §14 conclusion and open problems, with Appendices A (references), B (ERC compliance), C (formal specs).

1.7 Notation and conventions

Encrypted types follow the substrate’s convention: **euint64** denotes a ciphertext over a 64-bit unsigned integer plaintext domain (the canonical amount type for six-decimal stablecoins), **eaddress** over a 160-bit account identifier, **ebool** over a single-bit value (§4.2). Cross-references use **§N.M** for sections, **Definition N.M**, **Proposition N.M**, **Eq. (N.M)**, **Fig. N.M**, and **Tab. N.M**; external citations follow the form **[Author Year]** resolved in Appendix A. The abbreviation **RSS** denotes the Reineira Settlement Standard throughout. Open Problems and Implementation Notes are numbered blockquote callouts whose convention is defined in §3 and collated in §14.

§2 — Background & Related Work

Reineira sits at the intersection of six prior research programmes: encrypted state on EVMs, conditional and streaming settlement, decentralized insurance, Operator and restaking networks, cross-chain stablecoin transports, and the cryptographic primitives that make encrypted state computable. This chapter situates each approach and identifies the gap that the present protocol occupies.

2.1 Encrypted state on EVM chains

Encrypted state on programmable chains has been pursued through two cryptographic families: zero-knowledge proofs and fully homomorphic encryption. Aztec applies the zkSNARK family to a UTXO-shaped private state and exposes a separate programming model (Noir) compiled to circuits, with private notes spent and created inside proofs rather than mutated in place [Aztec 2022]. Penumbra takes a similar shielded-pool approach in the Cosmos ecosystem, with delegated proving and asset-agnostic notes [Penumbra 2023]. The distinction relevant to settlement is that ZK approaches prove statements about private inputs at the

moment of state transition; ongoing computation over the encrypted values themselves is not native to the model.

The FHE-on-EVM lineage pursues the alternative: keep ciphertexts on chain and compute over them homomorphically. Zama’s `fhEVM` extends the EVM opcode set with TFHE-backed encrypted types and ships a developer kit for Solidity-style contracts that read and write ciphertexts [Zama 2023]. Inco Network packages a similar capability as an L1 with FHE precompiles and EVM equivalence at the Solidity layer [Inco 2024]. Both designs perform FHE evaluation in-protocol, which couples chain throughput to FHE-circuit cost.

Fhenix CoFHE — the substrate Reineira deploys against — separates the two concerns by delegating FHE evaluation to a coprocessor network and exposing operations as Solidity precompiles, with ciphertext integrity preserved through attestation [Fhenix 2024]. The result is an EVM environment whose gas cost is determined by precompile invocations rather than circuit synthesis, and whose ciphertext lifecycle (`allow`, `allowTransient`, `allowThis`) is managed by an explicit permission model rather than implicitly through circuit composition.

2.2 Conditional and streaming settlement

Conditional release of held funds is the oldest pattern in this lineage. Compound timelock escrows release on block-height or timestamp expiry; HTLCs (hash time-locked contracts) release on preimage disclosure and underlie cross-chain atomic swaps [Poon Dryja 2016]. OpenZeppelin’s `Escrow` and `ConditionalEscrow` base contracts canonicalised the pattern at the contract-library level [OpenZeppelin 2017]. All operate over plaintext amounts and plaintext owners; the existence of the escrow, its parties, and its condition state are public.

Streaming-payment primitives reframe the release schedule as a continuous function of time. Sablier introduced per-second linear streams as standalone contracts [Sablier 2020]; Superfluid generalised the construction to constant-flow agreements between accounts with off-chain net-flow accounting [Superfluid 2021]. Both extend the time dimension of release but retain the public-amount, public-counterparty surface.

Two adjacent constructions complete the design space. Aave’s Safety Module is a collateralised payout obligation backing protocol shortfalls, parameterised by a staking-derived backstop and a discretionary slash window [Aave 2020]. ERC-20 escrow patterns wrap fungible token transfers in release conditions but inherit the public-balance surface of the underlying token. The protocol described here applies the conditional-release pattern over encrypted amounts and encrypted ownership, with the release predicate evaluated homomorphically and the failure transcript made computationally indistinguishable from the success transcript — a categorical advance beyond plaintext conditional release.

2.3 Decentralized insurance and coverage

Decentralized insurance has converged on three approaches to the underwriting and claims problem. Nexus Mutual operates as a member-discretionary mutual: cover terms are denominated in NXM, claim adjudication runs through a member-vote process, and pool capital is mutualised across all members [Nexus 2019]. Sherlock and Risk Harbor apply a parametric model — coverage payouts trigger on machine-checkable conditions evaluated by designated model providers or claim authorities, with capital provided by an LP layer [Sherlock 2022], [RiskHarbor 2021]. Cover Protocol (subsequently discontinued in its original form) experimented with open pools whose claim conditions were oracle-driven [Cover 2020].

None of these designs price premiums over encrypted risk inputs, and none maintain encrypted positions on the LP-facing surface. The four-role separation that Reineira encodes in the Recourse primitive — Policy Builder writes the risk code, Pool Creator deploys the pool and retains policy-authorship rights, Pool Manager holds ongoing governance, and Operator routes execution and dispute submission — is also absent from these designs; the role-collapsed alternative (the same entity benefits from claim denials and judges them) is documented as a structural-conflict failure mode in the 2022 CeFi episode [CeFiPostmortem 2023].

2.4 Operator and restaking networks

The operator-set design space was reframed by EigenLayer, which separated cryptoeconomic security from execution by allowing Ethereum-staked ETH to be re-pledged to actively validated services (AVSs) with service-specific slashing conditions [EigenLayer 2023]. Symbiotic generalises the construction to multi-asset collateral and permissionless vault deployment [Symbiotic 2024]; Karak applies a similar re-pledging model with a focus on attester-style services [Karak 2024]. Across the three, the open design questions are task routing under stake-weighted selection, the slashing-evidence and challenge model, and the social-consensus layer for catastrophic-failure recovery.

The protocol described here applies a non-restaking Operator model: Operators stake cUSDC and are routed by Coordinators under cross-graph slashing consensus, which requires agreement across at least three independent Coordinator-Operator graphs before a slash executes (3-day challenge, 4-day voting, 14-day expiry). The design choice is independent: a restaking-derived Operator set could be substituted at the Coordinator boundary without affecting the Escrow or Recourse surfaces.

2.5 Cross-chain settlement transports

Cross-chain stablecoin movement has converged on two transport models. Circle’s Cross-Chain Transfer Protocol (CCTP, currently at V2) implements burn-on-source, attest-via-Iris, mint-on-destination semantics for native USDC, with the canonical issuer (Circle) acting as the attestation authority [Circle 2024]. LayerZero’s Omnichain Fungible Token (OFT) standard implements lock-or-burn on source, message-via-DVN-and-Executor, mint-or-unlock on destination, and underlies Tether’s LayerZero-native USDT (the USDT0 deployment) [LayerZero 2022], [LayerZero 2024]. The two differ in their attestation trust assumptions: CCTP delegates to the issuer; OFT delegates to a configurable Decentralized Verifier Network.

Hyperlane and Wormhole occupy adjacent positions. Hyperlane is a permissionless interchain messaging layer with modular Interchain Security Modules selectable per route [Hyperlane 2023]. Wormhole is a guardian-attested cross-chain message bus with a multisig validator set [Wormhole 2022]. The protocol described here uses CCTP V2 for USDC and the USDT0 OFT for USDT, in both cases as transport beneath an encrypted wrapping that is applied on the destination chain; the transport choice is orthogonal to the encrypted-settlement surface.

2.6 Account abstraction and meta-transactions

Two ERC tracks address the user-facing transaction surface. ERC-2771 specifies a trusted forwarder pattern in which a relayer submits the on-chain transaction on behalf of a user, with the user’s address extracted from the calldata suffix and verified against a per-contract trusted-forwarder allowlist [ERC-2771]. ERC-4337 implements account abstraction at the mempool layer, replacing externally-owned-account transactions with UserOperation objects bundled by an alternative mempool and executed against EntryPoint contracts [ERC-4337]. The two are not mutually exclusive but address different problems: ERC-2771 routes individual relayed calls into existing EOA-modelled contracts; ERC-4337 reshapes the account itself.

The protocol described here inherits ERC-2771 at the contract base layer (the protocol base contract composes the trusted-forwarder receiver alongside Initializable, Ownable, and ReentrancyGuard; UUPS is *not* in the inheritance chain under the immutable-contract posture of §11.1) and does not implement ERC-4337 in v1.0. The choice scopes the gasless-meta-transaction surface to relayer-bridged calls and leaves account-abstraction-derived features (sponsored bundling, session keys, paymaster economics) as future work.

Open Problem 2.1 (Account Abstraction). The v1.0 specification does not commit to an ERC-4337 integration path. Selection criteria: (a) paymaster economics under the encrypted-amount surface (a paymaster cannot price gas against a ciphertext balance without breaking confidentiality); (b) session-key semantics over ERC-7984 confidential transfer authorisations; (c) bundler-mempool privacy guarantees for UserOperations referencing encrypted state. Resolution target: post-chaos-net v1.0.

2.7 Settlement layers and multi-chain hosting

Application-specific settlement layers have emerged as a third tier above general-purpose L2s. Polygon AggLayer aggregates ZK-proven state from heterogeneous chains into a shared bridge with cross-chain atomic-composability claims [Polygon 2024]. The OP Stack Superchain composes OP-Stack-derived rollups under a shared sequencing and standardised bridge layer [OptimismSuperchain 2023]. Arbitrum Orbit allows permissionless deployment of L3 chains settling to Arbitrum One under either Arbitrum Nitro (fraud-proof) or AnyTrust (DAC) data-availability assumptions [ArbitrumOrbit 2024]. The design axes that separate them are data-availability cost, fraud-versus-validity proof model, finality assumption, and the sequencer-decentralisation roadmap.

Reineira launches in public mode on Arbitrum L2 (the canonical host), with Ethereum L1 hosting planned as a second deployment and OP-Stack hosting on the long-term roadmap. Third-party rollups — including Arbitrum Orbit L3s and (when supported) OP-Stack L3s — extend coverage by self-deploying ReineiraOS via Reineira Atlas, with operator-routed liquidity flowing over native rollup bridges. The full multi-chain treatment is in §9.

2.8 Synthesis — the gap

Each programme surveyed above addresses one axis of the settlement problem in isolation: ZK or FHE encrypted state without conditional release, conditional release without encrypted state, decentralized insurance without encrypted underwriting, restaking-derived Operator sets without an application-layer settlement surface, cross-chain stablecoin transport without confidentiality on the destination, and L3 substrates without a settlement-specific application layer. The present protocol occupies the gap between them: encrypted-state conditional settlement (confidential Escrow) with pluggable verification (Gate-shaped condition resolvers) and pluggable underwriting (four-role Recourse), deployed against a TFHE coprocessor substrate and an Operator set under cross-graph slashing consensus, with an open standard (RSS) governing the contract surface. The chapters that follow specify each construction formally.

§3 — System Architecture

3.1 Design Goals

The protocol is engineered around four design goals, stated as architectural invariants rather than aspirations of any single component.

1. **Encrypted state by default.** Sensitive ledger state is held as FHE ciphertext on the chains that support it, with plaintext used only where a deployment lacks an FHE coprocessor.
2. **Pluggable verification.** Every release condition is an external contract implementing a fixed interface, never a hardcoded predicate.
3. **Pluggable underwriting.** Coverage pricing and dispute adjudication are external contracts implementing a parallel interface, never a hardcoded curve.
4. **Decentralized operation.** The asynchronous work surface (cross-chain relay, agent calls, verdict submission, pool routing, dispute execution) is served by a single permissionless operator set bound by stake and slashing rather than by a whitelist.

3.2 The Five-Layer Stack

The protocol decomposes into five layers, each addressing a single concern and each replaceable in isolation. The stack is depicted in Fig. 3.1 and described in the paragraphs that follow.

Application	<i>Public API consumed by clients and adapters</i> create · fund · redeem · purchaseCoverage · dispute · invoke
Orchestration	<i>Task distribution, cross-chain relay, fee collection, dispute arbitration</i> OperatorRegistry · TaskExecutor · FeeManager · OperatorSlashingManager
Storage	<i>Encrypted state, immutable contracts, gasless user paths</i> ERC-7201 namespaced slots · immutable singleton deployments · ERC-2771 trusted forwarder
FHE	<i>Encrypted primitives and homomorphic arithmetic</i> euint64 · eaddress · ebool · Fhenix CoFHE precompiles
Settlement	<i>Cross-chain native stablecoin flow</i> Circle CCTP V2 (USDC) · LayerZero OFT / USDT0 (USDT) · ConfidentialUSDC / ConfidentialUSDT

Listing 1: **Fig. 3.1 — Reineira five-layer stack.** Higher layers depend only on the interfaces of layers below them; lower layers are unaware of which higher-layer caller invokes them. The Application and Settlement layers (lightly shaded) bracket the protocol on the user-facing and chain-facing sides.

The **Application** layer is the external API. It is the only surface a client integrates against: an escrow is created and redeemed, coverage is purchased and disputed, an agent job is invoked. Every other layer is implementation detail.

The **FHE** layer provides confidentiality. Encrypted types are evaluated by the Fhenix coprocessor and exposed as Solidity precompiles; the host EVM sees only ciphertext handles. The full cryptographic surface (types, operations, permission model, silent-failure pattern) is examined in §4.

The **Storage** layer enforces the discipline that makes the other four layers safe to evolve across deployment versions. Every contract uses ERC-7201 namespaced slots (Definition 3.1), inherits from the protocol base contract, and ships with a `uint256[50] private __gap` reserve. Contracts are immutable singletons; functional evolution ships as parallel deployments at new addresses (§11.1). The storage-layout discipline does not protect against in-place upgrades — there are none — but against storage-layout incompatibility between deployment versions that share namespaced interfaces.

The **Orchestration** layer turns declared intent into settled state. Coordinators register on-chain and dispatch tasks over SSE to staked operators; operators claim, execute through the appropriate bridge handler, and collect fees. The decentralisation properties of this layer are treated in §8.

The **Settlement** layer carries value across chains using native stablecoin rails: burn-and-mint via Circle CCTP V2 for USDC, OFT lock-and-release via LayerZero / USDT0 for USDT. Wrapping the asset itself is not used. Bridge handlers and the cross-chain flow are detailed in §9.

3.3 Package Topology

The on-chain protocol is a pnpm monorepo of five Solidity-bearing packages plus three supporting packages. Each package has a single ownership boundary, and contracts never reach across that boundary except through `@reineira-os/shared`.

Package	Owns
@reineira-os/shared	the protocol base contract, the substrate adapters, interfaces (<code>ICore</code> , <code>IConditionResolver</code> , <code>IUnderwriterPolicy</code> , <code>IEscrow</code> , <code>IAgenticJob</code> , <code>IExecution</code>), mocks
@reineira-os/tokens	<code>ConfidentialUSDC</code> , <code>ConfidentialUSDT</code> (ERC-7984 wrappers; immutable, non-upgradeable)
@reineira-os/escrow	<code>ConfidentialEscrow</code> , bridge receivers (<code>CCTPV2EscrowReceiver</code> , <code>LZ0FTEscrowReceiver</code>), matching forwarders, condition-resolver extensions
@reineira-os/insurance	<code>ConfidentialCoverageManager</code> , <code>RecoursePool</code> , <code>PoolFactory</code> , <code>PolicyRegistry</code> , <code>QuorumAttestedPolicy</code>
@reineira-os/orchestration	<code>OperatorRegistry</code> , <code>CoordinatorRegistry</code> , <code>TaskExecutor</code> , <code>FeeManager</code> , <code>OperatorSlashingManager</code> , <code>QuorumRegistry</code> , task handlers (<code>CCTPHandler</code> , <code>LZ0FTHandler</code> , <code>AgentCallHandler</code> , <code>VerdictSubmitHandler</code>)
@reineira-os/offchain	Coordinator service, operator node runner, CLI
@reineira-os/plugin-examples	Reference Gate and underwriter-policy plugins
@reineira-os/sdk	TypeScript client bindings against the Application layer

The **shared** package is the only allowed cross-package Solidity import path; all other packages depend on it and never on each other directly. This is the rule that keeps the storage invariants of §3.4 enforceable across the monorepo.

3.4 Storage and Immutability Model

The protocol uses two storage conventions across every contract.

Definition 3.1 (ERC-7201 Namespaced Storage). A storage layout in which each logical group of state variables is held in a struct at a deterministic slot derived from a unique namespace string, computed as `keccak256(abi.encode(uint256(keccak256(namespace)) - 1)) & ~bytes32(uint256(0xff))`, so that adding fields in a base contract cannot collide with derived-contract slots. The protocol namespaces every contract’s storage under `reineira.storage.<ContractName>`.

Definition 3.2 (Reference Implementation). A deployment of contracts published by Reineira Labs Limited under the Software Vendor regime (§11.2) that conforms to the published Reineira Settlement Standard (§5). A Reference Implementation is one of potentially many conforming implementations; the standard does not privilege the Reineira Labs Limited–published deployment over any other implementation that satisfies the same interface and behavioural conformance tests.

Immutability. Every protocol contract — escrow engine, coverage manager, pool factory, policy registry, operator registry, slashing manager, fee manager, task executor, all token wrappers, all task handlers — is deployed as an **immutable singleton** at a fixed address. No contract uses a UUPS proxy. No contract has an `_authorizeUpgrade` hook. No address holds upgrade authority over any deployed contract; the “upgrade-authority” trust assumption that conventional protocol designs require (and that this paper’s prior editions specified) does not exist in v1.0. The full immutability posture and its implications are treated in §11.1.

Versioning. Functional evolution ships through *new contract deployments at new addresses*. The migration semantics are opt-in: state held in v1 contracts remains accessible at the v1 addresses indefinitely; users wishing to use v2 functionality withdraw from v1 and deposit into v2 at their own initiative. Reineira Labs Limited–published v2 deployments coexist with v1 on-chain. Storage-layout discipline (ERC-7201

namespacing plus `uint256[50] private __gap` reserves) preserves cross-deployment interface stability so plugins compiled against v1 surfaces continue to recompile against v2 surfaces without source changes when the storage layout is intentionally preserved.

The canonical base contract `TestnetCoreBase` (legacy naming retained from §0 manifest) inherits `Initializable`, `OwnableUpgradeable` (for initialiser and owner semantics on operations, not for upgrade authorisation), `ReentrancyGuardUpgradeable`, and `ERC2771ContextUpgradeable` in a single composition and exposes a single initialiser. Every contract in the protocol extends this base.

Cross-package contracts import only via the `@reineira-os/shared` workspace path; copying contracts between packages is prohibited, and any contract used by more than one package belongs in `shared`. This is the import rule that keeps the storage invariants enforceable as the codebase grows.

3.5 Meta-Transaction Model

The protocol uses the ERC-2771 trusted-forwarder pattern for gasless user paths. A trusted forwarder address is fixed in the implementation constructor of every contract that extends the protocol base contract. The overridden sender and calldata accessors consult this address so that an upstream relayer can pay gas while the contract still sees the original signer as the caller.

Open Problem 3.1 (Account-Abstraction Convergence). The protocol commits to ERC-2771 for v1.0 gasless paths. Whether to add an ERC-4337 entry-point integration alongside (preserving 2771 for backwards compatibility) is unresolved. Selection criteria: (a) bundler-market depth on Arbitrum at chaos-net v1.0; (b) per-call gas overhead of 4337 vs 2771 for the confidential-redeem path; (c) wallet-side integration cost for design-partner builders. Decision target: prior to RSS v0.2 (Q4 2026).

3.6 Dual-Mode Operation — Public and Encrypted

The protocol runs in two modes with identical external interfaces and economic models. In *public mode*, all state variables are stored in plaintext and the protocol deploys on any EVM chain without a CoFHE coprocessor. In *encrypted mode*, sensitive state variables (escrow `owner`, `caller`, `amount`, `paidAmount`, `isRedeemed`; coverage `holder` and `coverageAmount`; risk scores; dispute verdicts; attached fee amounts) are stored as FHE ciphertexts and computations on those variables happen homomorphically.

The two modes share the same Solidity interfaces, function signatures, event shapes, and task and fee handlers. The only architectural difference is whether state flows through `euint64` / `eaddress` / `ebool` with `FHE.*` operations or through `uint64` / `address` / `bool` with native EVM operations.

A deployment in *public mode* runs on any EVM chain without waiting on FHE coprocessor availability, which is why the chaos-net launches in July 2026 in public mode on Arbitrum L2 (§9.7). Because every protocol contract is immutable (§11.1), public mode and encrypted mode are **separate deployments at separate contract addresses**, not two states of a single upgradeable proxy. Encrypted-mode contracts deploy at v1.0 mainnet hardening (Q4 2026) once Fhenix CoFHE becomes available on Arbitrum L2; the public-mode chaos-net deployment remains operational thereafter for any legacy escrows that did not migrate. Users transition from public-mode to encrypted-mode by withdrawing from the public-mode escrow and creating a new escrow in the encrypted-mode deployment; the migration is user-initiated and opt-in, never forced. Storage-layout discipline (ERC-7201 namespacing under `reineira.storage.<ContractName>, __gap[50]` reserves on every contract) preserves interface compatibility across deployment versions so plugins and integrations recompile without source changes. Plugins are mode-independent: `IConditionResolver` implementations return a plaintext boolean regardless of mode, and `IUnderwriterPolicy` implementations return encrypted values in encrypted mode and plaintext values in public mode via mode-scoped base classes, so the Gate ecosystem matures in public mode and transitions without rewrites.

The *silent-failure pattern* (the property that authorised and unauthorised redemption attempts produce computationally indistinguishable on-chain transcripts) has different strengths in the two modes. In

encrypted mode it is a cryptographic indistinguishability property under **LWE** hardness, formalised in §4.6. In public mode the same control-flow structure preserves uniform gas, identical events, and identical ciphertext-shape transfers; that is a weaker but still useful non-leakage property for observers without access to the chain's full call-data index. The crypto-level treatment is deferred to §4.6.

3.7 The Four Primitives — At a Glance

The protocol composes everything from four primitives: *Escrow* holds value in encrypted form until a release condition is met (treated in detail in §6); *Gate* is the pluggable release-condition interface implemented as any contract conforming to **IConditionResolver** (treated as part of the engine in §6 and as a reference-plugin gallery in §5); *Recourse* provides LP-backed coverage with encrypted risk scoring and policy-driven adjudication (§7); *Operators* form the permissionless workforce that executes the asynchronous work spanning cross-chain relay, agent calls, verdict submission, pool routing, and dispute execution (§8). The detailed cryptographic treatment of the encrypted-mode primitives is in §4. The Reineira Settlement Standard layer (the standard that any conforming implementation must satisfy) is specified in §5.

3.8 Deployment Substrate

Arbitrum L2 is the canonical deployment substrate and remains so across all roadmap horizons. The chaos-net target is Arbitrum One in Q3 2026 (public-mode mainnet, July 2026 chaos-net); a mainnet v1.0 lock follows in Q4 2026; Ethereum L1 hosting is planned as a second host; OP-Stack hosting is on the long-term roadmap. The protocol is not migrated off Arbitrum L2 — additional hosts run in parallel rather than as cutover targets. Third-party rollups self-deploy ReineiraOS via Reineira Atlas under the L3-owner maintenance model and reuse operator-routed liquidity over native rollup bridges; the full multi-chain treatment is in §9. The Fhenix CoFHE FHE substrate is a separately-administered service consumed by the protocol via the Solidity precompile interface examined in §4; the protocol does not operate the CoFHE coprocessor.

3.9 Conformance vs Reference Implementation

The protocol follows a *standards-fork pattern*: the standard layer (the Reineira Settlement Standard (RSS), specified in §5) is published free and OSI-aligned; the Reference Implementation (Definition 3.2; the contracts under **packages/**) ships under BUSL-1.1 with a Change Date of 2029-06-01 and a Change License of Apache 2.0 (§13). Other parties may implement the standard against their own contract code, and conformance is established by satisfying the RSS interface and the §5 conformance tests rather than by deploying the Reference Implementation. The standard-and-implementation split lets the standard layer compound builder-side adoption while preserving a distinct, versioned implementation layer.

§4 — Confidential Settlement Primitives

TFHE ciphertexts hold every sensitive settlement value the Reineira protocol records: escrow ownership and amount, coverage holder and risk score, per-pool liquidity, fee handles, and silent-failure intermediates. The substrate is fully homomorphic encryption over the torus, deployed through a coprocessor that holds the secret key and processes ciphertexts off the EVM execution path. This section defines the encrypted types (§4.2), homomorphic operation set (§4.3), access-list model (§4.4), silent-failure pattern (§4.5), and the invariants downstream chapters depend on (§4.6, §4.7).

4.1 Cryptographic substrate

The protocol's confidentiality guarantees rest on TFHE (Fully Homomorphic Encryption over the Torus) instantiated through Fhenix's coprocessor-based FHE (CoFHE), a service co-located with the EVM that holds the substrate secret key, evaluates homomorphic circuits on submitted ciphertexts, and returns

attested results to the calling contract. TFHE security reduces to the Learning With Errors (LWE) hardness assumption [Chillotti et al. 2020], [Regev 2005]; the CoFHE deployment exposes homomorphic operations as Solidity precompiles, and full decryption requires a coprocessor round-trip outside the EVM execution context (see §4.3) [Fhenix CoFHE 2024]. The trust model treats the coprocessor as a separately-administered execution counterparty: its operator quorum, key-management, and attestation surface are external to the EVM trust boundary and are revisited in §4.8 and §10.

Definition 4.1 (Encrypted Type). A ciphertext produced by the TFHE encryption of a plaintext value under the substrate-controlled secret key, opaque to the EVM execution context and accessible only via substrate-mediated operations and permissioned decryption.

4.2 Encrypted types

The TFHE substrate exposes a fixed set of ciphertext types, each carrying a plaintext-domain width and a permitted operation set. The protocol uses **euint64** as the canonical amount type (covering six-decimal stablecoin amounts up to ~18.4 T units), **eaddress** for encrypted account identifiers, and **ebool** for the encrypted-Boolean intermediate that drives the silent-failure pattern (§4.5). Client-side encryption produces input handles (**InEuint64**, **InEaddress**) that are verified at contract entry through the substrate’s input adapters; the storage layout follows the ERC-7201 namespacing discipline established in §3.4 (Definition 3.1), so encrypted handles live in versioned, gap-reserved struct slots like any other state variable.

Type	Bit width	Plaintext analogue	Permitted operations
ebool	1	bool	and, or, not, eq, ne, select
euint8	8	uint8	add, sub, mul, eq, ne, lt, lte, gt, gte, and, or, select
euint16	16	uint16	as above
euint32	32	uint32	as above
euint64	64	uint64	as above (canonical amount type)
euint128	128	uint128	as above (rate-limited; see §4.8)
euint256	256	uint256	as above (rate-limited; see §4.8)
eaddress	160	address	eq, ne, select only

Table 4.1: encrypted types and their permitted homomorphic operation classes.

The per-escrow record carries five ciphertext fields: encrypted owner and caller (each **eaddress**), encrypted target amount and cumulative paid amount (each **euint64**), and an encrypted redemption flag (**ebool**). No settlement-bearing field is stored in plaintext.

4.3 Homomorphic operations

The substrate exposes the operation set used by §6 and §7 as Solidity precompiles. Operation cost is asymmetric across two axes: the bit width of the operand types and the structural class of the operation. Bitwise Boolean operations and equality comparisons are cheapest; ordered comparisons and addition / subtraction are mid-tier; integer multiplication and the conditional-select gate (homomorphic select between two ciphertexts on an encrypted condition) sit in the upper-mid tier because they compose multiple programmable-bootstrap calls; full decryption or re-encryption requires a CoFHE round-trip and is the most expensive class because it is bound by network latency to the coprocessor rather than by on-chain gas.

Operation	Cost class
<code>FHE.and</code> , <code>FHE.or</code> , <code>FHE.not</code>	cheap
<code>FHE.eq</code> , <code>FHE.ne</code>	cheap
<code>FHE.add</code> , <code>FHE.sub</code>	medium
<code>FHE.lt</code> , <code>FHE.lte</code> , <code>FHE.gt</code> , <code>FHE.gte</code>	medium
<code>FHE.select</code>	medium-expensive
<code>FHE.mul</code>	expensive
<code>FHE.div</code>	expensive (open; see §4.8)
decrypt / re-encrypt	substrate round-trip (latency-bound)

Table 4.2: operation cost classes.

Property 4.1 (Computation Cost Asymmetry). For any homomorphic operation `op` and its plaintext analogue `op*`, evaluating `op` on ciphertexts of bit width `b` is monotonically more expensive than evaluating `op*` on plaintexts of the same bit width.

Proof sketch. TFHE evaluation of any non-trivial gate is dominated by programmable bootstrapping; the asymptotic complexity is $\tilde{O}(n)$ per bootstrap where $n = \theta(\lambda)$ is the LWE dimension and λ the security parameter, against $O(1)$ EVM word-level cost for the plaintext analogue [Chillotti et al. 2020] §V. The asymmetry is structural and cannot be eliminated by implementation tuning; it bounds the design space for any silent-failure or conditional-select branch (§4.5). $\square$

4.4 Permission model

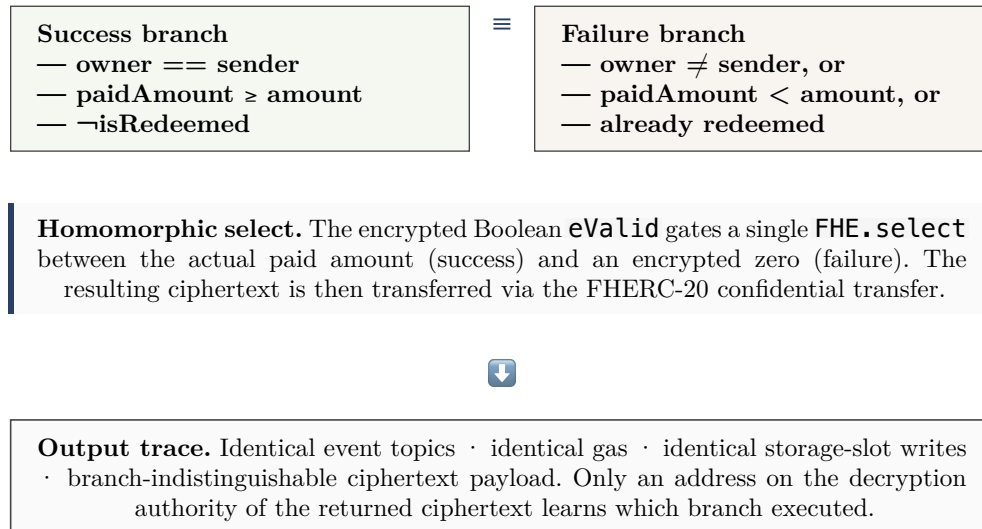
Ciphertexts are addressable on-chain by handle, but the substrate enforces that the plaintext is recoverable only by addresses on the ciphertext’s access list. Access-list entries are themselves created and revoked through contract calls: a persistent grant installs an address-keyed permission on a handle; a transient variant installs the same grant scoped to the current transaction and is revoked by the substrate at transaction end; and `FHE.allowThis()` grants the executing contract permission to read its own encrypted state — which is required to write the handle back into storage. The access list is metadata enforced by the coprocessor at decryption time, not by EVM code; revoking access is therefore a substrate-side state change with the same latency profile as decryption.

Selective disclosure to an auditor or counterparty applies the same primitive to a specific recipient: granting `allow` to a designated `complianceOwner` role permits that role — and no other observer — to request decryption, leaving the on-chain ciphertext untouched and other observers’ view unchanged. This is the mechanism §11 builds on for audit and supervised-disclosure flows.

Definition 4.2 (Decryption Authority). The set of EVM addresses authorised, by a ciphertext’s substrate-side access-list policy, to request decryption of that ciphertext.

4.5 The silent-failure pattern

The silent-failure pattern is the protocol’s load-bearing confidentiality design. On every state-mutating call whose outcome would otherwise leak through a revert, the contract evaluates the decision predicate homomorphically and combines the per-condition `ebool` handles into a single encrypted Boolean, `eValid`. It then routes the state change through a homomorphic select between the intended effect and a no-op effect, gated on `eValid`, so that the on-chain trace is identical across the success and failure branches. The redemption path in the Escrow engine is the canonical instance: ownership, payment sufficiency, and non-duplication are computed as encrypted Booleans, combined under homomorphic AND, and used to select between the actual paid amount and an encrypted zero before the encrypted transfer executes. Fig. 4.1 shows the indistinguishability construction.



*Listing 2: **Fig. 4.1 — Silent-failure indistinguishability construction.** Two semantically distinct branches share an identical EVM trace under TFHE IND-CPA. Proposition 4.3 formalises the indistinguishability bound; the bound is conditional on the adversary not holding decryption authority on any intermediate or output ciphertext.*

Reverting on a decryption-revealed condition would publish the evaluation of that condition to every chain observer, defeating the confidentiality goal. Routing both branches through the same encrypted-transfer call preserves indistinguishability at the EVM trace layer: emitted events, storage-slot identities, and ciphertext shape are identical by construction; gas charged is identical under the warm-storage and equal-cost-precompile-path assumption stated in §4.3. Only an address on the decryption authority of the returned ciphertext learns the branch outcome.

Proposition 4.3 (Silent-Failure Indistinguishability). Let `T_succ` and `T_fail` be the EVM transaction traces produced by the success and failure branches of a silent-failure-pattern call, restricted to events, gas, storage diffs, and ciphertext payloads. For any probabilistic polynomial-time adversary **A** that does not hold a key on the decryption authority of any intermediate or output ciphertext, the advantage $|\Pr[\mathbf{A}(\mathbf{T}_{\text{succ}}) = 1] - \Pr[\mathbf{A}(\mathbf{T}_{\text{fail}}) = 1]|$ is negligible in the substrate security parameter, conditional on TFHE IND-CPA security.

Proof sketch. Construction-side: the two branches differ only in the plaintext underlying the encrypted-amount payload of one transfer call, and in the plaintexts underlying any intermediate `ebool` and `euint` handles touched by `FHE.select`. By TFHE IND-CPA [Chillotti et al. 2020], ciphertexts encrypting distinct plaintexts of the same type are computationally indistinguishable to any party without decryption authority. Event payloads, gas, and storage-slot identities are constructed to be branch-independent. A distinguisher reduces to a TFHE IND-CPA distinguisher on the affected ciphertext set; the advantage is the underlying scheme’s IND-CPA advantage, which is negligible under LWE [Chillotti et al. 2020] §IV. The bound is conditional: an adversary on the decryption authority can request decryption of any reachable intermediate, and the branches become trivially distinguishable. □

4.6 Observable vs confidential state

Some state must remain plaintext for the protocol to function: event topics that indexers consume, gas-payer addresses that the EVM bills, operator identities that the slashing pipeline (§8) holds collateral against, and sequential escrow IDs that downstream cross-chain handlers route on. The split between what is observable and what is confidential is intentional and is encoded in the contract surface, not left to deployment-time configuration. The table below summarises the encrypted-mode boundary. In the protocol’s public mode (§3.6), items in the right column become observable; the access-control and formal-property surfaces are unchanged.

Quantity	Observable (plaintext)	Confidential (ciphertext)
Escrow existence	escrow ID (sequential)	owner, caller, amount, paid amount, redemption
Funding	funder address (when not intermediated)	funded amount, fee, premium
Redemption	event emission, sender	recipient, amount, success / failure outcome
Coverage	pool address, policy address	holder, coverage amount, risk score
Dispute	dispute event, caller	verdict, payout amount
Operator relay	operator identity (required for slashing)	which operator relayed which escrow funding
Bridge boundary (CCTP / LayerZero OFT)	wrapped amount at burn / mint	post-wrap state at the cUSDC / cUSDT layer

Table 4.3: observable vs confidential quantities under encrypted mode.

Definition 4.4 (Public Skeleton). The set of quantities the protocol deliberately exposes in plaintext to support indexing, gas accounting, slashing collateral attribution, and bridge-boundary token-flow attestation, enumerated in the left column of Table 4.3.

4.7 Encrypted-state invariants

Two invariants on encrypted-state behaviour are load-bearing for §6 (escrow accounting) and §7 (coverage accounting).

Proposition 4.5 (Balance Conservation under FHE). Let $b_1, \dots, b_n$ be plaintext stablecoin balances held as ciphertexts $\text{Enc}(b_1), \dots, \text{Enc}(b_n)$ in the **ConfidentialUSDC / ConfidentialUSDT** storage. For any sequence of **FHE.add** and **FHE.sub** operations producing a ciphertext S , decryption of S yields $\sum \pm b_i \bmod p$, where p is the substrate’s plaintext modulus for the **euint64** type.

Proof sketch. The TFHE encryption of unsigned integers is additively homomorphic over the plaintext ring $\mathbb{Z}/p\mathbb{Z}$ for the type’s plaintext modulus [Chillotti et al. 2020] §III; the substrate’s **FHE.add** and **FHE.sub** precompiles compose this property without intermediate decryption. Modular wrap is bounded by the protocol’s amount-domain check: **euint64** supports values up to $\sim 1.8 \times 10^{19}$, which dominates the worst-case sum across the maximum batch size (**MAX_BATCH_SIZE = 20** in **ConfidentialEscrow**) for any stablecoin denominated in six decimals. $\square$

Proposition 4.6 (No Ciphertext Forge). No EVM-side party can construct a ciphertext that decrypts to a chosen plaintext under the substrate’s secret key, conditional on (a) TFHE IND-CPA security and (b) the Fhenix CoFHE input-attestation layer: ciphertexts entering the EVM must be authenticated as substrate-issued. The proposition does not bind ciphertext non-malleability under TFHE’s additive homomorphism — an adversary holding $\text{Enc}(x)$ can always compute $\text{Enc}(x+1)$, but cannot produce a ciphertext whose decryption is uncorrelated to any known input and equals a chosen target value.

Proof sketch. A successful forge in the sense above would produce a ciphertext whose decryption is a function chosen by the adversary independent of any substrate-issued input, contradicting TFHE IND-CPA [Chillotti et al. 2020] §IV: an IND-CPA distinguisher can be built from any forging oracle by querying the oracle on candidate plaintexts and checking the recovered ciphertext against the challenge. The contract-side **asEuInt** / **asEAddress** adapters additionally bind input handles to the submitting sender at the substrate (**the substrate input-attestation layer**), so a structurally valid ciphertext cannot be reattributed to another caller without breaking the substrate’s input-attestation layer [Fhenix CoFHE 2024]. The conjunction (a) $\wedge$ (b) is therefore load-bearing; IND-CPA alone is insufficient. $\square$

4.8 Limits and open questions

The substrate is not a general-purpose confidential computer. Four limits constrain the design space for the chapters that follow. There is no general-purpose branching on ciphertext: every conditional must be

expressed as a homomorphic select between two ciphertexts on an encrypted condition, which evaluates both branches and selects between the resulting ciphertexts, doubling the worst-case cost. There is no efficient integer division: `FHE.div` is supported but expensive enough that the protocol routes premium and payout pricing through precomputed inverses where feasible (an open optimisation problem in TFHE per [Chillotti et al. 2020]). Throughput is subject to substrate limits per [Fhenix CoFHE 2024] rather than primarily bounded by EVM gas, which shapes the batch-size cap in the Escrow batch-redemption path (`MAX_BATCH_SIZE = 20`); the precise envelope is pending substrate-side publication. The CoFHE coprocessor is a single-substrate trust counterparty: liveness, key management, and attestation rest on its Operator quorum, not on the host chain’s validator set.

The coprocessor’s single-substrate trust model is the largest open question in this chapter.

Open Problem 4.1 (CoFHE Decentralization). The Fhenix CoFHE deployment is a single-substrate coprocessor under a single administrative quorum. Decentralization of the FHE service is a precondition for reducing the trust assumption to the EVM validator set. Selection criteria: (a) threshold-FHE distribution of the secret key across a permissioned committee with formal threshold-decryption proofs; (b) MPC-distributed key generation and operation with a published security model against malicious minorities; (c) hardware-attested operator quorum with remote attestation of the FHE evaluation enclave and on-chain attestation verification. Forward reference: §8 specifies the slashing-secured execution context that any decentralized FHE operator set would inherit.

§5 — Reineira Settlement Standard (RSS)

5.1 Standard versus Reference Implementation

The Reineira Settlement Standard (RSS) is the abstract conformance specification for confidential settlement across the four primitives enumerated in §3 (Escrow, Gate, Recourse, and Operator). The contracts under `packages/` are one *Reference Implementation* (Definition 3.2), maintained by Reineira Labs Limited (RAK DAO Free Zone); other parties may publish conforming implementations against their own contract code, with conformance established by the criteria in §5.4 and the test surface in §5.7.

Publishing the standard as a layer separate from the implementation is the protocol’s load-bearing commitment, detailed in §5.9. The split presents any competing settlement contract with two choices: implement RSS (legitimising the standard and inheriting its plugin ecosystem) or fork it (fragmenting the addressable plugin and Operator surface relative to the conforming majority). The protocol charges nothing; the open standard and the Reference Implementation are complementary rather than substitutable.

5.2 Scope

RSS covers (a) the four-primitive interface surface — Escrow, Gate, Recourse, and Operator — and the canonical interfaces in §5.5; (b) the encrypted-type contract conventions of Definition 4.1 and the silent-failure pattern of Proposition 4.3; (c) the storage conventions of Definition 3.1 (ERC-7201 namespacing under `reineira.storage.<ContractName>`, `__gap[50]` reserves) and ERC-2771 meta-transaction support; and (d) the cross-chain transport-binding contract by which any bridge funnels into the Escrow funding sink (`fund From`, §6). The v1.0 Reference Implementation is deployed under the immutable-contract posture of §11.1; RSS does not mandate upgradeability, and conforming implementations may be upgradeable, immutable, or hybrid at the implementer’s choice — the conformance criteria below test interface and behaviour, not deployment-time mutability. RSS does not cover the Software Vendor’s governance posture (§11), the choice of execution stack for third-party self-hosted deployments (§9.10), or the builder stack (Reineira Atlas, Reineira Code) used to author conforming plugins (§1.1).

5.3 Versioning

RSS is semver-versioned per the §0 manifest: **v0.1** ships at testnet (June 2026), **v0.2** at chaos-net (Q4 2026) with the first five Reineira Improvement Proposals (RIPs), and **v1.0** at mainnet stabilisation (Q3 / Q4 2027). Minor-version bumps are interface-compatible additions (new optional interfaces, new optional events, new optional fields in tagged structs); major-version bumps are breaking and require a coordinated reference-implementation upgrade or a **v(N).0** adapter contract that translates the deprecated surface. The RIP process governs the inclusion of any change in a versioned release.

5.4 Conformance Criteria

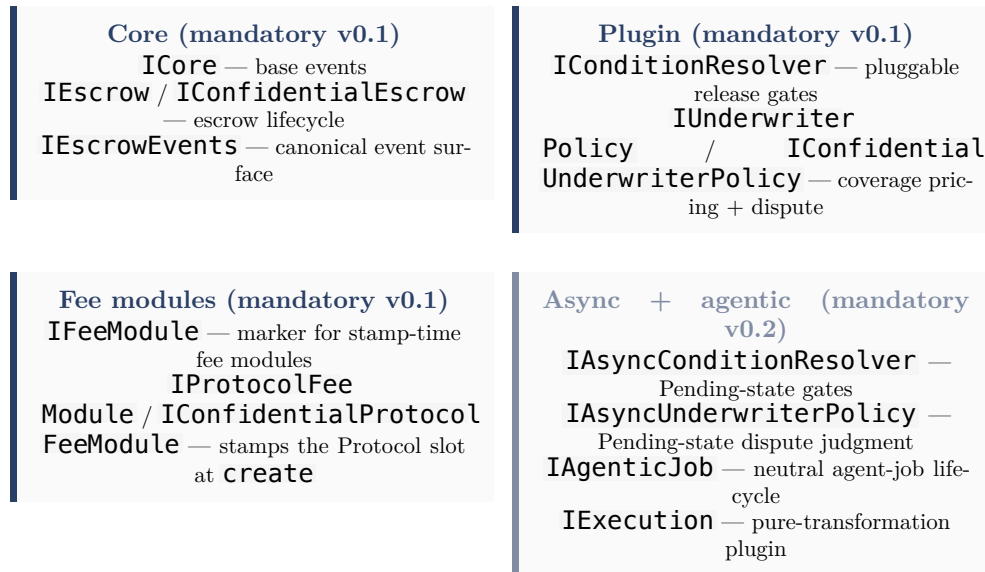
A deployment is RSS-conformant if and only if it satisfies clauses (a)–(f) below. The publication ships the conformance test suite of §5.7; passing that suite is necessary for conformance, but the criteria here are the authoritative definition.

- (a) **Interface coverage.** Implements every interface in the standard’s core and plugin interface surfaces declared mandatory for the deployment’s mode (public or encrypted), per the canonical surface enumerated in §5.5.
- (b) **Encrypted state.** In encrypted mode, stores every owner identifier, payment amount, paid-amount accumulator, and condition-derived intermediate as an encrypted type per Definition 4.1; in public mode, runs the same contracts with plaintext types per the dual-mode design of §2.4 of the technical reference, preserving the same interface shape so plugins and integrations remain binary-compatible across modes.
- (c) **Silent-failure pattern.** Every redemption path that evaluates an encrypted condition conforms to Proposition 4.3 — the success and failure transcripts (events, gas, storage diffs, ciphertext payloads) are computationally indistinguishable to a PPT adversary without decryption authority on the relevant intermediates.
- (d) **Namespaced storage layout.** All contracts use ERC-7201 namespaced storage per Definition 3.1, inherit from the protocol base contract, and carry a `uint256[50] private __gap` reserve for cross-deployment-version forward compatibility. Implementations that elect an upgradeable deployment additionally follow the UUPS proxy pattern of [EIP-1822] over the [EIP-1967] proxy slot layout; the v1.0 reference implementation is immutable (§11.1) and does not deploy under a proxy.
- (e) **Meta-transaction support.** Sponsored user paths route through an [ERC-2771] trusted forwarder so that `msg.sender` is recovered from the appended forwarder context, not from the originating EOA, allowing gasless flows without compromising authorisation.
- (f) **Canonical events.** Emits the events declared in **ICore** and **IEscrowEvents** (and the analogous canonical event sets for the recourse and operator surfaces) so that cross-implementation indexers, slashing oracles, and bridge-receipt monitors can observe state transitions without depending on implementation-specific event schemas.

Definition 5.1 (RSS-Conformant Implementation). A deployment of contracts that satisfies clauses (a)–(f) of §5.4 and passes the conformance test suite of §5.7 at the version of RSS it claims to implement.

5.5 Core Interfaces

The mandatory interface surface as of RSS v0.1 is anchored to source. Fig. 5.1 groups the interfaces by responsibility tier. The async variants (**IAsyncConditionResolver**, **IAsyncUnderwriterPolicy**), the agentic adapter interface (**IAgenticJob**), and the transformation interface (**IExecution**) are specified in the technical reference and become mandatory at RSS v0.2 (Q4 2026); see the Implementation Note below.



Listing 3: **Fig. 5.1 — Reineira Settlement Standard interface surface.** Three tiers are mandatory at v0.1 (red): core lifecycle, plugin extension points, and the protocol-fee module. The fourth tier (light red) becomes mandatory at v0.2 (Q4 2026) and covers async resolution semantics and the agentic-composition layer.

5.5.1 IConditionResolver

The pluggable release-Gate interface inherits [ERC-165] for interface detection. It registers per-Escrow Gate configuration at creation and reports whether the Gate has cleared at each redemption.

Function	Purpose	Reverts iff
isConditionMet	view-only; returns the gate state for the given escrow id	never (view)
onConditionSet	stores per-escrow configuration at gate-binding time	caller is not the registered escrow contract; init data malformed
getConditionFee	view-only; returns the resolver-author's fee in basis points and the recipient address (consumed by the escrow at condition-set to stamp the Condition fee slot)	never (view)

5.5.2 IAsyncConditionResolver

Extends IConditionResolver with the four-state machine **Unresolved** → **Pending** → **Resolved** | **Rejected**, for gates whose verdict requires off-chain work (quorum attestation, oracle fetch, human review). Release blocks on **Resolved**.

Function	Purpose	Reverts iff
requestResolution	initiates the off-chain verdict pipeline for a given escrow id	status is not Unresolved
status	view-only; returns one of Unresolved , Pending , Resolved , Rejected	never (view)

5.5.3 IUnderwriterPolicy and IConfidentialUnderwriterPolicy

Two sibling interfaces govern coverage pricing and dispute adjudication: a plaintext **IUnderwriterPolicy** for public-mode deployments and an encrypted **IConfidentialUnderwriterPolicy** whose return values are FHE ciphertexts so competing underwriters cannot reverse-engineer pricing or verdict reasoning from on-chain traces.

IUnderwriterPolicy (plaintext variant):

Function	Purpose
<code>evaluateRisk</code>	returns the plaintext risk score used to compute premium
<code>judge</code>	returns the plaintext verdict on a filed dispute

`IConfidentialUnderwriterPolicy` (FHE variant):

Function	Purpose
<code>onPolicySet</code>	registers per-coverage configuration at coverage creation
<code>evaluateRisk</code>	returns the encrypted risk score used to compute premium
<code>judge</code>	returns the encrypted verdict; if true under decryption and accepted, the pool pays out

5.5.4 **`IAsyncUnderwriterPolicy`**

Extends `IUnderwriterPolicy` for policies that delegate judgment to a quorum or off-chain arbiter.

Function	Purpose
<code>requestJudge</code>	initiates the off-chain judgment pipeline for a given coverage id
<code>verdict</code>	view-only; returns the encrypted verdict and a settled flag

5.5.5 **`IEscrow`**

The Escrow lifecycle interface supports creation, funding, redemption, queries, coverage-manager authorisation, and protocol-fee-module registration. The encrypted variant `IConfidentialEscrow` substitutes encrypted types (`eaddress`, `euint64`) for the plaintext counterparts shown here; both share function names and ordering so composition contracts and adapters compile against the same abstract surface.

Function	Purpose	Event emitted
<code>create</code>	opens a new escrow with owner, target amount, and optional resolver	<code>Escrow Created</code>
<code>fund</code>	records a payment into the escrow	<code>Escrow Funded</code>
<code>redeem</code>	settles a single escrow under the silent-failure pattern	<code>Escrow Redeemed</code>
<code>redeemMultiple</code>	batch settlement with branchless silent skip per id	<code>Escrow BatchRedeemed</code>
<code>setCoverageManager</code>	binds the coverage-manager address authorised to stamp the <code>Underwriter</code> fee slot	<code>Coverage Manager Set</code>
<code>setUnderwriterFee</code>	called by the coverage manager only; stamps the underwriter fee slot with <code>(bps, recipient)</code> , branchless-zeroed when holder $\notin$ {owner, caller}	<code>FeeStamped</code>
<code>setFeeModule</code>	registers a fee module at a fixed slot (currently only slot 0, <code>FeeModule Protocol</code> , is populated)	<code>FeeModule Registered</code>
<code>exists / getOwner / getAmount / getPaid Amount / getRedeemedStatus / total / getCaller</code>	view accessors	none

5.5.6 **`IFeeModule`, `IProtocolFeeModule`, `IConfidentialProtocolFeeModule`**

Pluggable fee-module surface called only at lifecycle stamp time (create, condition-set, coverage-purchase) — never on the redeem hot path. The base **IFeeModule** is the marker interface; **IProtocolFeeModule** (and its FHE variant) returns the per-escrow protocol-fee parameters consumed by the escrow at **create** to stamp slot 0. The module’s bps value is governance-settable within an immutable bytecode ceiling (**MAX_PROTOCOL_FEE_BPS = 50**); raising the ceiling requires deploying a new immutable fee-module at a new address per §11.1.

Function	Purpose
IProtocolFeeModule.getProtocolFee	view-only; returns (uint16 bps , address recipient) for slot 0
IConfidentialProtocolFeeModule.getProtocolFee	view-only; returns (euint64 bps , address recipient) — FHE variant
setProtocolFee	governance-only; updates the module’s bps within the immutable bytecode cap

5.5.7 IAgenticJob

The neutral agent-job lifecycle interface implemented by adapters (**AgentInvocationAdapter**, **EIP8183Adapter**) so intent semantics are portable across execution frameworks. Phase enum: **Open**, **Accepted**, **Submitted**, **Evaluated**, **Settled**, **Refunded**.

Function	Purpose
invoke	opens a new agent-job for a given agent and input bundle
accept / submit / evaluate / settle / refund	phase transitions on an open job
view	view-only; returns the current job state

5.5.8 IExecution

The pure transformation / attested-execution plugin sibling to **IConditionResolver** and **IUnderwriterPolicy**, invoked by adapter pipelines to transform state or attest to off-chain computation.

Function	Purpose
executionType	view-only; returns the execution-type discriminator
execute	runs the pure transformation on a job’s input and returns the output

Implementation Note (Plugin Surface, as of June 2026). The plain (non-FHE) **IUnderwriterPolicy** and the encrypted **IConfidentialUnderwriterPolicy** are shipped to disk under the standard’s plugin interface surface. The async variants (**IAsyncConditionResolver**, **IAsyncUnderwriterPolicy**), the agentic adapter interface (**IAgenticJob**), and the transformation interface (**IExecution**) are specified in the technical reference and slated for the RSS v0.2 release alongside the first RIPs at chaos-net (Q4 2026). Until then, conformance clauses §5.4(a)–(f) bind only over the v0.1 mandatory subset.

5.6 Composability via Plugins

Definition 5.2 (Plugin). A contract implementing one of the extension interfaces (**IConditionResolver**, **IUnderwriterPolicy**, **IConfidentialUnderwriterPolicy**, **IAgenticJob**, **IExecution**) that may be composed with the reference implementation’s core contracts without modification of those core contracts.

The four-primitive surface lets third-party plugins provide custom release Gates (zkTLS proofs from Reclaim Protocol or zkPass, Chainlink price feeds, UMA Optimistic Oracle outcomes, M-of-N multisig, time locks, and AND/OR composites) and custom underwriter policies (parametric flight delay, gas spike, oracle failure, and stablecoin depeg) without forking the protocol core. The **plugin-examples/** workspace ships **Simple**

Policy (a fixed-risk-score `IConfidentialUnderwriterPolicy`) as a reference template; plugin authors compile against the published interfaces in `@reineira-os/shared` and deploy independently.

Builder-side mechanisation. The Reineira Code repository (`reineira-code`, §1.1) supplies an AI-assisted authoring environment that generates RSS-conformant resolver and policy contracts from natural-language specifications. Slash commands (`/new-resolver`, `/new-policy`, `/scaffold-test`, `/audit`, `/integrate`) emit a Hardhat project pre-wired with cofhejs FHE test fixtures, ERC-165 declarations against the canonical interface, and a deployment script that registers the plugin with the relevant registry. Operationally, a plugin author can move from “I need a resolver that releases when a Stripe charge captures” to a passing test suite within a working session, and from there to a deployed contract on testnet without writing FHE plumbing or ERC-165 boilerplate by hand. Reineira Atlas (the venture-side counterpart) wires the resulting contract into a working application (`platform-modules/backend`, `platform-modules/app`) via the `/integrate` skill. Neither tool is a normative part of RSS — plugin authors may work in any environment — but the builder stack is the recommended entry point for Policy Builders, Pool Underwriters, LP Stakers, and Operators.

5.7 Conformance Test Surface

The RSS publication ships the conformance test suite under `packages/<pkg>/test/`. The current Foundry test surface contains approximately 36 unit-test contracts in `escrow/test/`, `insurance/test/`, `orchestration/test/`, and `tokens/test/`, comprising FHE and plaintext unit tests, integration tests (`EscrowSettlement{,FHE}.t.sol`, `InsuranceFlow{,Confidential}.t.sol`), and invariant tests (`EscrowInvariants.t.sol`, `RecoursePoolInvariants.t.sol`, `FeeManagerInvariants.t.sol`). The RSS v0.1 conformance suite freezes a subset as the binding battery for §5.4 conformance; passing this battery is necessary for a deployment to claim RSS-Conformant Implementation status under Definition 5.1. The RSS v1.0 suite extends to the async, agentic, and execution surfaces.

Open Problem 5.1 (Formal Conformance Suite Release). The exact set of test files that constitute the binding RSS v0.1 conformance suite is unselected as of v1.0 publication. Selection criteria: (a) every interface function in §5.5.1, §5.5.3, and §5.5.5 is exercised by at least one positive and one negative test; (b) the silent-failure indistinguishability of Proposition 4.3 is checked by a trace comparison on authorised and unauthorised redemption paths; (c) the ERC-7201 storage namespace of every upgradeable contract is checked against the namespace template; (d) the conformance suite is platform-agnostic, runnable against any RSS-conformant deployment by pointing Foundry at the deployed addresses. Decision target: RSS v0.2 (Q4 2026), bundled with the first RIPs.

5.8 ERC and EIP Alignment

The reference implementation aligns with the following external standards. Appendix B carries the full per-contract compliance matrix; this table is the executive summary.

External standard	Surface	Reineira implementing artifact		Notes
[ERC-165]	Interface detection	IConditionResolver , IUnderwriterPolicy , Policy	IConfidential IUnderwriter	Mandatory on every plugin so registries can validate the surface before binding
[ERC-7201]	Name-spaced storage	Every UUPS-upgradeable contract under packages/		Namespace template reineira.storage.<ContractName>
[ERC-2771]	Meta-transactions	the protocol base contract and every derived contract		Trusted-forwarder context decoding for gasless paths
[EIP-1822] / [EIP-1967]	UUPS proxy + proxy slot	All upgradeable contracts		Pairs with the __gap[50] reserve and ERC-7201 namespacing
[ERC-7984]	Confidential token	ConfidentialUSDC , ConfidentialUSDT , generic-wrapper factory		Immutable, non-upgradeable by design

5.9 Protocol-level consequences of the standards-fork pattern

An RSS-conformant alternative implementation reduces vendor lock-in for protocol users. Any deployment that satisfies §5.4 and passes §5.7 can host the same plugins, accept the same bridge handlers, be observed by the same indexers and slashing oracles, and present the same SDK surface — independently of who maintains its contracts. The protocol charges nothing; the Reference Implementation competes on operational quality and audit history, not on incompatibility.

The reference implementation maintained by Reineira Labs Limited (RAK DAO Free Zone) ships under BUSL-1.1 with a Change Date of 2029-06-01 and a Change License of Apache 2.0 (§13). The RSS specification itself — the interfaces, the conformance criteria, the RIP process — is published under an OSI-aligned licence and is unencumbered for any party to implement against. The two licences address different layers and are designed to be coherent jointly; neither permits the other to be dropped without forfeiting the standards-fork commitment.

§6 — Escrow Engine

6.1 Role in the four-primitive system

The Escrow engine is the conditional-release primitive among the four enumerated in §3.7 (Escrow / Gate / Recourse / Operators). It holds value, or the right to value, until a pluggable resolver signals that the release condition has been met. The confidential variant **ConfidentialEscrow** ships as the Reference Implementation (Definition 3.2) of this primitive, with state expressed in the encrypted types of Definition 4.1 and a redemption path that satisfies the silent-failure indistinguishability of Proposition 4.3. A non-confidential sibling, **Escrow**, ships in the same package for the dual-mode deployments described in §3.6.

6.2 State Shape

The per-escrow record is held in the **Escrow** struct. Field types and visibility are summarised in Tab. 6.1.

Tab. 6.1. Per-Escrow state. Existence is tracked by integer comparison against the Escrow counter rather than a per-record Boolean. Per-Escrow fee state is held in the parallel four-slot fee array described below.

Field	Type	Visi- bility	Description
owner	eaddress	ci- pher- text	Address entitled to redeem
caller	eaddress	ci- pher- text	Address that called create , retained distinctly from owner for fee-authorisation checks
amount	euint64	ci- pher- text	Target funding amount
paid Amount	euint64	ci- pher- text	Cumulative funded amount
isRedeemed	ebool	ci- pher- text	Redemption flag

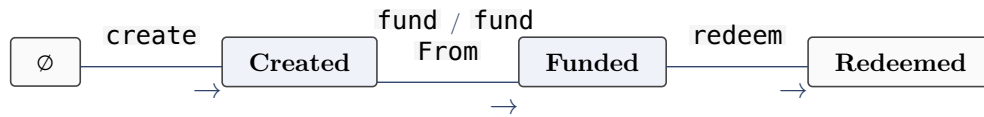
Per-escrow fee data lives in a fixed four-slot array of **Fee** records, one slot per fee kind in a **FeeKind** enum: **Protocol** (slot 0, reserved/unused — the protocol charges no fee and holds no treasury), **Condition** (slot 1, paid to the resolver author), **Underwriter** (slot 2, paid to the recourse pool backing the escrow), and **Reserved** (slot 3, currently unused; tracked for the creator-royalty extension). Each **Fee** record carries **bps** (basis points; encrypted as **euint64** in the confidential variant and plaintext **uint16** in the public variant), a plaintext **recipient**, and a plaintext **set** presence flag. The escrow tracks a per-escrow cumulative sum of stamped bps; the protocol enforces the invariant that the sum of stamped fees never exceeds **MAX_TOTAL_BPS = 10000** (100 %). Records are indexed by a monotonically increasing **uint256** escrow counter; presence of an escrow is the public predicate **escrowId < counter**, used uniformly by the contract’s **exists** view and by the existence-check helper.

Pluggable-condition state lives in a separate ERC-7201 namespace declared by the **EscrowCondition** base. The mapping **resolvers: escrowId → IConditionResolver** is held at the storage slot **0xddbe0eff506238e35addb19b3c2d99c00660b3894c8f79a6042996acddf4e100**, computed from the legacy namespace **privara.storage.ConfidentialEscrowCondition** and preserved verbatim across deployment versions so plugins and adapters compiled against the testnet bytecode resolve against the same slot on mainnet.

Implementation Note (Storage Namespace, as of May 2026). The §3.4 storage convention specifies the **reineira.storage.<ContractName>** namespace prefix. **EscrowCondition** keeps the inherited slot under the legacy **privara.storage.ConfidentialEscrowCondition** namespace to preserve cross-deployment-version interface compatibility with the testnet deployment. Migration to the canonical prefix would require a new immutable deployment at a new address per the §11.1 versioning pattern; the legacy slot is retained at v1.0 for plugin-author convenience.

6.3 Lifecycle

Fig. 6.1 sketches the state machine. Tab. 6.2 details each transition. Conditions are evaluated through the resolver mapping of §6.5; the silent-failure path of Proposition 4.3 is taken when the encrypted predicate **owner == sender ∧ paidAmount ≥ amount ∧ ¬isRedeemed** decrypts to false.



🕒 **redeem** self-loop on *Funded* when the encrypted predicate decrypts to false (silent-failure transcript; Proposition 4.3)

🔑 **setUnderwriterFee** stamps the Underwriter fee slot at coverage-purchase time (no state transition; §6.4.6)

🔑 **redeemMultiple** extends **redeem** to up to 20 escrows per call (§6.4.5)

Listing 4: Fig. 6.1 — Confidential escrow state machine. Three principal states (*Created*, *Funded*, *Redeemed*) connected by three principal transitions. Fee stamping happens at **create** (the Condition slot only — the Protocol slot is reserved and never stamped), at coverage purchase (Underwriter slot), and never on **redeem**. The silent-failure self-loop is computationally indistinguishable from a successful **redeem** to non-key-holders.

Tab. 6.2. Lifecycle of a confidential Escrow. Source-line ranges in the rightmost column refer to the Reference Implementation’s Escrow contract.

From → To	Call	Pre-condition	Post-condition	Event
∅ → Created	create	counter available; resolver code-bearing if non-zero	record stored; the Protocol slot is left unstamped (the protocol charges nothing); the Condition slot is stamped from the resolver if a resolver is bound	EscrowCreated , ConditionSet , FeeStamped (per stamped slot)
Created → Funded	fund	escrowId < counter	paidAmount += actualPayment	EscrowFunded
Created → Funded	fund From	same; caller holds euInt64 handle with transient allowance	as above	EscrowFunded
any Underwriter-stamped	setUnderwriterFee	caller is the bound coverage manager; the slot is unset; the stamped bps does not exceed remaining budget	Underwriter slot (slot 2) stamped with (bps , recipient), branchless-zeroed when holder $\notin$ {owner, caller}	FeeStamped
Funded → Redeemed	redeem	resolver, if bound, returns true	proportional distribution of paidAmount × bps / 10000 to each set slot’s recipient, net to owner	EscrowRedeemed , FeeDistributed (per non-zero slot)
Funded → Funded (silent)	redeem	resolver returns true but encrypted predicate false	zero-amount encrypted transfer; redemption flag unchanged in plaintext-observable way	EscrowRedeemed
{Created, Funded} → Redeemed (batch)	redeem 1 ≤ length ≤ 20; Multiple	per-id existence and resolver gate; bad ids skipped	per-id branchless redemption summed into one transfer	EscrowBatch Redeemed

The only **revert** paths on the funding/redemption hot path are existence (the existence check), batch-size (the batch-size check), and a resolver-gated **ConditionNotMet** revert exposed by the check-condition extension hook. All other failure cases — wrong owner, under-funded, already-redeemed — are absorbed into the encrypted **canRedeem** predicate and resolve to a zero-amount transfer with no public state diff.

6.4 Canonical Functions

6.4.1 create

The **create** operation opens a new Escrow with an encrypted owner and an encrypted target amount, optionally binding a condition resolver atomically. The contract assigns the next sequential Escrow ID; stores the encrypted owner, caller, amount, a zero-initialised **paidAmount**, and a false-initialised redemption flag; and grants the contract permission to read all five ciphertexts, with a persistent grant to the caller. At creation, the **Protocol** fee slot (slot 0 of the module registry) is left unstamped — the protocol charges no fee and holds no treasury — so the sum-of-stamped-bps invariant covers only the third-party fee slots. When a coverage manager is bound, it inherits visibility on the amount, owner, and caller for the later underwriter-fee stamp. When a non-null resolver is supplied, the operation rejects non-code addresses, persists the resolver binding, invokes the resolver’s on-condition-set hook with the supplied init data, and immediately stamps the **Condition** fee slot by reading the resolver’s **getConditionFee** view. It emits **EscrowCreated**, **FeeStamped** (for the Condition slot when a resolver is bound), and **ConditionSet** when a resolver is bound.

6.4.2 fund

The **fund** operation records a confidential transfer into the Escrow contract and accumulates the observed balance delta in the Escrow’s encrypted **paidAmount**. The contract reads its pre-transfer encrypted balance, grants the payment token a transient FHE allowance on the input ciphertext, invokes the confidential-transfer-from path on the payment token, derives the actual payment by homomorphically subtracting the pre-transfer balance from the post-transfer balance, and accumulates the result in **paidAmount** by homomorphic addition, reasserting the contract’s read permission on the new handle. It reverts on the existence check or any payment-token transfer failure and emits **EscrowFunded**.

6.4.3 fundFrom

The **fundFrom** operation has the same balance-delta accounting as **fund** but consumes a pre-existing encrypted-amount handle rather than an input ciphertext, used by cross-chain receivers (§6.4.7) and by composition contracts that fund on behalf of a third party. The input handle must already carry transient FHE allowance for the payment token; absent that grant, the inner confidential-transfer-from reverts. Reverts only on the existence check. Emits **EscrowFunded**.

6.4.4 redeem

The **redeem** operation settles a single Escrow under the silent-failure pattern of Proposition 4.3. The contract computes a single encrypted Boolean, **canRedeem** — the conjunction of owner equality, payment sufficiency, and non-duplication — and uses it to homomorphically select between the paid amount and an encrypted zero. The redemption flag is updated to the disjunction of its prior value and the same predicate. Fee distribution iterates the four-slot fee array (§6.2): for each **set** slot, the contract computes the per-slot fee as the homomorphic product of the redemption amount and the stamped bps, divided by 10 000 in ciphertext, and performs a confidential transfer to the slot’s recipient; the residual net is transferred to the redeemer. Because all bps were stamped at lifecycle hooks and the sum invariant was enforced at stamp time, the redeem path makes no external calls to fee modules — only the token transfers themselves — reducing the reentrancy surface on the hot path. The only revert paths are the existence check and a resolver-side **ConditionNotMet** revert; all other failure cases — wrong owner, underfunding, or prior redemption — are absorbed into the encrypted predicate and resolve to a zero-amount transfer with no public state diff. The operation emits **EscrowRedeemed** and one **FeeDistributed** event per non-zero slot.

6.4.5 redeemMultiple

The **redeemMultiple** operation extends **redeem** to a batch with branchless silent skip per id, capped at **MAX_BATCH_SIZE = 20** and rejecting empty input. For each id outside the existence bound or with a resolver returning false, the loop continues in plaintext — a deliberate leak limited to the gate predicate, never to the encrypted ownership-or-funding predicate. Per-id encrypted accounting is identical to **redeem**, accumulated into a single net ciphertext that drives one outbound confidential transfer to the redeemer after the loop; per-id fee transfers are issued inside the loop. Emits **EscrowBatchRedeemed**.

6.4.6 setUnderwriterFee

The `setUnderwriterFee` operation stamps the Underwriter slot (slot 2) of the four-slot fee array on behalf of the bound coverage manager. It is the only fee-stamping entry-point that fires outside escrow `create`: the Protocol slot is stamped at create from the registered protocol-fee module, the Condition slot is stamped at create from the resolver, and the Underwriter slot is stamped here at coverage-purchase time. The operation is gated by the coverage-manager modifier and by a per-slot single-write guard; the contract branchlessly zeroes the stamped bps when the holder is neither the escrow owner nor the original caller — by homomorphic select of the supplied bps against the authorization predicate, which is structurally required in the confidential variant where reverting would leak which holder was tested. The sum-of-stamped-bps invariant is enforced silently at the stamp via a homomorphic capping select; the plaintext variant of the same enforcement reverts with `FeeBudgetExceeded`. Emits `FeeStamped`.

6.4.7 Cross-chain entry — receiver-side `fundFrom`

Cross-chain receivers are external entry points that bridge into the escrow’s `fundFrom` path. The confidential CCTP V2 receiver validates a Circle Iris attestation, computes the actual USDC delta, extracts `escrowId` from CCTP hook data at byte offset 376, wraps the USDC to confidential USDC, and funds the destination escrow. A plaintext sibling receiver performs the same flow against the non-confidential `Escrow`. Cross-chain mechanics, replay protection, and operator economics are addressed in §9.

6.5 Condition Resolver Pluggability

A condition resolver is a Plugin in the sense of Definition 5.2: an external contract implementing `IConditionResolver` and ERC-165. The Escrow binds at most one resolver per `escrowId`. On `create`, when a non-null resolver is supplied, the condition extension rejects non-code addresses, invokes the resolver’s on-condition-set hook with the resolver-init data, and reads the resolver’s `getConditionFee` view to stamp the Condition fee slot (§6.2) with the resolver author’s bps and recipient. On `redeem`, the extension consults the resolver’s `isConditionMet` view and reverts with `ConditionNotMet` if false; the batch path returns the Boolean for plaintext skip. A null resolver is treated as the always-true condition with a zero Condition fee. The downstream encrypted predicate of §6.4.4 still applies, so a true resolver result is necessary but insufficient — the silent-failure path of Proposition 4.3 still hides any failure of the ownership or full-funding predicates.

6.6 Virtual Escrows

Implementation Note (Virtual Escrow Surface, as of May 2026). The v1.0 specification lists “virtual escrow (event-triggered without pre-funding)” alongside batch and conditional escrow as a supported mode. The shipped `ConfidentialEscrow` does not expose a dedicated virtual-escrow code path: an unfunded record is indistinguishable from a created-but-not-yet-funded record, and any cross-chain receiver can call `fundFrom` against an existing `escrowId` to satisfy the event-triggered semantics. The convergence question — whether a separate virtual-escrow surface is needed, or whether the existing `create` / `fundFrom` pair is the canonical realisation — is tracked for chaos-net v1.0.

6.7 Reentrancy and Economic Safety

Proposition 6.1 (Reentrancy Safety). Every external mutator in `ConfidentialEscrow` (`create`, `fund`, `fundFrom`, `redeem`, `redeemMultiple`, `setUnderwriterFee`, `setCoverageManager`, `setFeeModule`) carries `nonReentrant`. The `redeem` path follows checks-effects-interactions: the encrypted redemption flag is updated and the `FHE.allowThis` recommit happens before any `confidentialTransfer` to the redeemer or any fee recipient. No fee module is called on `redeem` — all fee bps were stamped at lifecycle hooks (§6.4.1, §6.4.6) and the `redeem` path only iterates the stored four-slot fee array. No external call from inside `redeem` can re-enter the same `escrowId` because (i) the OpenZeppelin `ReentrancyGuardUpgradeable` slot blocks any re-entry into the contract’s `nonReentrant` scope, and (ii) the encrypted-flag update has already moved the redemption flag past the false-branch of the `canRedeem` predicate. *Proof sketch.* By inspection of the `redeem` path: the encrypted redemption-flag mutation precedes every subsequent `confidentialTransfer`, and the wrapping `nonReentrant` modifier from the protocol base contract is in effect for the whole call. □

Proposition 6.2 (Encrypted Amount Conservation). For the set E of escrow ids and the contract's encrypted token balance B , the invariant $\text{Dec}(B) = \sum_{\{i \in E\}} \text{Dec}(\text{paidAmount}_i) - \sum_{\{i \in E, \text{redeemed}\}} \text{Dec}(\text{paidAmount}_i)$ holds modulo 2^{64} . Each `fund` / `fundFrom` adds the homomorphic balance-delta to `paidAmount`, so the contract's token balance and the per-escrow `paidAmount` accumulators move together by Proposition 4.5. Each `redeem` removes exactly `paidAmount` — the sum of the per-slot fee transfers ($\sum_k \text{paidAmount} \times \text{bps}_k / 10000$) plus the residual net transfer to the redeemer — or zero under the silent-failure branch. By the stamp-time invariant $\sum_k \text{bps}_k \leq 10000$, the per-slot sum is bounded by `paidAmount` and the net transfer is exactly the residual. *Proof sketch.* Induction over the lifecycle in Tab. 6.2: each transition's effect on the contract balance equals its effect on the sum of `paidAmount` ciphertexts under decryption, by additive homomorphism (Proposition 4.5). $\square$

6.8 Reference Plugins (Condition Resolvers)

The escrow package ships no condition-resolver implementations under its extensions surface; the only file there is the abstract `EscrowCondition` base described in §6.2. The first-party resolver implementations (PayPal via Reclaim, oracle, multisig, prediction-market) named in the v1.0 specification currently live outside the escrow package and are excluded from the v1.0 escrow audit scope. Tests reference a `MockConditionResolver` used to exercise the `set-condition` / `check-condition` paths.

Implementation Note (Reference Resolvers, as of May 2026). The v1.0 specification names zkTLS (Reclaim), oracle (Chainlink), prediction-market (UMA / Polymarket), and multisig gates as reference resolver implementations. None ship under the escrow package's extensions surface in v1.0. The `PayPalConditionResolver` referenced in development notes is staged in a separate package outside this whitepaper's scoped audit boundary. Enumeration of shipped reference resolvers is deferred to the plugin-examples package documentation.

6.9 Known Limitations and Open Problems

The known limits are as follows. (a) `MAX_BATCH_SIZE = 20` caps `redeemMultiple` at twenty IDs per call. (b) Each fee slot in the four-slot fee array may be stamped once per Escrow: a single-write guard per slot prevents fee rotation, and the cumulative $\sum \text{bps} \leq 10000$ invariant is enforced at stamp time. The Protocol slot (slot 0) is reserved and unused — the protocol charges no fee and holds no treasury — so the active fee surface comprises the resolver-author Condition slot and the Recourse-pool Underwriter slot; the only revenue-bearing slots route to third parties, never to the protocol. (c) The legacy ERC-7201 namespace `privara.storage.ConfidentialEscrowCondition` is retained for cross-deployment-version storage-layout compatibility (§6.2 Implementation Note). (d) The plaintext Escrow counter leaks the cumulative Escrow count, a deliberate concession for indexer compatibility consistent with Definition 4.4 (Public Skeleton). (e) `redeemMultiple` leaks per-ID existence and resolver success through the plaintext `continue` branches; the encrypted ownership and funding predicates remain hidden. (f) The `confidential_distributeFees` path computes `FHE.div(FHE.mul(paidAmount, bps), 10000)` on `euInt64`; with `bps` capped at 10 000, the safe `paidAmount` ceiling is $\sim 1.84 \times 10^{15}$ wei ($\sim \$1.84$ B per Escrow at six-decimal stablecoins). Wider scale requires `euInt128`, subject to substrate support.

Open Problem 6.1 (Batch-Redemption Fee-Order Atomicity). `redeemMultiple` performs per-id `confidentialTransfer` fee payouts inside the loop while accumulating the redeemer's net into a single post-loop transfer. A revert in any per-id fee transfer aborts the entire batch under `nonReentrant`, even though successful prior-id encrypted state mutations to `isRedeemed` (via a homomorphic OR) have already been written. Open question: should batch redemption pull fees through `Recourse Manager`-controlled withdraw addresses (avoiding per-id external calls), defer fee payout to a post-loop summed transfer per recipient, or accept the abort-on-fee-failure semantics? Decision target: chaos-net v1.0.

§7 — Recourse

7.1 Role in the four-primitive system

The Recourse primitive provides FHE-priced parametric coverage against settlement-adjacent risk: resolver failure, condition-evaluation failure, and Operator misbehaviour. It is a marketplace in which independent underwriters deploy policy contracts (cf. Definition 5.2) and compete for LP capital. Coverage is bound to an existing Escrow position (cf. Definition 3.2), premium pricing is performed homomorphically over encrypted risk scores (cf. Definition 4.1), and the encrypted-balance accounting of §4 (Proposition 4.5) extends to pool liquidity, premium flow, and claim payouts.

7.2 Role separation

The Recourse surface separates four operational roles and a Guardian safety role. Fig. 7.1 maps the four operational roles to the contract surfaces they control. The Operator role is detailed in §8 and appears here only where it touches Recourse; the Guardian role is described after the table.

Policy Builder	Controls: Authors <code>\UnderwriterPolicy`</code> ; registers in <code>\ConfidentialPolicyRegistry`</code>	Earns: Adoption fees + reputational signal (not per-coverage)
Pool Creator	Controls: Deploys pool via factory; retains <code>\addPolicy`</code> / <code>\removePolicy`</code> ; sets immutable creator royalty	Earns: Creator royalty share of premium flow
Pool Manager	Controls: Parameter custody, allocation, admission controls; transferable to DAO / multisig / Operator-as-a-service	Earns: Manager fee on premium flow (escrow fee surface)
Operator (§8)	Controls: Executes pool-routing + unstake-pullback tasks under operator-mediated repayment	Earns: Task-execution fee from <code>\FeeManager`</code> ; subject to slashing
Guardian — safety	Controls: Veto pending timelocked changes; emergency-deallocate routed liquidity (immutable-pool failure mode)	Earns: Safety backstop — no routing, no fee

Enforcement at the contract layer. Three distinct authorization modifiers: **onlyUnderwriter** (Pool Creator at the pool surface), **onlyCoverageManager** (the coverage-manager call path for payouts and premiums), and **onlyOwner** on the registry and factory.

*Listing 5: **Fig. 7.1 — Role separation in the recourse primitive.** Among the four operational roles, no single role both prices a coverage and adjudicates its dispute, nor both creates a pool and earns from it without the LP-side capital commitment. The Pool Creator is the only role granted persistent policy-authorship rights across manager transfers, which limits role-collapse pressure when management is delegated. The fifth Guardian role is a safety backstop — it earns nothing and cannot route capital — added for the immutable-pool failure mode and sitting outside the operational separation.*

Role	Action surface	Compensation
Policy Builder	Authors an IC ConfidentialUnderwriterPolicy contract (the evaluateRisk , judge , onPolicySet triple); registers it in ConfidentialPolicyRegistry .	Adoption fees if the policy is selected by pools; reputational signal via on-chain registration event.
Pool Creator	Deploys a new pool through the confidential pool factory for a chosen payment token; retains policy-authorship rights (the addPolicy / removePolicy pair) across manager transfers; sets an immutable creator royalty at pool creation.	Creator royalty share of premium flow, persistent across manager transfers.
Pool Manager	Holds parameter custody, allocation decisions, admission controls (open/closed mode, closed-pool invite vouchers). Defaults to the creator at deployment; transferable to any address (DAO, multisig, operator-as-a-service).	Manager fee on premium flow, delivered to the manager through the escrow fee surface (not the protocol).
Operator	Executes pool-routing and unstake-pullback tasks against RecoursePool under operator-mediated repayment mode (see §8).	Task-execution fee from Fee Manager ; subject to slashing on misbehaviour.
Guardian	Safety backstop for the immutable-pool failure mode: vetoes pending timelocked configuration changes and emergency-deallocates routed liquidity. Cannot route capital, set parameters, or earn. LP/DAO-controlled.	None — safety role.

The four-role split is enforced at the contract layer through three distinct authorisation modifiers: **onlyUnderwriter** (Pool Creator at the pool surface), **onlyCoverageManager** (the coverage-manager call path for payouts and premiums), and **onlyOwner** on the registry and factory. The Guardian sits outside this operational split: a dedicated safety address (EOA, multisig, or DAO timelock) whose only powers are to veto pending timelocked configuration changes and emergency-deallocate routed liquidity. It cannot price coverage, route capital, set parameters, or earn; it exists solely for the immutable-pool failure mode and supplements, rather than replaces, the four operational roles (see the routing-config gap in §7.10).

7.3 Contract surface

The on-disk surface as of June 2026 comprises four core contracts, one factory, and one policy registry. The plaintext variants (**RecoursePool**, **CoverageManager**, **PolicyRegistry**, **PoolFactory**) are intended for the mainnet launch path where FHE is not deployed; this chapter specifies the confidential variant, which is the canonical primitive. The plaintext variants implement the same role split with **uint256** in place of **euint64**.

Contract	Concern	Mandatory events
Confidential Coverage Manager	Coverage purchase, dispute, FHE-priced premium	CoveragePurchased , DisputeFiled , CoverageClaimed , CoverageExpired
Confidential RecoursePool	Per-underwriter LP capital, encrypted liquidity, claim payout, premium receipt	Staked , Unstaked , ClaimPaid , PremiumReceived , RewardsClaimed , PolicyAdded , PolicyRemoved
Confidential Policy Registry	Permissioned registry of validated policies; ERC-165 interface check	PolicyRegistered
Confidential PoolFactory	Pool deployment via ERC1967Proxy ; payment-token allowlist	PoolCreated , TokenAllowed , TokenRemoved
IC Confidential Underwriter Policy	Plugin interface for risk and dispute (encrypted return values)	(interface — no events)

The interface name **IUnderwriterPolicy** refers to the plaintext-launch variant; the confidential variant carries the **IC Confidential** prefix and returns **euint64** / **ebool** rather than **uint256** / **bool**.

7.4 Encrypted coverage purchase

The purchase lifecycle binds a coverage position to an existing Escrow, with the premium computed entirely in ciphertext. The eight steps below identify the calling role; each step states the encrypted-versus-plaintext distinction where relevant.

Pool access modes. A pool is fixed at creation as either *open* or *closed* (immutable; Nexus precedent). An open pool admits any buyer. A closed pool admits only buyers presenting a Pool-Manager-signed **EIP-712 invite voucher** — the **CoverageInvite** struct from **CoverageInviteLib** (@reineira-os/shared):

The buyer submits the voucher and the Manager’s signature with the purchase. The coverage manager recovers the signer over the EIP-712 typed-data digest (**ECDSA.recover**) and admits the purchase iff the signer holds the pool’s Manager role, **deadline** is unexpired, **invitee == msg.sender**, and the per-voucher use count is below **maxUses**. The digest itself is the on-chain usage key — changing any signed field yields a different digest, so usage counters never collide across distinct invites. Storage is written only on use or revocation, so invited wallets are never enumerated on-chain (an on-chain allowlist would publicly leak the Manager’s business relationships). The voucher binds to the public submitter (**msg.sender**), leaving the encrypted holder identity (step 1) untouched. This admission gate covers coverage purchase only; LP staking is a separate, orthogonal access axis that defaults to open on both pool types.

The cryptographic primitive (**CoverageInviteLib**) ships in @reineira-os/shared; the open/closed pool flag and the coverage-manager admission check land with the multi-owner pool work (§7.10).

1. **(Buyer)** Submits a coverage purchase against a chosen pool, policy, and escrow, supplying encrypted holder identity and encrypted coverage amount, plus plaintext expiry, policy data, and risk proof.
2. **(Manager)** Validates the pool against the factory and the policy against the pool; rejects expired or duplicate coverage. All checks are plaintext.
3. **(Manager)** Invokes the policy’s registration hook with the policy data; the hook’s effect is policy-specific.
4. **(Manager)** Reads the encrypted escrow amount and caps coverage homomorphically to the lesser of requested coverage and escrowed amount.
5. **(Policy)** Returns an encrypted risk score **euint64** from its risk-evaluation entry point.
6. **(Manager)** Computes premium homomorphically as the encrypted product of capped coverage and risk score divided by a basis-points constant (**10000**). No decryption.
7. **(Manager)** Attaches the encrypted premium to escrow realization via the escrow’s recourse-fee surface, stamping the **Underwriter** slot (§6.2).
8. **(Manager)** Stores the coverage record (**holder**, **plaintextHolder**, **pool**, **policy**, **escrowId**, **coverage Amount**, **expiry**, **Active**) and marks the escrow covered. Emits **CoveragePurchased**.

The FHE-priced premium pattern is concentrated in the coverage manager’s premium-computation block: the underwriter policy returns an encrypted risk score, the manager multiplies it homomorphically against the (already capped) encrypted coverage amount, divides by a basis-points constant in ciphertext, and forwards the resulting **euint64** to the escrow’s fee surface. Neither the score nor the per-buyer premium ever appears in plaintext on-chain. By Proposition 4.5, the pool’s encrypted balance accounting absorbs the premium without leakage.

7.5 Underwriter policy interface

IConfidentialUnderwriterPolicy (per Definition 5.2) is the pluggable surface that lets any third party ship a competing policy. It is a three-function ERC-165 interface; all functions are external and called only by the coverage manager. The three operations are **onPolicySet**, **evaluateRisk**, and **judge**.

The **onPolicySet** hook registers policy-specific configuration at coverage creation (for example, a dispute identifier). It is invoked atomically inside the coverage-purchase entry point before risk evaluation; any revert is policy-defined, and the function returns no value.

The **evaluateRisk** operation converts an opaque proof bundle — for instance, a zkFetch attestation of an external dispute datum — into an encrypted risk score consumed by the FHE premium formula. The policy must have been registered in **ConfidentialPolicyRegistry** (gated by ERC-165 introspection) and

allowlisted on the pool via **addPolicy**. The function returns a single **euint64** denominated in basis points; the manager treats values up to **10000** as 100%. Any revert is policy-defined and indicates an invalid proof.

The **judge** operation adjudicates a filed dispute and returns an encrypted Boolean. The coverage must be in **Disputed** status, and the caller must be the coverage manager. The function returns a single **ebool**; the manager combines the verdict with the coverage amount through homomorphic select so that an invalid verdict produces an encrypted-zero payout, preserving the balance-conservation property of Proposition 4.5. Any revert is policy-defined.

7.6 Dispute flow

A dispute is filed by the plaintext coverage holder and resolved synchronously in the same transaction for the v1.0 confidential surface.

1. The holder submits a dispute against the coverage from the plaintext-holder address recorded at purchase.
2. The manager validates that the coverage is still **Active** and, if expiry has passed, flips status to **Expired** and reverts.
3. Status flips to **Disputed** (CEI-safe ordering) and **DisputeFiled** emits.
4. The manager invokes the policy's **judge** entry point, which returns an encrypted boolean verdict.
5. The encrypted payout is the verdict combined with the coverage amount via homomorphic select (zero ciphertext if the verdict is false); the result is passed to the pool's pay-claim entry point.
6. The pool caps the payout against the pool's encrypted total liquidity in ciphertext (selecting the lesser of the requested amount and current liquidity homomorphically), debits the pool's encrypted total liquidity, transfers via the FHERC-20 surface, and returns the actual-payout ciphertext.
7. The manager forwards the payout to the plaintext holder and flips status to **Claimed**; **CoverageClaimed** emits.

Where a pool is gated by an Operator-quorum policy, the policy's **judge** implementation defers the verdict to an external quorum attestation collected through **QuorumRegistry** and submitted through the orchestration layer (forward-reference §8). The v1.0 on-disk policy surface assumes a synchronous return; the asynchronous variant is flagged in §7.10.

7.7 LP capital and economic invariants

The pool's encrypted accounting layer admits two invariants that hold per block under FHE-balance conservation (Proposition 4.5).

Proposition 7.1 (LP Solvency). At any block height

$$h$$

, the sum of outstanding encrypted claim payouts processed by **payClaim** does not exceed the pool's encrypted FHERC-20 balance available to the pool's encrypted total liquidity at that height. *Proof sketch.* The pay-claim path computes a capped amount as the homomorphic select of the lesser of the requested amount and the pool's encrypted total liquidity, debits the pool's encrypted total liquidity by exactly that capped ciphertext, then invokes the confidential transfer. The transfer is then bounded by the same encrypted ciphertext that bounded the debit. By Proposition 4.5, the FHERC-20 layer enforces that the confidential transfer cannot exceed the sender's encrypted balance. The conjunction of pool-side capping and token-side balance conservation guarantees that every payout is funded. $\square$

Proposition 7.2 (Premium / Loss-Ratio Disclosure Bound). The plaintext exposed to external observers from the recourse surface is at most: (a) coverage lifecycle events without amounts (**CoveragePurchased**, **DisputeFiled**, **CoverageClaimed**, **CoverageExpired**), (b) the policy and pool addresses bound to each coverageId, and (c) the pool-creation event with payment-token address. Per-buyer premium, per-buyer payout, individual stake amounts, total liquidity, and total premiums are all **euint64** ciphertexts. *Proof sketch.* By inspection of the event surface: **ClaimPaid** and **PremiumReceived** carry no parameters; **CoveragePurchased** and **DisputeFiled** carry only the coverage identifier. Every monetary field in storage

(total liquidity, total premiums, individual stakes, coverage amounts) is typed `euint64`. This matches the public skeleton bound of Definition 4.4. $\square$

7.8 Risk-score attestation

External attestors may post signed encrypted risk scores via a `RiskScoreAttestor` registry keyed by `(agentId, scoreType)` and gated against `QuorumRegistry` membership. The registry verifies signatures, stores the ciphertext alongside `modelVersion`, `decimals`, `sourceQuorumId`, and `attestedAt` metadata, and exposes the latest attested score per key. Underwriter policies that source risk inputs off-chain read the stored ciphertext and feed it into their `evaluateRisk` implementation. The contract is part of the v1.0 specification but lives in a separate `@reineira-os/identity` package that is not yet present in the on-disk monorepo — see §7.10.

7.9 Reference plugins

The Recourse package ships no reference underwriter policies as of June 2026. The on-disk tree contains only the core contracts, interfaces, and test mocks; no extensions or first-party plugin policies are shipped. The mocks (`MockConfidentialUnderwriterPolicy`, `MockUnderwriterPolicy`) are test fixtures, not production plugins. Reference plugin implementations are tracked separately and are out of scope for the v1.0 specification (see §7.10).

7.10 Specification-vs-implementation gaps

Implementation Note (`QuorumAttestedPolicy`, as of June 2026). The v1.0 specification calls for a `QuorumAttestedPolicy` as the abstract `IAsyncUnderwriterPolicy` counterpart to `QuorumAttestedResolver` — dispute judgment via quorum verdict, gated by `QuorumRegistry` membership lookup. The on-disk recourse package as of June 2026 contains no abstract policy base; `ICConfidentialUnderwriterPolicy` returns a synchronous `ebool` from `judge`. Convergence to an `IAsyncUnderwriterPolicy` surface that supports a Pending verdict state is tracked toward chaos-net v1.0.

Implementation Note (`RiskScoreAttestor`, as of June 2026). The v1.0 specification calls for a `RiskScoreAttestor` (storing `(euint64 encryptedScore, uint16 decimals, uint32 modelVersion, bytes32 sourceQuorumId, uint64 attestedAt)` per `(agentId, scoreType)` key) in a separate `@reineira-os/identity` package. No `identity` package and no `RiskScoreAttestor` contract is present in the on-disk monorepo as of June 2026. The risk-score input path is therefore policy-internal: `evaluateRisk` implementations consume the `riskProof` calldata directly rather than reading from a shared attestor registry.

Implementation Note (`PoolFactory` v1.0 surface, as of June 2026). The v1.0 specification calls for a pool-creation entry point that accepts open/private access, multi-owner thresholds, creator royalty, liquidity-routing config (`protocol[]`, `allocationBps[]`, `utilizationTriggerBps`, `rebalanceCadence`), and repayment-scheduling mode (`immediate | queued | operator`). The on-disk pool-creation entry point accepts only the payment token; the pool deploys with a single underwriter and no autorouting configuration. The `routeLiquidity / pullLiquidity / rebalance / requestUnstake / processUnstakeQueue` surface is absent from `ConfidentialRecoursePool`. The shared primitives this surface builds on are already on-disk in `@reineira-os/shared` — the EIP-712 admission helper (`CoverageInviteLib`) and the routing reserve/adaptor primitives (`PoolRiskLib`, `IYieldAdapter`). Wiring them in is split across two work items: the open/closed pool flag, the Pool-Creator/Pool-Manager separation, and the coverage-manager voucher check land with the multi-owner pool work; the routing-config surface, the `StrategyRouter`, and the Guardian safety role land with the liquidity-routing work. Convergence to the full v1.0 surface is tracked toward chaos-net v1.0.

7.11 Open problems

Open Problem 7.1 (FHE-priced dynamic-curve premium). The current premium formula is the encrypted product of capped coverage and risk score divided by a fixed basis-points constant; it is linear in the (encrypted) risk score and does not react to claim history. Whether the premium curve can be updated per-block from the pool’s own loss experience without revealing individual claims is unresolved. Selection criteria: (a) per-block FHE arithmetic budget for curve update; (b) information leakage from any public update event or rate-change signal; (c) underwriter manipulation surface when the curve parameters are themselves ciphertext; (d) interaction with Proposition 7.2 if any plaintext loss-ratio signal is published. Decision target: prior to chaos-net v1.0.

Open Problem 7.2 (Asynchronous judge return). The current `judge` entry point on `ICConfidentialUnderwriterPolicy` returns `ebool` synchronously, forcing `dispute` to settle in a single transaction. Async policies — the v1.0 specification marks `judge` as potentially returning a Pending verdict — require a Pending-state lifecycle: the manager must distinguish “judge has not yet returned a verdict” from “verdict returned false.” Selection criteria: (a) state-machine impact on `CoverageStatus` (introducing a Pending state); (b) replay protection against duplicate verdict submissions; (c) interaction with operator-quorum policies that complete asynchronously off-chain; (d) gas and FHE-handle lifetime for ciphertexts held across transactions. Decision target: prior to chaos-net v1.0.

§8 — Operator Set & Decentralization

8.1 Role in the four-primitive system

Operators are the off-chain execution layer of the Reineira Settlement Standard (RSS). They register **permissionlessly on-chain** in `OperatorRegistry`, post stake denominated in `cUSDC` (the immutable ERC-7984 wrapper of USDC, §9.6), claim and execute tasks dispatched by `TaskExecutor`, receive relay or task fees through `FeeManager` (§8.8), and remain subject to slashing through `OperatorSlashingManager`. The Operator role is the fourth protocol primitive alongside Escrow, Gate, and Recourse (see §3): it continuously turns declared intent and standing policy into settled state without requiring user-side liveness.

`cUSDC` denomination keeps the Operator economy entirely stablecoin-denominated: the Operator set is functional, incentivised, and slashable from chaos-net Day 1. The Operator economy needs no token — every stake and fee is denominated in `cUSDC` or another stablecoin, and its operation does not depend on a protocol-native token.

Permissionless from Day 1. The contract-level admission criteria are stake-and-sanctions-only: any address that meets the `minStake` threshold and passes the sanctions oracle check (§8.4) can register, regardless of jurisdiction. The protocol contracts do not discriminate by jurisdiction.

8.2 Why a separate operator set

The Operator set is neither a validator set nor a restaking deployment. Operators execute tasks the host EVM cannot evaluate natively: parsing and submitting cross-chain attestations (Circle CCTP V2 Iris attestations, LayerZero DVN sets), driving FHE ciphertext settlement against the CoFHE coprocessor, and submitting off-chain quorum verdicts that move asynchronous resolvers from **Pending** to **Resolved**. A dedicated Operator set, rather than a restaking adapter, lets the protocol enforce a single task-routing rule (the three-tier authorization window of Definition 8.1) and a single slashing surface across every asynchronous fulfilment domain.

The choice of a native bond over a restaking-adapter integration (EigenLayer, Symbiotic, Karak, or peers) is a deliberate tradeoff along four axes. Each axis names a property the protocol requires; the native-bond design satisfies it directly, the restaking-adapter design satisfies it only through additional middleware whose security and liveness assumptions stack on top of the protocol's own.

Liveness equivalence under restaked-only stake. The three-tier task authorization window (Definition 8.1) requires that within the exclusive window only the claiming operator may execute, and within the active-set window only non-slashed operators may. A native operator registry resolves both predicates with a single storage read against **OperatorRegistry**. A restaking adapter must (a) prove that the would-be operator has opted into the Reineira AVS, (b) prove that the operator's restaked balance currently exceeds **minStake** after middleware-side withdrawal queues, and (c) prove the operator has not been slashed *by another AVS* in a way that compromises its Reineira-side eligibility. The third clause has no clean cross-AVS resolution today.

Slashing-message bridging cost. **OperatorSlashingManager** runs a three-day challenge / four-day vote pipeline (§8.7) and applies the slash by SafeERC20 transfer from the registry's stake pool. A native bond closes the loop in one transaction. A restaking adapter must serialise the slashing decision into a middleware-readable message, bridge it to the restaking protocol, wait for the middleware's own delay (typically 7–14 days of veto / arbitration), and accept whatever fraction of the operator's stake the middleware decides to honour. The cumulative slashing-finalisation horizon under restaking is two to three weeks; under the native bond it is the seven-day window already encoded in **CHALLENGE_PERIOD + VOTING_PERIOD**.

Correlated-slashing exposure to the coverage pool. The Recourse primitive (§7) takes underwriter risk through **IConfidentialUnderwriterPolicy** and prices it homomorphically. If Operators stake exclusively in restaked positions, a single mass-slashing event on the underlying restaking protocol — triggered by an unrelated AVS — can simultaneously make an Operator ineligible for Reineira tasks and reduce the Recourse pool's effective collateral if the same restaked positions back coverage policies. The native bond separates the failure modes: an Operator slashing event does not propagate into coverage-pool LP positions, and a coverage-pool stress event does not reduce the Operator security budget.

Economic-security accounting versus the native bond. The protocol must publish, at any point in time, the total stake securing each task-type domain (**TASK_CCTP_RELAY**, **TASK_AUTOMATION**, **TASK_AGENT_CALL**). With a native bond this is a single sum of **OperatorRegistry** stake. Under a restaking adapter the same number requires querying every restaking middleware the operator has opted into, applying middleware-specific slashing-eligibility filters, and resolving any pending withdrawal-queue commitments — and the result is only a *snapshot* because operators may opt out of the Reineira AVS asynchronously through the middleware's own UI. Restaked economic-security accounting is a moving target; native-bond accounting is a state read.

The protocol is not closed to restaking. A future RSS revision may admit a restaking surface as an *additive* path alongside the native bond, gated on cross-AVS slashing semantics maturing and on a middleware that exposes its withdrawal queue and slashing veto as a deterministic state read. The conditions and selection criteria for that revision are tracked as Open Problem 8.2 (§8.9).

8.3 Contract surface

The orchestration package ships five concrete on-chain contracts. Contracts named in the v1.0 specification but not yet present on disk are tracked as Implementation Notes in §8.9.

Contract	Concern	Mandatory events
OperatorRegistry	Stake (in cUSDC), unbond, task-claim authorization, slashing sink	OperatorRegistered , StakeAdded , UnbondRequested , StakeWithdrawn , TaskClaimed , TaskExecuted , OperatorSlashed , MonitorUpdated , SlashingManagerUpdated , ConfigUpdated
TaskExecutor	Task-type routing to registered ITaskHandlers ; operator-fee routing trigger	TaskExecuted , HandlerRegistered , HandlerRemoved
FeeManager	Operator relay/task-fee calculation and routing to the executing operator; takes no protocol cut (see §8.8)	FeeCollected , FeeConfigUpdated , FeeCollectorUpdated , FeeTokenUpdated
OperatorSlashingManager	Optimistic propose/challenge/vote/execute slashing pipeline	SlashProposed , SlashChallenged , Voted , SlashExecuted , SlashRejected , ProposalExpired , BondReturned , BondSlashed
CCTPHandler	CCTP V2 burn-message + attestation handler; implements ITaskHandler	per ITaskHandler and CCTP-specific

All five are deployed as immutable singletons per §11.1. None carries an upgrade hook. Cross-contract authority (registry trusting slashing manager; task executor trusting handlers) is established at deployment and is non-modifiable. Each inherits the protocol base contract from `@reineira-os/shared` and ERC-2771 meta-transactions per the monorepo convention.

8.4 Registration, stake, and admission

Registration is **permissionless from Day 1** and gated by three contract-level constraints: (a) the offered stake meets `minStake` (denominated in cUSDC; deployment-set baseline), (b) the caller is not previously slashed (**PermanentlySlashed**), and (c) a configured sanctions oracle returns false for the caller. There are no jurisdictional or KYB requirements at the protocol layer.

On success the operator is added to an enumerable set of active operators and an **OperatorRegistered** (`operator`, `stake`) event fires. The lifecycle is **Active** → **Unbonding** → **Withdrawn** with a terminal **Slashed** state; the unbond request immediately removes the operator from the active set, and stake withdrawal requires `block.timestamp >= unbondRequestTime + UNBOND_PERIOD` where `UNBOND_PERIOD = 7 days`.

Stake is held in cUSDC (the immutable ERC-7984 wrapper of USDC). The **OperatorRegistry** binds cUSDC as its staking token at deployment; this binding is immutable. cUSDC inheritance means stake is denominated in USDC value 1:1 at the wrapper boundary, with the confidential-balance properties of §9.6 applying to per-operator stake balances. Additional stake may be added at any time via `addStake(amount)` provided no unbonding is in progress (**UnbondingInProgress**). Slashing is invoked by the configured **slashingManager** address (or **owner** as a backstop) and transfers the slashed cUSDC out to the slashing manager, where it is subsequently disbursed via the dispute-resolution flow described in §8.7.

Definition 8.1 (Three-tier task authorization). Let

$$t_c$$

be the timestamp at which an operator

$$o$$

called `claimTask(taskHash)`, let

$$W_e$$

be the **exclusiveWindow**, and let

$$W_p$$

be the `permissionlessDelay`. For any caller

$$c$$

, `canExecuteTask(c, taskHash)` returns true iff

$$c$$

is not slashed, the task is not yet executed, and one of: (i) the task is unclaimed and

$$c$$

is active; (ii)

$$c = o$$

; (iii)

$$t - t_c \geq W_p$$

; or (iv)

$$W_e \leq t - t_c < W_p$$

and

$$c$$

is active.

8.5 Task routing and execution windows

The task-executor entry point looks up the handler registered for the given task type, computes the canonical task hash through the handler, checks that the caller is authorised to execute under the registry's three-tier rule (Definition 8.1), executes the task, marks it complete, and, for fee-bearing task types, triggers fee collection through the fee manager. The **owner** registers the handler set, subject to a cross-check that each handler's declared task type matches its registration key.

Three task-type constants are defined on-chain in `TaskLib`. Only `TASK_CCTP_RELAY` has a shipped handler; the other two are reserved selectors.

Constant	Value	Handler on disk	Status
<code>TASK_CCTP_RELAY</code>	<code>keccak256("CCTP_RELAY")</code>	<code>CCTPHandler</code>	Shipped
<code>TASK_AUTOMATION</code>	<code>keccak256("AUTOMATION")</code>	none	Reserved selector
<code>TASK_AGENT_CALL</code>	<code>keccak256("AGENT_CALL")</code>	none	Reserved selector; see §8.9

The authorization window (Definition 8.1) is the protocol's stake-weighted liveness rule. The claiming operator holds the exclusive write right for the first

$$W_e$$

seconds; any active operator may execute between

$$W_e$$

and

$$W_p$$

seconds after the claim; after

$$W_p$$

seconds any address may execute. Fig. 8.1 traces the three tiers along the time axis. The contract does not implement weighted assignment within the active set: routing among eligible operators is first-come, first-served at the on-chain layer and dispatched by the off-chain coordinator service (§8.11). Window parameters are governance-set via `setConfig(minStake, exclusiveWindow, permissionlessDelay)`.

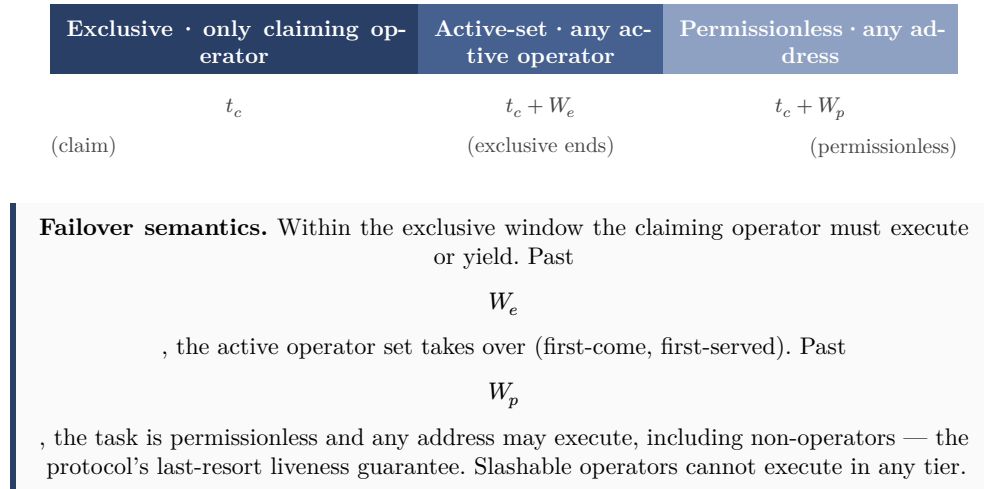


Figure 2: **Fig. 8.1 — Three-tier task authorization window.** Time progresses left-to-right from the claim timestamp

t_c
 . The exclusive window
 $[0, W_e)$
 grants the claiming operator a monopoly; the active-set window
 $[W_e, W_p)$
 broadens execution rights to any non-slashed operator; the permissionless tail
 $[W_p, \infty)$
 opens execution to any address. The Reineira reference deployment uses
 $W_e = 60$
 s and
 $W_p = 600$
 s .

Window tier	Bound	Operator subset	Failover
Exclusive	$0 \leq \Delta t < W_e$	Claiming operator only	None — claim holder must execute or yield
Active-set	$W_e \leq \Delta t < W_p$	Any non-slashed active operator	Within the active set
Permissionless	$\Delta t \geq W_p$	Any address	Full fallback

8.6 Quorum and dispute

Stake-weighted voting on operator misconduct is implemented in the slashing manager's vote entry point; vote weight is the voter's current stake recovered from the registry, and quorum is `(totalStaked * QUORUM_BPS) / 10000` with `QUORUM_BPS = 1000` (10 % of total active stake). This is the only quorum primitive shipped in the v1.0 orchestration package. A general-purpose `QuorumRegistry` for plugin-side N-of-M attestation (forward-referenced in §7.6) is not present on disk and is tracked in §8.9; the recourse-side

quorum-attested policy primitive forward-referenced in §7 will consume that registry when shipped. Cross-chain bridge-attestation quorum semantics are deferred to §9.

Why 10 % rather than a BFT supermajority. The slashing pipeline is *not* a single stake-weighted vote — it is a four-stage dispute moat whose 10 % quorum is one stage among several, calibrated so that a coordinated minority cannot turn slashing into a griefing primitive. The conventional BFT-grade convention ($\geq 2/3$ stake-weighted quorum) is the right target for *consensus* primitives where the vote *is* the decision; the slashing manager is an *adjudicated* primitive where the vote is only the final stage of a multi-week pipeline. The protocol’s safety argument against minority capture rests on the cumulative cost an attacker must pay across every stage, not on the quorum threshold alone. Specifically:

- (a) *Evidence-hash precondition (§8.7).* **SlashProposal** carries an **evidence** hash recorded in the **Operator Slashed(operator, amount, evidence)** event. A coordinated minority cannot slash without producing evidence that a downstream auditor, monitor, or off-chain verifier can replay. Stake weight alone is insufficient — the vote *adjudicates* a claim that must externally justify the slash. Proposition 8.1 binds this property: every decrement to Operator stake traces to an **Executed** proposal whose **evidence** argument is observable.
- (b) *Asymmetric bond requirement (§8.7).* Both proposer and challenger post a 5 % bond of the slash amount. A coordinated minority attempting frivolous slashing must lock 5 % of the targeted slash for the duration of the dispute pipeline, with full forfeiture if the proposal is rejected. The bond is denominated in the same staking token as the slashable position, so the attacker is paying for the attack in the same currency they would be slashing.
- (c) *Slow-dispute pipeline (§8.7).* **CHALLENGE_PERIOD = 3 days** (any party may challenge an unchallenged proposal) is followed by **VOTING_PERIOD = 4 days** (a stake-weighted vote on the challenged proposal). Unchallenged proposals execute permissionlessly only after the three-day window closes. A coordinated minority must hold its 10 % stake position open for at least seven days under public observation; honest stakers observing a malicious proposal have three full days to post the 5 % challenger bond and force the vote into the 10 %-quorum stage, where the attacker must defeat any honest counter-vote.
- (d) *Owner backstop with constrained discretion (§8.7).* **OperatorRegistry.slash** accepts calls from either **slashingManager** or **owner**. The **owner** path exists for emergency response (e.g., a coordinated-minority attack visibly in flight). The **owner** role is operational and bounded — it cannot upgrade the contract (there is no upgrade hook, §11.1) and cannot modify slashing parameters; it can only invoke the same **slash** entry point with the same checks-effects-interactions discipline. The backstop converts the worst case from “minority succeeds against honest absence” to “minority succeeds against honest absence *and* against the backstop veto.”
- (e) *Current-stake weighting (§8.6 vote entry point).* Vote weight is **OperatorRegistry.stakeOf(voter)** at the moment of the vote, not a snapshot. An attacker accumulating stake before the proposal must keep that stake in the active set through the seven-day dispute pipeline. Stake added in the unbonding state does not count toward vote weight, and unbonding immediately removes the operator from the active set — so an attacker cannot stake-then-unstake to amplify their own quorum weight without paying the seven-day capital cost.

The four-stage construction (evidence + bond + slow dispute + backstop) carries the safety argument; the 10 % quorum is calibrated not to block honest slashing under a thin Operator set. A BFT-supermajority quorum at chaos-net

$$N \in [5, 10]$$

would effectively require all Operators to vote on every slash, converting slashing from an adjudicated dispute into a unanimous-consent primitive — the protocol could not slash a misbehaving Operator unless every other Operator was online and voting simultaneously. The 10 % threshold preserves slashing liveness through the bootstrap window, while the cumulative dispute moat preserves safety. As

grows past the basin threshold (

$$N \geq 30$$

, §8.10), `QUORUM_BPS` is governance-tunable upward and may be scheduled on a stake-set-size-dependent curve; Open Problem 8.1(c) records the scheduling decision target.

8.7 Slashing

`OperatorSlashingManager` implements an optimistic dispute flow with parameters fixed at the contract level. A proposer posts a bond of `(amount * 500) / 10000` and opens a `SlashProposal` with an `evidence` hash. The proposal enters `Pending` for `CHALLENGE_PERIOD = 3 days`. If unchallenged, anyone may call `execute(proposalId)` to apply the slash; the proposer recovers their bond plus a slasher reward of `(slash Amount * 1000) / 10000`. If challenged within the window, the challenger posts a matching 5 % bond, and the proposal enters `Challenged` for `VOTING_PERIOD = 4 days` of stake-weighted Operator voting. The losing party's bond is forfeited to the winning party; if the proposal expires after `EXPIRY_PERIOD = 14 days`, both bonds are returned.

The shipped contract recognises a single generic slashing class parameterised by `(operator, amount, evidence)`; the v1.0 specification's distinction between liveness, correctness, and equivocation faults is encoded off-chain in how `evidence` is computed and which monitor publishes the proposal. Slashed stake is transferred to the calling slashing manager at the registry layer through `SafeERC20`, after which the resolution branch distributes the slasher reward and returns or forfeits bonds. Routing of residual slashed stake (e.g., to the Recourse pool or a remediation pool) is governance-configurable and not hard-coded in the registry's slash entry point; in no configuration does it accrue to a protocol treasury, which does not exist.

Proposition 8.1 (Slashing Safety). *Let*

$$\Sigma$$

denote the set of slashing executions that decrement an operator's stake. Then for every

$$s \in \Sigma$$

there exists an on-chain `SlashProposal` whose final state is `Executed` and whose `evidence` hash is recorded in the corresponding `OperatorSlashed(operator, amount, evidence)` event. Proof sketch. `OperatorRegistry.slash` reverts unless `msg.sender ∈ {slashingManager, owner}` and emits `OperatorSlashed` with the `evidence` argument. The only path through `OperatorSlashingManager` to that call is the slash-execute path or the vote-resolve path with `slashApproved == true`, both of which set `proposal.state = ProposalState.Executed` and invoke `registry.slash(...)` with `proposal.evidence`. The owner backstop is an operational escape that exists for emergency response; under the immutable-contract posture of §11.1, `owner` is bound to the same `slash` entry point and is not an upgrade-authority hook (which does not exist). $\square$

8.8 Operator-fee routing and the protocol's zero-charge posture

The protocol charges nothing. The protocol takes no fee, no commission, and no share of settlement or relay volume — before mainnet and after. There is no protocol treasury and no activation block at which a protocol charge switches on. `FeeManager` is not a protocol-revenue contract; it is an **operator-fee router**. Its only job is to compute an operator's relay/task fee on an executed task and transfer that fee to the executing operator. No protocol cut is taken at any point, and `FeeManager` holds no protocol balance.

The operator's relay/task fee is the operator's own charge for the cross-chain relay work it performs — set within bytecode-pinned bounds and kept by the operator, not by Reineira Labs (§12.2). On fee collection the operator share is transferred immediately via `SafeERC20` to the executing operator. The fee is denominated in the moved asset (USDC via CCTP V2, USDT via LayerZero OFT). Because there is no protocol share,

the configuration is a single operator rate per task type rather than a split; the rate is bounded above by immutable bytecode (`MAX_OPERATOR_FEE_BPS`, etc.) and validated by an `InvalidFeeConfig` guard.

The protocol extracts no value from users at any point: the operator economy is sustained by operator relay fees, never by a protocol take. No value is captured at the protocol layer and routed to the vendor.

Fee component	Source	Recipient	Notes
Cross-chain operator fee	<code>FeeManager.collectFee TASK_CCTP_RELAY</code>	Executing operator (immediate SafeERC20)	Operator's own relay charge; no protocol cut taken
Slasher reward, 10 % of slashed stake	<code>OperatorSlashingManager</code> slash-execute / vote-resolve paths	Successful slash proposer	Denominated in cUSDC; paid from the slashed stake, not a protocol fee

The operator fee rate sits between the bytecode-pinned floor (zero) and the bytecode-pinned operator-fee ceiling, set by governance within those bounds. Raising the ceiling requires deploying a new immutable `FeeManager` at a new address; operators and integrators opt in to the new schedule by directing flows through the new manager. The protocol take is zero at every rate, by construction — there is no configuration of `FeeManager` that routes any value to a protocol treasury, because no such treasury exists.

8.9 Specification-vs-implementation gaps

The following surfaces are referenced in the technical specification but are not yet shipped in the orchestration package as of the publication snapshot.

Implementation Note (CoordinatorRegistry, as of May 2026). The v1.0 specification calls for a `CoordinatorRegistry` for on-chain coordinator registration and subscription, with the cross-graph slashing guarantee requiring slashing votes to span at least three independent coordinator-operator graphs. The shipped `OperatorSlashingManager` enforces a single stake-weighted quorum across the full active-operator set and does not partition operators by coordinator. Convergence with the coordinator-registry specification is a v1.0-track item.

Implementation Note (Agentic handlers, as of May 2026). The v1.0 specification calls for three additional `ITaskHandler` implementations — `LZ0FTHandler` (LayerZero OFT relay for USDT0), `AgentCallHandler` (on-chain agent invocation), and `VerdictSubmitHandler` (off-chain quorum verdicts), plus `PoolRoutingHandler` for `IRecoursePool` routing. Only `CCTPHandler` ships in v1.0. `TASK_AGENT_CALL` is reserved as a selector in `TaskLib` but has no registered handler. Treatment of the agentic-composition layer is out of scope for v1.0 RSS.

Implementation Note (Agent identity and quorum registries, as of May 2026). The v1.0 specification calls for `AgentIdentityRegistry` ([ERC-8004]), `QuorumRegistry`, `RiskScoreAttestor`, `AgentInvocationAdapter`, `EIP8183Adapter`, `ModelCallTicketRegistry`, and `AgentConfigRegistry`. None are shipped in the orchestration package or in any sibling package as of the snapshot. The agentic-composition layer is deferred from v1.0 RSS until these contracts ship.

Open Problem 8.2 (Restaking integration). No on-chain integration code with EigenLayer, Symbiotic, Karak, or any other restaking protocol ships with v1.0. Whether a future RSS revision admits a restaking surface — as an alternative to or in addition to the native cUSDC bond — is an open question. Selection criteria: (a) liveness equivalence under restaked-only stake; (b) slashing-message bridging from `Operator`

SlashingManager to restaking middleware; (c) coverage-pool exposure to correlated slashing events; (d) economic-security accounting versus the native bond.

8.10 Decentralization roadmap

The operator set is **permissionless at the contract layer from chaos-net Day 1** — any address that meets stake + sanctions + not-previously-slashed criteria can register (§8.4). The roadmap below describes expected organic set growth, not gated transitions.

At chaos-net (July 2026) the active operator set is expected at

$$N \in [5, 10]$$

entities — bounded by the still-emerging market for cross-chain confidential-settlement relayers rather than by contract-level admission gating. The set grows to

$$N = 20$$

by end of 2026 and crosses the basin threshold

$$N \geq 30$$

by Q1 2028. Permissionless growth is sustained by operator relay fees (kept by the operators, §8.8); the operator economy needs no token to scale (§12).

Thin-set risk budget during the bootstrap window. The 21-month interval from chaos-net (Jul 2026) to basin threshold (

$$N \geq 30$$

, Q1 2028) is operationally thin even though contract-level admission is permissionless from Day 1. The contract treats every registered operator identically; the thinness reflects market emergence, not protocol gating. The risk budget that justifies operating through this window rests on four compensating controls, each of which materially reduces the harm a misbehaving operator can cause during the thin-set window.

- (a) *Public mode at chaos-net hides no encrypted value.* The chaos-net deployment runs the protocol in public mode (Definition 3.1; §3.6). Encrypted-mode state — encrypted owner, amount, paid amount, redemption flag, coverage holder, coverage amount, risk score, premium, payout — does not exist on-chain until encrypted mode unlocks at v1.0 mainnet hardening (Q4 2026, gated on Fhenix CoFHE deployment). The thin-set window for *encrypted* value is therefore at most six months (

$$N \approx 20$$

by end of 2026 through

$$N \geq 30$$

by Q1 2028, less the public-mode portion), not the full 21 months.

- (b) *Zero-charge protocol posture.* The protocol takes no fee at any point (§12.2), so there is no protocol-revenue lever that thin-set concentration could inflate. Operator relay fees are sized for permissionless competition and kept by the operators; the protocol does not optimise for thin-set operator yield, because it captures nothing to redistribute. With no protocol take to grow, there is no fee-revenue incentive to delay set expansion — the zero-charge posture forecloses that channel entirely.
- (c) *Slashing pipeline operates from*

$$N = 1$$

. The dispute moat constructed in §8.6 — evidence-hash precondition, asymmetric proposer / challenger bonds, 7-day cumulative dispute window, owner backstop, current-stake vote weighting — operates identically at

$$N = 5$$

and at

$$N = 100$$

. The slashing primitive does not degrade under a thin set; the 10 % quorum threshold ensures that slashing remains *executable* under a thin set while the four-stage dispute moat preserves safety. Liveness of the slashing primitive across the entire bootstrap window is the property that lets the protocol respond to a misbehaving operator without waiting for set expansion.

- (d) *Permissionless protocol layer is observable on-chain*. The contract-level **OperatorRegistry** admits any sanctions-clean, sufficiently-staked address from chaos-net Day 1 (§8.4). Any external party can verify at any point that the protocol layer is non-discriminating. The contract code itself encodes no admission gate beyond stake and sanctions.

The cumulative effect of (a)–(d) is that the 21-month set-expansion window is bounded by the public-mode → encrypted-mode boundary (six months of encrypted value at risk under a thin set), constrained by the conservative fee posture, hardened by the slashing pipeline that operates from

$$N = 1$$

, and made externally auditable by the protocol-layer permissionlessness. None of these controls makes the bootstrap window decentralised — they bound the per-period harm an attacker can extract during it.

Open Problem 8.1 (Operator set growth trajectory). The expected operator-set growth path (

$$N \in [5, 10]$$

chaos-net →

$$N = 20$$

end-2026 →

$$N \geq 30$$

Q1 2028) reflects market emergence under permissionless registration. Actual trajectory depends on: (a) cross-chain confidential-settlement market growth; (b) the competitiveness of operator relay fees relative to operators' cost of capital and execution; (c) integration with **AgentIdentityRegistry** (see §8.9) when shipped, which may strengthen identity-attribution under high

$$N$$

. No **PERMISSIONLESS_SUNSET_BLOCK** or admission-gate-removal event is required since admission is permissionless from Day 1.

8.11 Off-chain operator services

The protocol-side off-chain components are three: a coordinator service that monitors source chains for fee-bearing events and dispatches tasks to subscribed operators, an operator node that claims and executes tasks on-chain, and a CLI for registration, staking, unbonding, and status queries. Each is a standalone TypeScript service with its own OpenAPI surface and Docker artifacts. The off-chain transport, dispatch policy, and coordinator-operator authentication are deployment details outside the scope of the on-chain protocol specification; the on-chain contract layer is sound under any off-chain dispatcher that respects Definition 8.1.

Any party may operate an independent coordinator against the same on-chain **OperatorRegistry / Task Executor** surface; the protocol does not require operators to subscribe to any particular coordinator. The operator node and CLI are open-source, distributed via GitHub, and runnable by any operator from their own infrastructure.

§9 — Cross-Chain Settlement & Multi-Chain Hosting

Reineira routes value across chains through bridge-handler contracts that accept native-stablecoin transports (Circle CCTP V2 for USDC, LayerZero OFT for USDT) and funnel into a single confidential-balance escrow surface. The protocol is hosted on Arbitrum L2 as the canonical chain across all roadmap horizons; Ethereum L1 is planned as a second host; OP-Stack hosting is on the long-term roadmap. Third-party rollups extend coverage by self-deploying ReineiraOS via Reineira Atlas under an L3-owner maintenance model. Part A specifies the cross-chain settlement surface (the on-protocol bridge-handler abstraction); Part B specifies the multi-chain hosting roadmap and the Atlas-deployed-L3 model. The academic comparator framing for CCTP V2 and LayerZero OFT is given once in §2.5 and not repeated here.

Part A — Cross-Chain Settlement

9.1 Stablecoin-agnostic and bridge-agnostic design

The protocol accepts ERC-20 stablecoins through a confidential-balance wrapper layer. The on-disk reference implementation ships **ConfidentialUSDC** for USDC; **ConfidentialUSDT** for USDT (USDT0 path) and a generic wrapper factory for additional payment tokens such as DAI and EURC are specified and tracked toward chaos-net v1.0. Value is routed across chains through a bridge-handler abstraction in **@reineira-os/orchestration**. The reference implementation (Definition 3.2) ships native USDC settlement via Circle CCTP V2; USDT settlement via LayerZero OFT / USDT0 is on the roadmap. Additional bridge protocols integrate by registering further handlers without modifying the escrow engine.

9.2 Bridge handler abstraction

Every bridge integrates as a parallel pair of forwarder (source-chain) and receiver (destination-chain) contracts that funnel into the same **ConfidentialEscrow.fundFrom** sink. Each handler registers with **Task Executor** and implements **extractAmount(payload)** so **FeeManager** can compute relay fees on the true relayed amount. Replay protection is application-level and enforced by the receiver.

Bridge handler	Transport (as-set · status)	Sink behaviour
CCTPHandler	Circle CCTP V2 burn-mint · USDC · Shipped	Validates Iris attestation; mints USDC to receiver; wraps to cUSDC ; funds the destination escrow
CCTPV2Confidential EscrowReceiver	Circle CCTP V2 (FHE sink) · USDC · Shipped	Verifies the CCTP V2 attestation, derives usdcReceived from the balance delta, wraps to cUSDC , funds escrow
CCTPV2Escrow Receiver	Circle CCTP V2 (public-mode sink) · USDC · Shipped	Same as above, plaintext mode
CCTPV2Forwarder	Circle CCTP V2 (source side) · USDC · Shipped	Encodes escrowId into CCTP hook data at byte offset 376; burns USDC via Circle TokenMessenger
LZ0FTHandler / LZ0FTEscrow Receiver / LZ0FTEscrow Forwarder	LayerZero OFT (USDt0) · USDt · Roadmap	Validates DVN attestation set; receives USDt0; wraps to cUSDt ; funds the destination escrow. Replay guard keyed by LayerZero guid

9.3 Same-chain funding

Same-chain funding requires no bridge. The funder approves the payment token to the escrow contract and calls the **fund** operation with the encrypted payment; the contract records the pre-transfer balance, executes the confidential transfer, records the post-transfer balance, and derives the actual payment as the homomorphic subtraction of the post- and pre-balance, accumulating the encrypted result into **paidAmount** by homomorphic addition. The balance-delta pattern is resilient to token fees, rounding, and rebasing.

9.4 Cross-chain funding

Cross-chain funding via CCTP V2 walks the canonical burn-then-mint path. The source forwarder encodes the destination **escrowId** into the CCTP hook payload; an off-chain coordinator monitors the burn and fetches the Circle Iris attestation; a registered operator claims the relay task and submits the attested message to the destination receiver, which mints USDC, wraps it, and funds the escrow via **fundFrom**. Replay protection is application-level (the message-replay set).

The eight-step sequence below traces a single cross-chain funding. Each step is tagged (**chain · caller**); the bridge intermediate follows when non-trivial.

1. (**Source · Funder**) Approves USDC to the CCTP V2 forwarder.
2. (**Source · Funder**) Submits a cross-chain transfer to the forwarder for the target escrow and destination domain. The forwarder encodes **escrowId** at hook offset 376 and burns the USDC via Circle's TokenMessenger.
3. (**Off-chain · Coordinator**) Monitors the source-chain burn event and fetches the attestation from Circle Iris.
4. (**Off-chain · Coordinator**) Distributes the relay task over SSE to subscribed operators.
5. (**Destination · Operator**) Claims the relay task from the operator registry within the 0–60 s exclusive claim window.
6. (**Destination · Operator**) Executes the relay task on the CCTP handler, which calls the receiver's settle entry point with the Iris-attested CCTP V2 message.
7. (**Destination · Receiver**) Receives the attested CCTP V2 message (minting USDC), wraps to **cUSDC**, and funds the destination escrow with the encrypted payment and funder identity.

8. **(Destination · Receiver)** Triggers fee collection in the fee manager and marks the task executed in the operator registry.

Proposition 9.1 (Cross-Chain Atomicity Bound). Cross-chain funding via CCTP V2 is atomic at the level of the burn-then-mint primitive: the source burn destroys USDC and the destination mint is gated on a valid Iris attestation of that burn. The gap between burn confirmation on the source chain and the destination-chain `settle` call is bounded only by source finality, Iris attestation latency, and operator relay scheduling — empirically tens of seconds to several minutes. The protocol therefore distinguishes a `funding_in_flight` state observable on the destination chain prior to the receiver’s `fundFrom` call: the escrow’s encrypted `paidAmount` does not advance, and `redeem` returns the silent-failure transcript, until the destination `settle` lands. □

9.5 Settlement bridging back

Redemption settles on the destination chain — the chain on which the escrow was created and the encrypted `paidAmount` was accumulated. The redeemer receives the unwrapped underlying token (USDC for `cUSDC`-funded escrows, USDT for `cUSDT`-funded escrows) on that same chain. Carrying value back to a different chain requires a separate CCTP V2 or LayerZero OFT transfer initiated by the redeemer; that transfer is outside the escrow protocol’s atomicity guarantee.

9.6 Stablecoin wrappers

The reference implementation ships two confidential wrappers at v1.0: `ConfidentialUSDC` (cUSDC) for USDC, and `ConfidentialUSDT` (cUSDT) for USDT (USDT0 via LayerZero OFT). Both are ERC-7984 confidential wrappers, 1:1 convertible to their underlying stablecoin, and **immutable** under the §11.1 posture — upgradeability on the confidential token boundary would be a critical attack surface, and the immutable-singleton pattern applies uniformly. Operations expose `confidentialBalanceOf`, `confidentialTransfer`, `confidentialTransferFrom`. Wrapping accepts the unwrapped underlying stablecoin and mints the encrypted-balance wrapper (Definition 4.1, encrypted type `euint64`); unwrapping burns the wrapper and is asynchronous, since it requires owner permission to decrypt the balance handle.

The operator bond is denominated in cUSDC only (§8.1), keeping the operator economy single-staking-token. Recourse pools, escrows, and user-facing flows accept either cUSDC or cUSDT at v1.0; per-pool denomination is set at pool creation. Additional wrappers (EURC, USDS, PYUSD when GENIUS-compliant) ship in future versions as new immutable wrapper deployments per the §11.1 versioning pattern; no factory holds upgrade authority over existing wrappers.

Part B — Multi-Chain Hosting Roadmap

9.7 Canonical host: Arbitrum L2

Arbitrum L2 is the canonical host across every roadmap horizon — chaos-net, mainnet v1.0, and beyond. The chaos-net launch deploys to Arbitrum L2 in mid-July 2026, with v1.0 mainnet hardening across Q4 2026 (first independent operator-set audit complete by end of Q4 2026). The choice is anchored on three properties. Phenix CoFHE — the FHE coprocessor on which encrypted mode depends — targets Arbitrum-compatible deployment, so the public-mode → encrypted-mode in-place upgrade path requires no chain migration. Arbitrum is also a mature L2 with a settled fee market and a permissionless Orbit programme that lets third-party rollups inherit the same bridge primitives (§9.10). And the standard Arbitrum L1↔L2 gateway already supports L1-issued ERC-20s, so payment-token wrappers and other L1-issued assets list on Arbitrum without bespoke ceremony.

9.8 Planned host: Ethereum L1

Ethereum L1 hosting is on the deployment roadmap as a second host, not a cutover target — the protocol runs on Arbitrum L2 and on Ethereum L1 in parallel once L1 deployment lands, and the canonical host remains Arbitrum L2. L1 hosting addresses two markets that the L2 host cannot serve symmetrically:

applications that require Ethereum L1 finality for regulatory or counterparty reasons (institutional escrow flows where the L1-vs-L2 distinction matters for trade-settlement accounting), and applications that prefer to keep value at the L1 layer (high-value recourse coverage where the per-claim payout offsets the L1 gas premium). L1 deployment requires a public-mode-only host until an FHE coprocessor lands on L1 — encrypted mode remains exclusive to Arbitrum L2 until then. L1 host activation is targeted as a post-mainnet horizon.

9.9 Long-term host: OP-Stack

OP-Stack hosting is on the long-term roadmap as an additional parallel host, not a primary deployment. Activation timing and the choice of specific OP-Stack chain are TBD per OP ecosystem maturity — in particular, fraud-proof maturity, Superchain bridge standardisation, and Fhenix CoFHE availability on the chosen chain. The host is included in the multi-chain inventory because the operator economic model assumes a non-trivial set of host chains over time (§9.10): planning for an OP-Stack third host is a hedge against single-vendor concentration on the Arbitrum stack and is not a load-bearing assumption of v1.0 economics.

9.10 Atlas-deployed third-party L3s

Third-party rollups — including Arbitrum Orbit L3s settling to Arbitrum One, and (when supported) OP-Stack L3s — extend ReineiraOS coverage by **self-deploying** the protocol via Reineira Atlas. The L3 owner uses the **/bootstrap** and protocol-deployment skills in Atlas to scaffold an RSS-conformant deployment on their chain, then maintains it on their own cadence — upgrades, audits, fee-parameter changes, and operator-set governance all sit with the L3 owner. Reineira Labs does not operate or upgrade those deployments. The L3-owner maintenance posture is what makes self-deployment scalable: ReineiraOS is published as a public standard with a working reference implementation, and any rollup that wants the four primitives runs them on its own chain without entering a commercial relationship with Reineira Labs.

What the L3 *does* require from the ReineiraOS network is **liquidity** — and that comes from the canonical operator set. Operators registered on Arbitrum L2 route cross-chain funding into L3-deployed escrows over the **native rollup bridge** of the parent chain: for an Arbitrum Orbit L3, this is the Arbitrum L2→L3 bridge produced by the Orbit deployment; for an OP-Stack L3, this is the corresponding native bridge (vendor TBD per §9.9). The operator set is a single pool spanning all hosts, and the per-handler fee model (§9.2) settles fees on whichever host receives the bridge sink. Self-deployed L3s do not ship their own CCTP V2 or LayerZero OFT handlers — those bridge transports operate at the L1/L2 layer, not L3 — and the native rollup bridge is the only required transport. Fig. 9.1 shows the host-and-bridge inventory.

Arbitrum L2		Canonical host — chaos-net Jul 2026, v1.0 mainnet Q4 2026, all roadmap horizons	Live
		Bridges: CCTP V2 (USDC), LayerZero OFT (USDT0, planned), Arbitrum L1↔L2 gateway (L1-issued assets)	
Ethereum L1		Planned host — institutional + L1-finality use cases	Planned
		Bridges: CCTP V2 (USDC L1↔L2)	
OP-Stack (TBD)	chain	Long-term host — vendor TBD per OP ecosystem maturity	Roadmap
		Bridges: TBD per OP ecosystem; CCTP V2 (USDC) once Circle adds support	
Third-party (self-deployed Atlas)	L3 via	L3 owner deploys and maintains; ReineiraOS network supplies liquidity via operators	Open
		Bridges: native parent-rollup bridge only (Arbitrum L2↔Orbit L3; OP-Stack L2↔L3 TBD)	

Figure 3: **Fig. 9.1 — Multi-chain host inventory.** Arbitrum L2 is the canonical host across all horizons. Ethereum L1 is the planned second host; OP-Stack is on the long-term roadmap. Third-party L3s self-deploy ReineiraOS via Reineira Atlas: L3 owners maintain the deployment; the canonical operator set supplies cross-chain liquidity over the native parent-rollup bridge. The protocol is not migrated off Arbitrum L2 — additional hosts run in parallel.

9.11 Finality model

Escrow redemption requires destination-chain finality and never settles optimistically. On Arbitrum L2, finality follows Arbitrum’s model: one-confirmation soft finality for UX, full Ethereum-anchored hard finality after the Arbitrum challenge window. On Ethereum L1 (when the L1 host activates), redemption inherits L1 finality directly. On OP-Stack hosts (long-term), finality is governed by the chosen OP-Stack chain’s fault-proof model. On Atlas-deployed third-party L3s, finality is governed by the L3’s own proof model — fraud-proof L3s inherit a challenge-period hard finality; validity-proof L3s inherit a proof-verification finality bound by their prover cadence. The rule that redemption must wait for destination-chain finality holds in every case. Operators routing liquidity across a parent rollup’s native bridge wait for parent-chain finality before settling the destination escrow’s `fundFrom`.

9.12 Why Arbitrum L2 as canonical host

Settling under Ethereum L1 security (transitively, via Arbitrum L2) inherits the broadest economic security available to a public chain at L2 fee cost. An independent L1 or a Reineira-operated L3 would have to bootstrap a validator set or sequencer, a self-funded security budget, and an external bridge to reach the same stablecoin liquidity. By staying on Arbitrum L2 and inviting third-party L3s to self-deploy via Atlas, the protocol externalises that substrate-bootstrap cost to L3 owners who already operate their own sequencer for their own application — the multi-chain inventory grows without the protocol operating any substrate of its own.

The model also preserves an exit. In adversarial scenarios (sequencer capture, governance attack, regulatory pressure on Arbitrum) the canonical host can be re-targeted to Ethereum L1 once L1 deployment lands, or to an OP-Stack host once that horizon activates; the protocol’s contract surface is rollup-agnostic by construction. Atlas-deployed L3s exit independently — each L3 owner decides whether to migrate or wind down their deployment without affecting the canonical host.

Open Problem 9.1 (OP-Stack host native bridge). Activation timing and the choice of OP-Stack chain for the long-term OP-Stack host (§9.9) are TBD per OP ecosystem maturity. The relevant maturity inputs are: fraud-proof activation on the candidate chain, Superchain bridge standardisation, and Fhenix CoFHE availability. Decision target: deferred to the OP-Stack host activation milestone.

§10 — Security Model & Threat Analysis

The security model formalises the adversary classes the protocol defends against, the assumptions that must hold for the formal properties of prior chapters to compose, and the boundary between confidential state and the public skeleton.

Chaos-Net Audit Posture (June 2026 → Q4 2026). The chaos-net deployment ships without a completed external smart-contract audit. The audit-readiness package summarised in §10.6 — an internal Phase 7 self-review with ~85 manual findings and eight remediated mainnet blockers, plus Slither / Aderyn tool-baselines and a Safe-plus-TimelockController deploy posture — is the entirety of the published security evidence for the v1.0 chaos-net surface. The external auditor is not yet engaged at draft publication; firm selection is gated on the Standard-tier audit-budget activation. **Use of chaos-net deployments during the June 2026 → Q4 2026 window is undertaken at the user's own risk.** The threat model and trust assumptions below state what the protocol is designed to defend against; they do not constitute an audit finding. External-audit completion is committed for the v1.0 hardening pass at Q4 2026 mainnet stabilisation, at which point the whitepaper drops the **Draft** marker and this section is revised against the auditor's published findings.

10.1 Threat model

The protocol enumerates eight threat classes. Each is anchored to the chapter where the threat is operative. Fig. 10.1 maps each class to its primary defence. The model classifies adversaries by interaction mode — passive observer A1, active participant A2, malicious operator A3, network adversary A4, compromised zkTLS notary A5, front-runner A6, compromised owner A7 — and the eight classes below project that taxonomy onto the protocol's deployed surface.

T1	Adversarial counterparty (funder / redeemer / coverage holder)	→	Homomorphic ownership checks + silent-failure (Prop 4.3)
T2	Adversarial resolver or underwriter plugin	→	Per-coverage scope isolation + plugin-author trust acknowledged
T3	Adversarial operator (selective / equivocating relay)	→	Three-tier task auth (Def 8.1) + stake-weighted slashing (Prop 8.1)
T4	Adversarial bridge attester (CCTP Iris / LayerZero DVN)	→	Bridge transport attestation + funding_in_flight bound (Prop 9.1)
T5	Substrate-level adversary (CoFHE compromise)	→	CoFHE operator quorum (Open Problem 4.1 tracks decentralization)
T6	Passive network observer (chain analytics, MEV)	→	Silent-failure indistinguishability (Prop 4.3) + disclosure bound (Prop 7.2)
T7	Governance / upgrade-vector capture	→	Immutable contracts foreclose any upgrade or state-mutation attack (§11.1)
T8	Software Vendor publication integrity (TA7)	→	External pre-deployment audit + on-chain bytecode verification; immutable means no post-deployment injection

Figure 4: **Fig. 10.1 — Threat-to-defence map.** Each of the eight threat classes is paired with the primary protocol-layer defence enumerated in §10.1. Residual exposure for each is itemised in §10.8.

1. **T1 — Adversarial counterparty.** A funder, redeemer, or coverage holder that submits malformed, replayed, or unauthorised state-mutating calls against **ConfidentialEscrow** or **ConfidentialCoverage Manager** in an attempt to redeem an unowned escrow, over-claim a coverage payout, or probe encrypted state. Defended by homomorphic ownership checks and the silent-failure pattern (Proposition 4.3).
2. **T2 — Adversarial resolver or underwriter plugin.** A pluggable **IConditionResolver** (escrow gate) or **IConfidentialUnderwriterPolicy** (recourse policy) registered into a coverage that returns dishonest evaluations to release escrowed funds, deny legitimate disputes, or mis-price encrypted premiums. Defended at the protocol layer by isolating plugin failure to the per-coverage scope; not defended against plugin-author malice on plugins that escrow authors voluntarily wire in.
3. **T3 — Adversarial operator.** A registered operator that fails to relay a claimed cross-chain task within the **permissionlessDelay**, executes selectively, or equivocates on attested off-chain work. Defended by the three-tier task-authorization predicate (Definition 8.1), stake-weighted slashing per §8.7, and Proposition 8.1.
4. **T4 — Adversarial bridge attester.** A Circle CCTP V2 Iris attester (USDC rail) or LayerZero DVN (USDT0 rail, planned per §9.2 Implementation Note) that issues a forged or withheld attestation on a cross-chain message. Defended only to the extent of the bridge transport’s own attestation surface; the protocol’s **funding_in_flight** state (Proposition 9.1) bounds blast radius to the in-flight transfer.
5. **T5 — Substrate-level adversary (CoFHE).** An adversary that compromises the Fhenix CoFHE coprocessor service to forge ciphertexts, exfiltrate the substrate-held secret key, or return incorrect homomorphic evaluations. Defended only by the CoFHE operator-quorum and attestation surface; tracked as Open Problem 4.1 (CoFHE Decentralization).
6. **T6 — Passive network observer.** A chain-analytics actor, MEV searcher, or commercial counterparty that mines public EVM traffic for per-escrow amounts, counterparties, premiums, or payouts. Defended by Proposition 4.3 (Silent-Failure Indistinguishability) and Proposition 7.2 (Premium / Loss-Ratio Disclosure Bound).
7. **T7 — Governance / upgrade-vector capture.** Under the immutable-contract posture of §11.1, immutability prevents any upgrade or state-mutation attack: no coalition can reach into deployed

bytecode to mutate state-bearing contracts. The only residual surfaces are (a) the canonical-deployment registry, which is a documentation artifact rather than on-chain authority, and (b) the **owner** role on **OperatorRegistry** for the slashing-backstop path (§8.6). Defended by the structural impossibility of an upgrade attack.

8. **T8 — Software Vendor publication integrity (TA7).** A publisher emits bytecode at the canonical-deployment addresses that differs from the audited source code. The attack must occur at deployment moment, not afterwards (the immutability posture closes the post-deployment window). Defended because the Software Vendor publishes audited bytecode matching the audited source; independent parties verify via bytecode-hash comparison and deterministic deployment (external pre-deployment audit per §10.6; on-chain bytecode verification via Sourcify / Etherscan post-deployment); and immutability prevents post-deployment injection.

The A5 (compromised zkTLS notary) class is absorbed into T2 — zkTLS notaries surface to the protocol only as evidence inputs to **IConditionResolver**-style plugins, and the protocol layer treats them as plugin-internal trust. The A6 (MEV at the wrapping boundary) class is acknowledged in T6 but its mitigation is sequencer-ordering at the substrate, not a protocol-layer defence; it is restated under the Limits paragraph in §10.8.

10.2 Trust assumptions

Six trust assumptions underwrite the protocol's formal properties. Each pairs a primitive assumption with the surface that depends on it and the failure mode if the assumption breaks.

- **TA1 — TFHE IND-CPA security.** TFHE ciphertexts are computationally indistinguishable under chosen-plaintext attack, conditional on the hardness of Learning With Errors [Regev 2005]. *Surface:* all **euint**-typed state in **ConfidentialEscrow**, **ConfidentialUSDC**, **ConfidentialUSDT**, **ConfidentialRecoursePool**, **ConfidentialCoverageManager**. *Failure mode:* confidentiality is lost; attestation integrity is preserved insofar as the substrate attestation commits to ciphertext handles rather than to plaintexts.
- **TA2 — CoFHE service correctness and secret-key custody.** The Fhenix CoFHE coprocessor evaluates homomorphic circuits correctly and retains custody of the substrate secret key. *Surface:* every **FHE.*** precompile call. *Failure mode:* forged or incorrect ciphertexts cause silent loss of funds, since the encrypted balance invariants in Proposition 4.5 and Proposition 6.2 are constructed against correct evaluation. Decentralisation of this trust point is tracked as Open Problem 4.1.
- **TA3 — Bridge transport finality and attestation honesty.** Circle CCTP V2 source-domain finality and Iris attestation are honest for the duration of the in-flight cross-chain transfer; LayerZero OFT DVN attestation set is honest for USDT0 transfers. Both bridge rails ship at v1.0 (§9.2). *Surface:* **CCTPHandler**, **CCTPV2ConfidentialEscrowReceiver**, **LZ0FTHandler**, **LZ0FTEscrowReceiver**. *Failure mode:* unauthorised mint at destination or stranded burn at source on the affected rail. Bounded by Proposition 9.1 to the in-flight transfer; redemption does not depend on this assumption once **paidAmount** has accumulated. Per-rail trust assumptions are independent — a CCTP V2 attestation failure does not propagate to USDT0 flows and vice versa.
- **TA4 — Upgrade-authority custody.** *Eliminated under the immutable-contract posture of §11.1.* No protocol contract has an upgrade-authorisation hook; no entity holds upgrade authority over deployed bytecode. The threat-assumption surface that conventional protocol designs require for TA4 does not exist in v1.0. The reference deployment moves protocol evolution through new-contract deployment plus opt-in user migration; users hold funds in contracts whose bytecode is frozen at deployment, and no party can reach into a deployed contract to modify it. *Residual surface:* the bytecode-verification discipline at deployment time — see TA7. *Net effect:* the rugpull-via-upgrade failure mode is structurally eliminated, not mitigated.
- **TA5 — Operator-set honesty.** At least one honest operator exists and is reachable through the coordinator surface so that the permissionless-fallback path of Definition 8.1 succeeds within bounded time; for slashing the dispute quorum requires **QUORUM_BPS = 1000** (10 % of total active stake) honest participation per §8.6, augmented by the §8.6 four-stage dispute moat. Operator registration is permissionless from chaos-net Day 1 at the protocol layer (§8.4); an off-chain recommended-operators list

aids operational recommendation but does not gate registration. *Surface*: **OperatorRegistry**, **OperatorSlashingManager**, **TaskExecutor**. *Failure mode*: indefinite task stall (liveness) or unjust slashing (correctness). The honesty bound here is task-availability, not 2/3-by-stake — the protocol’s quorum primitive is a 10 % stake-weighted threshold per §8.6, not a BFT supermajority.

- **TA6 — EVM execution correctness on the host chain.** Each host chain on which ReineiraOS is deployed (Arbitrum L2 as the canonical host, Ethereum L1 as planned second host, OP-Stack as long-term, plus any third-party rollup self-deploying via Atlas per §9.10) executes deployed bytecode correctly, orders included transactions deterministically, and preserves storage. *Surface*: every contract. *Failure mode*: arbitrary protocol-state corruption; outside the protocol’s threat surface and addressed at the substrate layer.
- **TA7 — Software Vendor publication integrity.** The Software Vendor (§11.2) publishes audited bytecode that matches the audited source code. Reference-deployment contract addresses are verified post-deployment by independent parties via deterministic-deployment factories and bytecode-hash comparison against the audited artifact. *Surface*: the published deployment artifact; the canonical-deployment registry (a documentation artifact). *Failure mode*: a publisher emits malicious bytecode that differs from the audited source. *Mitigation*: external pre-deployment audit (§10.6); on-chain bytecode verification via Sourcify or Etherscan post-deployment; deterministic CREATE2-style deployment at addresses derivable from source-hash. The immutability posture means a malicious publisher cannot inject post-deployment; the attack must occur at the deployment moment and is visible to any party who reads bytecode.

10.3 Formal properties

The protocol’s safety, liveness, and confidentiality properties are stated as numbered propositions in the chapters where they are operative. The list below restates each by number and anchor, then introduces a single system-level proposition tying them together.

- **Definition 4.1 (Encrypted Type)** and **Definition 4.4 (Public Skeleton)** — the typing boundary between confidential and observable state.
- **Proposition 4.3 (Silent-Failure Indistinguishability)** — success and failure transcripts of any silent-failure-pattern call are computationally indistinguishable to non-key-holding observers under TA1.
- **Proposition 4.5 (Balance Conservation under FHE)** — homomorphic sums over **euint64** ciphertexts decrypt to the corresponding plaintext sums modulo the substrate plaintext modulus.
- **Proposition 4.6 (No Ciphertext Forge)** — no EVM-side party can construct a ciphertext that decrypts to a chosen plaintext under TA1.
- **Proposition 6.1 (Reentrancy Safety)** — every external mutator in **ConfidentialEscrow** carries **non Reentrant** and follows checks-effects-interactions.
- **Proposition 6.2 (Encrypted Amount Conservation)** — escrow **paidAmount** accumulators and the contract’s encrypted token balance move together by additive homomorphism.
- **Proposition 7.1 (LP Solvency)** — pool-side encrypted capping plus token-side balance conservation guarantee every coverage payout is funded.
- **Proposition 7.2 (Premium / Loss-Ratio Disclosure Bound)** — the recourse surface exposes at most coverage-lifecycle events without amounts, policy and pool addresses, and pool-creation metadata; every monetary field is **euint64**.
- **Proposition 8.1 (Slashing Safety)** — every stake decrement traces to an **Executed SlashProposal** with a recorded **evidence** hash.
- **Proposition 9.1 (Cross-Chain Atomicity Bound)** — cross-chain funding is atomic at the burn-mint primitive level; a **funding_in_flight** state is observable on the destination chain until the receiver **settle** lands.

Proposition 10.1 (System Soundness). Conditional on (TA1–TA6), the protocol’s settlement outcome (paid-vs-unpaid status, payout amount, redemption recipient) depends only on (a) the truthful evaluation of any wired-in resolver or underwriter plugin per §6.5 and §7.5, (b) the operator-set’s honest task execution and slashing dispute behaviour per §8, and (c) the cross-chain transport’s burn-then-mint integrity per §9. No party outside those three surfaces can alter the settlement outcome. *Proof sketch.* Encrypted ownership and payment-sufficiency checks (Proposition 4.3, Proposition 6.1) prevent T1 from altering redemption.

Balance conservation (Proposition 4.5) and amount conservation (Proposition 6.2) prevent T1 and T5 from inflating the redeemed amount beyond **paidAmount**. The public-skeleton bound (Definition 4.4, Proposition 7.2) prevents T6 from extracting confidential quantities from the observable trace. Slashing safety (Proposition 8.1) constrains T3-induced slashes to the on-chain dispute pipeline. Cross-chain atomicity (Proposition 9.1) constrains T4 to the in-flight window. LP solvency (Proposition 7.1) bounds recourse payouts to encrypted pool liquidity. The residual dependence on plugin truthfulness, operator honesty per TA5, and bridge transport per TA3 is exactly the three-surface decomposition stated. $\square$

10.4 Observable vs confidential boundary

The disclosure surface is fixed by Definition 4.4 (Public Skeleton) and enumerated in Tab. 4.3 of §4.6. The protocol commits to keeping per-actor amounts, counterparties (escrow owner, coverage holder), per-escrow conditions, per-coverage premium and payout, and per-stake amounts confidential under encrypted mode. The protocol commits to publishing aggregate solvency through the public-skeleton fields (sequential escrow IDs, coverage IDs, contract addresses, event ordering, gas-payer addresses, and the bridged token amount at the CCTP burn/mint boundary). In public mode, items in the confidential column are observable on-chain; every other property in this chapter (threat-class mitigations, access controls, the formal properties at the logic layer) is preserved.

10.5 Access control

Five authorities partition the protocol’s privileged surface. The table records the current holder, the functions controlled, and the transition target where one is specified.

Authority	Held by (current)	Functions controlled	Transition target
Upgrade authority	None — contracts are immutable (§11.1)	None — no upgrade hook exists on any v1.0 mainnet contract	N/A
Trusted Forwarder (ERC-2771)	Forwarder address bound at construction of the protocol base contract	Sender attribution under the sender accessor — relayer pays gas, original signer attributed	Trusted-forwarder address bound at construction
slashing Manager	OperatorSlashingManager address, set at registry initialisation (with owner as back-stop)	OperatorRegistry.slash ; routing of slashed stake	Same address held by the protocol multisig
Underwriter plugin registration	Coverage creator (escrow author wiring a policy)	IConfidentialUnderwriterPolicy.evaluateRisk, judge invocations from ConfidentialCoverageManager	Per-coverage; remains author-controlled by design
Coverage Manager admin (owner operational role)	Protocol multisig	Admin setters setEscrow, setPoolFactory ; policy registration via PolicyRegistry . Cannot upgrade or migrate state-bearing contracts	Held by the protocol multisig
Recommended-operators list (chaosnet+)	Protocol multisig	Off-chain published list of operators for operational recommendation; does not gate on-chain registration	List is operational; not a protocol-layer authority

FHE-level access is gated separately: ciphertexts are readable only by addresses granted permission via **FHE.allow** or **FHE.allowTransient**, and a **complianceOwner** role supports selective disclosure for regulatory audit without exposing state to the public chain. Sanctions screening is wired into **OperatorRegistry** via an optional **ISanctionsOracle** that blocks registration of listed addresses .

10.6 Known audit posture

No external smart-contract audit is complete as of publication. The audit-readiness package is an internal Phase 7 self-review (scope-freeze 2026-05-05) covering the four in-scope contract packages — tokens, escrow,

orchestration, and recourse — with ~85 manual findings and eight mainnet blockers routed to remediation. A companion checklist (2026-05-07) records the pinned dependency hashes and Safe-plus-Timelock mainnet-deploy posture the external auditor will verify. Tool-baseline reports from Slither (v0.11.5) and Aderyn (v0.6.8), dated 2026-05-04, are retained internally. The external auditor is not yet engaged; firm selection is gated on the Standard-tier audit-budget activation.

10.7 Non-custodial software posture

The protocol is non-custodial software: it makes no regulated-entity claim and asserts no certification. The security evidence in this chapter concerns the contract surface and its trust assumptions, not any regulatory attestation.

10.8 Limits

The security model explicitly does not cover: the correctness of operator-binary off-chain code, which the protocol observes only through its attestation surface and slashing-evidence hashes; substrate-level (CoFHE) denial-of-service against homomorphic evaluation, against which the operator-set's fee-routing posture per §8.8 offers economic disincentive but no protocol-layer guarantee; bridge counterparty risk during in-flight cross-chain transfers, bounded by Proposition 9.1 but not eliminated; compromise of off-chain key material on operator hardware (HSM posture is the operator's responsibility, out of scope for the protocol layer); and the A6 MEV-at-wrap-boundary class, whose mitigation is the underlying-chain sequencer ordering described in §9.7, not a protocol-layer defence.

§11 — Governance Posture

11.1 Immutable contracts, no upgrade authority

Every protocol contract referenced in this whitepaper is deployed as an immutable singleton at a fixed address. There is no UUPS proxy, no `_authorizeUpgrade` hook, no Owner role with upgrade privileges, no admin key on any escrow, coverage manager, operator registry, or token wrapper. The protocol's storage layout is frozen at deployment; the protocol's logic is frozen at deployment. No party holds upgrade authority over the deployed contracts.

Ownership of the deployed contracts is renounced to reach this immutable state: any deployment-time owner role is relinquished once configuration is set, so that the contracts are left with no upgrade authority and no privileged controller. From that point the bytecode is the protocol.

Functional evolution ships through *new contract deployments at new addresses*. The migration pattern is opt-in: a user holding state in v1 contracts may continue to use v1 indefinitely, or may withdraw from v1 and deposit into v2 at their own initiative. v2 deployments coexist with v1 on-chain; no party can force-migrate users.

This posture has four consequences worth surfacing.

- (a) *No rugpull surface exists at the protocol layer.* Every “what if the escrow is maliciously upgraded to drain funds?” scenario is structurally impossible: there is no upgrade to execute. The threat surface that TA4 (upgrade-authority custody, §10) addressed in upgradeable-protocol designs is eliminated rather than mitigated.
- (b) *Bug fixes are migration events, not silent upgrades.* If a vulnerability is discovered in deployed bytecode, the response is a new contract deployment, a published security advisory, and a user-driven migration. Users hold funds in immutable contracts at their own risk; no party can reach into v1 to repair it. This is the same posture as Liquity v1 (LUSD), Uniswap v1/v2/v3, RAI, and Morpho Blue. It places a high

bar on pre-deployment audit quality and a high bar on the bytecode-verification discipline of any party deploying the reference implementation.

- (c) *Storage-layout conventions (ERC-7201 namespaces, `gap[50]` reserves) remain in force*, but for a different purpose. In an upgradeable design they protect against storage collisions across upgrades; in the immutable design they protect against storage-layout incompatibility across *deployment versions* — a v2 deployment that intends to interoperate with v1 (e.g., share an **OperatorRegistry** reference) inherits the storage-layout-stable interface surface via ERC-7201 namespaces.
- (d) *The Software Vendor classification is structurally preserved*. The party that authored and deployed the contracts holds no operational control over them. The “developer continues to operate” failure mode — where developers retain admin keys and are arguably continuing operators — does not apply: there are no keys to retain.

The single residual question is *which contract addresses are canonical*. A canonical-deployment registry (a documentation surface, not an on-chain contract) lists the addresses of v1.0 deployments across host chains (§9). Users may interact with any other deployment of the same bytecode at their own choice.

11.2 Non-custodial protocol, Software Vendor, no token, no fees

The protocol is non-custodial. Funds are held in escrow contracts owned by users (encrypted owner field); no third party holds a key to user funds. The protocol contracts are non-custodial, immutable, permissionlessly-accessible smart-contract code on a public chain.

Reineira Labs Limited is the **Software Vendor**: it authors the protocol contracts, publishes the Reineira Settlement Standard (RSS), distributes Reineira Atlas and Reineira Code via GitHub, maintains documentation, coordinates external audits, and deploys the canonical reference implementation. It does not operate the protocol, does not custody user funds, and does not custody admin keys on deployed contracts (there are none to custody, §11.1).

There is no protocol token. Reineira does not issue, allocate, vest, or distribute any token, and the protocol’s operation does not depend on a token existing (§12.1). The protocol charges no fee: it takes no commission and no share of settlement, relay, or coverage volume, and there is no protocol treasury (§12.2). The only fees in the system are set and kept by independent third parties for the work or capital they supply.

§12 — Economics

ReineiraOS is open protocol infrastructure that charges nothing for its own operation. There is no protocol token, no protocol fee, and no protocol treasury that accrues value from settlement volume. The fees that exist in the system are set and kept by independent third parties — Gate builders, recourse pools and their liquidity providers, and operators — for the work or capital they each supply. Reineira Labs receives none of them. Reineira Labs is a software company funded by equity.

12.1 No token

There is no REINEIRA token and no token sale. Reineira does not issue, allocate, vest, or distribute any token, and there is no points programme, snapshot, or airdrop that converts into one. Every economic position in the protocol is denominated in standard assets:

- **Operator stake** is posted in **cUSDC** — the immutable ERC-7984 wrapper of USDC (§9.6) — and bound as the staking token of **OperatorRegistry** at deployment (§8.1, §8.4).
- **Recourse pool capital** is supplied by liquidity providers in the pool’s stablecoin (§7).
- **All fees** (Gate condition fees, recourse premiums, operator relay/task fees) are paid in the moved, pooled, or settlement asset — never in a protocol-native token.

Reineira Labs is **equity-funded**. The protocol's operation does not depend on a token existing, and the operator economy in particular is fully functional, fully incentivised, and fully slashable from chaos-net Day 1 without one (§8.1). The protocol is operationally complete as software; no token is a prerequisite to any of it.

12.2 The protocol charges nothing

The protocol takes no fee, no commission, and no share of settlement volume. There is no fee-activation constant and no block at which a protocol charge switches on — the protocol charges nothing, before and after mainnet. Fees that exist in the system are configured and collected by independent third parties for the service or capital they provide, and route directly to those parties. Reineira Labs is a party to none of these flows.

Tab. 12.1 — Third-party fees in the system.

Fee	Configured by	Captured by	Denomination	Reineira Labs share
Gate condition fee	The Gate builder	The Gate builder (resolver author)	Escrow's payment token	None
Recourse premium	The recourse pool	The pool and its LPs	Pool stablecoin	None
Operator relay/task fee	The operator	The executing operator	Moved asset (USDC / USDT)	None

Reineira Labs receives none of the above. Each fee is set by the party that performs the work or supplies the capital, and is kept by that party. The protocol contracts do not skim a cut from any of them, and no protocol treasury accrues from settlement, relay, or coverage. **FeeManager** exists only to route an operator's relay/task fee to the executing operator (§8.8); it takes no protocol cut and holds no protocol balance.

12.3 Earning roles

Participants who supply work or capital to the protocol earn directly from the parties they serve. These flows accrue to ecosystem participants, not to Reineira Labs.

Tab. 12.2 — Earning roles.

Role	Activity that earns	Stake / capital	Reward denomination
Gate builder (resolver author)	Condition fee on every escrow gated by their resolver (returned by <code>IConditionResolver.getConditionFee</code>)	Optional reputation bond	Stablecoin (escrow's payment token)
Recourse underwriter / LP	Premium on each covered escrow, delivered to the backing pool and routed to its liquidity providers	Pool stablecoin liquidity	Pool stablecoin
Operator	Relay/task fee on cross-chain settled stablecoin (USDC via CCTP V2, USDT via LayerZero OFT), routed to the executing operator by FeeManager	cUSDC	Stablecoin (USDC / USDT)
Developer / app	Any fee an application prices on top of the protocol	—	Application-defined

Stake is denominated in cUSDC for operators; underwriting is denominated in the pool's stablecoin liquidity. No reward in this schedule is denominated in a protocol-native token, because none exists. The economic

claim of each role is on the fees its counterparties pay for the service it renders — and on nothing the protocol skims, because the protocol skims nothing.

12.4 Reineira Labs

Reineira Labs is a software company funded by equity. The protocol underneath is free, and stays free: there is no roadmap step, governance trigger, or activation block at which the protocol begins charging. The open protocol is **free forever**.

§13 — Licensing & Public-Goods Posture

13.1 Licensing Summary

The core protocol source is published under the Business Source License 1.1 ([MariaDB 2017]) with Change Date **2029-06-01** and Change License Apache License 2.0. **Licensor at publication: Reineira Labs Limited**, the Software Vendor that publishes the open-source software (§11.2). The Change Date is irrevocable and pre-announced at license publication; on that date the entire BUSL-licensed corpus converts to Apache License 2.0 by operation of the BUSL Change License clause, without further act of the Licensor. Pre-announcement is itself a load-bearing commitment: the §13.4 trigger schedule constrains the Licensor to only earlier conversions, never later.

13.2 Layer-Asymmetric License Vector

Different protocol layers carry different licenses at publication. Builder-facing layers (interfaces, SDK, token wrappers, plugin examples) are permissively licensed at publication to remove license-sensitivity friction for integrators; defensive layers (escrow, recourse, orchestration core) carry BUSL-1.1 during the contestability window and convert to Apache License 2.0 on the Change Date. The layer assignment is invariant under builder license-sensitivity $U1 \in [5\%, 40\%]$; each layer's license defends against a specific OpenR-class funding path (BUSL-core deters VC-backed, custodial-relayer, and grant-funded forks; MIT-SDK removes adoption friction).

Tab. 13.1 — Layer-asymmetric license vector at publication and after Change Date.

Layer	Repository / package	License at publication	License after 2029-06-01
Core protocol (escrow, recourse, orchestration on-chain core)	escrow, recourse, orchestration packages	BUSL-1.1	Apache License 2.0
Shared interfaces, libraries, base contracts	shared package	Apache License 2.0	Apache License 2.0
Confidential token wrappers (immutable ERC-7984: cUSDC, cUSDT)	tokens package	MIT	MIT
SDK and integration libraries, plugin templates, cofhejs glue	SDK and plugin-examples packages	MIT	MIT
Operator services (off-chain orchestrator, coordinator, CLI)	offchain and operators packages	Apache License 2.0	Apache License 2.0
Documentation (whitepaper, lightpaper, architecture specs)	docs/	CC-BY-4.0	CC-BY-4.0

Rows mirror the layer taxonomy applied to the monorepo package layout declared in the repository root **LICENSE** file.

13.3 What BUSL-1.1 Permits and Prohibits

(a) **Permits.** BUSL-1.1 grants the recipient the right to copy, modify, create derivative works of, redistribute, and make non-production use of the Licensed Work. Source availability is unrestricted; academic review, audit, security research, and non-production integration require no commercial license.

(b) **Prohibits.** Production use of the Licensed Work is restricted during the BUSL window. The Reineira Labs Limited Additional Use Grant is “None”; production deployment of the core protocol therefore requires a commercial license from the Licensor until the Change Date. The production-use restriction is framed as a contestability-window defense against OpenR-class hard-fork attacks during the 36-month basin-formation period, not as a long-run posture.

(c) **Change License conversion.** Effective on the Change Date **2029-06-01**, the Licensor grants every recipient rights under Apache License 2.0 to all then-existing versions of the Licensed Work, and the production-use restriction terminates by operation of the BUSL Change License clause. Per the BUSL spec, the Change Date applies separately to each version of the Licensed Work and may not be extended past the four-year statutory maximum from first publication of that version. The §13.4 trigger schedule may accelerate but cannot delay conversion.

13.4 Five-Trigger Relicense Schedule

The five triggers below are pre-announced at license publication and committed by the Licensor. Each is an early-conversion accelerator: any trigger that fires shortens the path to Apache License 2.0 but cannot extend it. Pre-announcement defangs the Hashicorp-pattern attack (mid-life license tightening) by foreclosing the Licensor’s discretion to relicense more restrictively at any later date. Fig. 13.1 illustrates the trajectory.

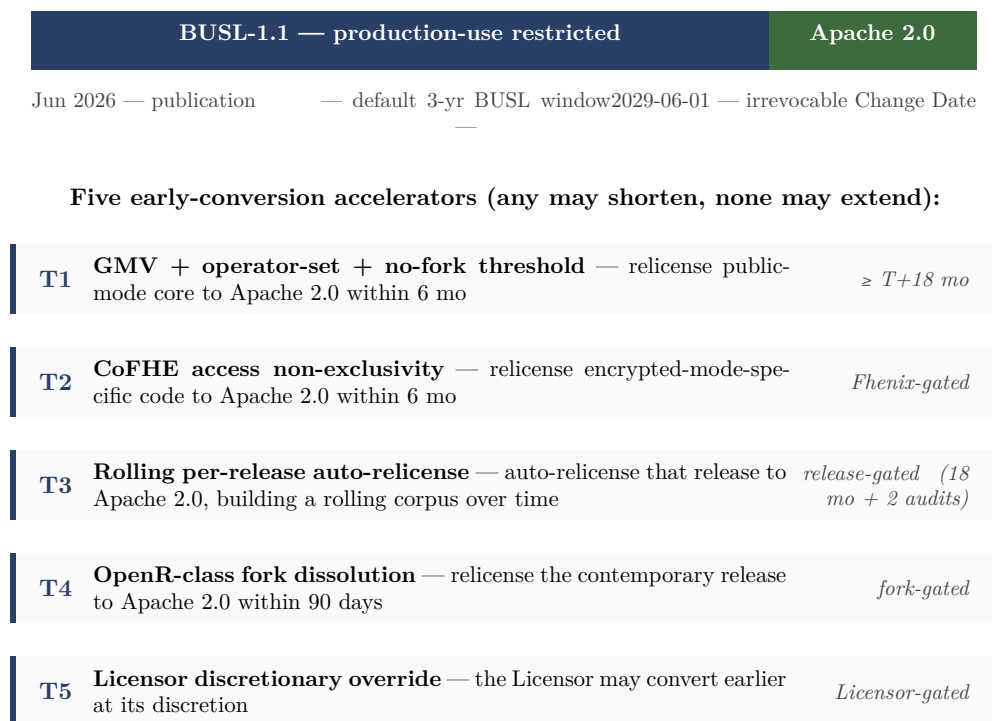


Figure 5: **Fig. 13.1 — License trajectory and acceleration triggers.** The default path (top) converts BUSL-1.1 → Apache 2.0 on the irrevocable Change Date **2029-06-01**. The five triggers below may each independently shorten the BUSL window; none can extend it. Pre-announcement at publication binds the Licensor.

1. **GMV + operator-set + no-fork threshold.** Cumulative GMV exceeds \$500M AND the operator set has at least 30 distinct entities AND no OpenR-class fork has reached \$50M GMV by T+18 months. Action: relicense the public-mode core to Apache License 2.0 within 6 months.

2. **CoFHE access non-exclusivity.** Phenix makes CoFHE access non-exclusive. Action: relicense encrypted-mode-specific code to Apache License 2.0 within 6 months.
3. **Rolling per-release auto-relicense.** A given release has been in production at least 18 months AND has accumulated at least 2 independent audits. Action: auto-relicense that release to Apache License 2.0 regardless of the BUSL N counter, producing a rolling Apache License 2.0 corpus over time.
4. **OpenR-class fork dissolution.** An OpenR-class fork launches, fails (no growth past T+12 months), and dissolves. Action: relicense the contemporary release to Apache License 2.0 within 90 days.
5. **Licensors discretionary override.** The Licensors may, by published commitment subject to public RFC and 30-day attestation publication, direct relicense of any layer ahead of schedule, executed within 60 days. This discretionary path is the codified pressure-release valve, ensuring a relicense route exists independent of the default Change Date and the four objective triggers above. It can only accelerate conversion, never restrict.

13.5 Boundary Conditions

From chaos-net Day 1, the protocol cannot (a) **rugpull contracts** — every protocol contract is immutable (§11.1); no party holds an upgrade authority that could replace deployed bytecode. The foreclosure is structural and code-enforced, not governance-dependent; (b) **extract rents** — the protocol charges no fee; there is no protocol take, treasury cut, or commission at the contract layer, and operators, Gate-builders, and underwriters keep their own fees independently of the protocol; (c) **relicense earlier than the §13.4 trigger schedule for restrictive terms, nor delay conversion past the Change Date 2029-06-01** — the BUSL Change License clause and the trigger pre-announcement jointly foreclose both directions.

The (a) foreclosure under the immutable-contract posture is categorically stronger than the equivalent foreclosure in upgradeable-protocol designs. There is no “DAO under timelocks” required to constrain upgrade authority; there is no upgrade authority to constrain.

§14 — Conclusion & Open Problems

14.1 What this paper specified

ReineiraOS is the union of three published artifacts: the on-chain Reineira Settlement Protocol (immutable contracts on Arbitrum L2 with multi-chain hosting roadmap, §9), the Reineira Settlement Standard (RSS) conformance specification (§5), and the AI-native builder stack of Reineira Atlas and Reineira Code (§1.1, §5.6, §9.10). The reference implementation ships at chaos-net v1.0 (July 2026) and is published by the Software Vendor as non-custodial software (§11.2).

The protocol surface is four composable primitives: confidential escrow, pluggable release gates, encrypted-state recourse, and a stake-collateralised operator set. Each is defined over FHE-encrypted state and admits third-party verification, underwriting, and routing plugins. The four primitives and their plugin extension points compose without protocol-level coordination. Every protocol contract is deployed as an immutable singleton; no entity holds upgrade authority (§11.1).

The launch posture is **Public-Infrastructure**: the protocol charges nothing — operator, Gate, and underwriter fees accrue to those third parties, never to the protocol; operators stake cUSDC; operator registration is permissionless from chaos-net Day 1 at the protocol layer (§8.4).

The license vector is layer-asymmetric and pre-announced. Builder-facing layers (interfaces, SDK, wrappers, plugin examples) are permissively licensed at publication; defensive layers (escrow, recourse, orchestration core) carry BUSL-1.1 during the contestability window and convert to Apache License 2.0 on the irrevocable Change Date 2029-06-01. The §13.4 trigger schedule constrains the Licensors to only earlier conversions, never later.

14.2 What this paper deferred

The paper specifies the protocol layer (the on-chain Settlement Protocol and the RSS conformance surface) and the reference implementation. Off-chain operator services, the agentic / AI-execution composition layer (specified in the technical reference and tracked as Implementation Notes in §8.11), and Reineira Labs Limited commercial products sit above the protocol perimeter and are out of scope for this edition.

The **builder stack** — Reineira Atlas (venture scaffolding engine) and Reineira Code (resolver/policy authoring) — is described where it is load-bearing for the open-economy thesis (§1.1 overview, §5.6 plugin onramp, §9.10 Atlas-deployed third-party L3s) but is not specified as a normative surface. Atlas and Code are versioned alongside the protocol (**platform: "0.1"** in `reineira.json`) so that interface-compatible plugin authoring and venture scaffolding remain coupled to the contract surface across RSS versions; their own product specifications live in their respective repository docs and are out of scope for the whitepaper.

The protocol operates fully **without a token**: the operator economy is cUSDC-bonded. There is no token-based mechanism in the v1.0 design — no fee accrues to the protocol, and no protocol-issued instrument underwrites any surface in this edition.

Operational discipline is out of scope for the whitepaper. Off-chain operational mechanics are handled separately from the protocol specification. The whitepaper commits to the *structural rules* (immutable contracts §11.1) but does not bind operational parameters that would force whitepaper revisions for ordinary operational changes. This split keeps the whitepaper edition-stamped and stable while allowing operational practice to evolve as capacity and circumstances warrant.

14.3 Open Problems

The collated list of every numbered Open Problem from §2–§9. Each row's decision target is taken verbatim from the source callout.

ID	Title	Chapter	Decision target
Open Problem 2.1	Account Abstraction	§2	post-chaos-net v1.0
Open Problem 3.1	Account-Abstraction Convergence	§3	prior to RSS v0.2 (Q4 2026)
Open Problem 4.1	CoFHE Decentralization	§4	pre-mainnet
Open Problem 5.1	Formal Conformance Suite Release	§5	RSS v0.2 (Q4 2026)
Open Problem 6.1	Batch-Redemption Fee-Order Atomicity	§6	chaos-net v1.0
Open Problem 7.1	FHE-priced dynamic-curve premium	§7	prior to chaos-net v1.0
Open Problem 7.2	Asynchronous judge return	§7	prior to chaos-net v1.0
Open Problem 8.1	Operator set growth trajectory	§8	tracked through chaos-net operations; no pre-deployment gate
Open Problem 8.2	Restaking integration	§8	open (deferred indefinitely; conditional on a future RSS revision)
Open Problem 9.1	OP-Stack host native bridge	§9	deferred to OP-Stack host activation milestone

14.4 Implementation Notes — spec-vs-code deltas

Implementation Notes mark items where the v1.0 RSS specification describes a surface that is not yet shipped in the reference implementation. The chapters cite each Note inline; the index below collates them.

Subject		Chapter	Implementation target
Plugin	Surface (async variants, IAgenticJob , IExecution)	§5	RSS v0.2 (Q4 2026)
Storage	Namespace (privara.storage.* retained on legacy slot)	§6	retained at v1.0 for cross-deployment-version compatibility; migration requires new immutable deployment
	Virtual Escrow Surface	§6	chaos-net v1.0
Reference Resolvers	(zkTLS / oracle / prediction-market / multisig)	§6	plugin-examples package
	QuorumAttestedPolicy (async underwriter base)	§7	chaos-net v1.0
	RiskScoreAttestor (and @reineira-os/identity package)	§7	not on roadmap; risk-input is policy-internal
	PoolFactory v1.0 surface (full routing / unstake)	§7	chaos-net v1.0
	CoordinatorRegistry (cross-graph slashing partition)	§8	v1.0-track
Agentic handlers	(LZ0FTHandler , AgentCallHandler , VerdictSubmitHandler , PoolRoutingHandler)	§8	out of scope for v1.0 RSS
Agent identity / quorum registries	(AgentIdentity Registry , QuorumRegistry , etc.)	§8	deferred from v1.0 RSS
	LayerZero OFT Handlers (USDT0 rail)	§9	roadmap; CCTP V2 / USDC ships at v1.0

14.5 Closing statement

The protocol binds itself in four places, each code-enforced or contractually-enforced and pre-announced at publication. (a) **Immutable contracts (§11.1)**. Every protocol contract is deployed as an immutable singleton; no entity holds upgrade authority. The rugpull-via-upgrade failure mode is structurally eliminated, not mitigated. (b) **The protocol charges nothing (§8.8)**. No protocol fee accrues on any settlement path; the **FeeManager** routes operator, Gate, and underwriter fees to those third parties, never to the protocol, and there is no protocol-fee slot anyone can toggle on. (c) **Change Date 2029-06-01 (§13.1)**. Irrevocable; the §13.4 trigger schedule constrains the Licensor to only earlier conversions, never later. (d) **Layer-asymmetric license vector (§13.2)**. Builder-facing layers (interfaces, SDK, plugin examples) are permissively licensed at publication, so any third party can implement an RSS-conformant settlement venue without a commercial license.

Reading paths follow the chapter table in §0. Protocol researchers: §3 → §4 → §5. Security auditors: §4 → §6 / §7 / §8 → Appendix C. Governance and licensing readers: §11 → §13 → Appendix B. The Open Problems and Implementation Notes indexed in §14.3 and §14.4 are the binding next-decision surface for the v1.0 edition.

Appendix A — References

Entries are numbered [N] in ACM Reference Format, grouped by topic and ordered alphabetically by first-author surname within each group, with public standards (EIPs / ERCs / RFCs) collected in a single table at the end. Every in-text citation marker in §§1–14 resolves to exactly one entry below; conversely, every entry below is cited at least once in the body.

A.1 Cryptography and FHE

[1] Aztec Network. 2022. *Aztec Protocol: A Privacy-Preserving Layer-2 on Ethereum*. Vendor whitepaper and developer documentation. <https://docs.aztec.network> (accessed May 2026).

- [2] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. 2020. *TFHE: Fast Fully Homomorphic Encryption over the Torus*. Journal of Cryptology 33, 1 (2020), 34–91. <https://doi.org/10.1007/s00145-019-09319-x>
- [3] Fhenix Network. 2024. *Fhenix: A Confidential-Computation L2 Powered by FHE*. Vendor whitepaper. <https://www.fhenix.io> (accessed May 2026).
- [4] Fhenix CoFHE. 2024. *CoFHE: Coprocessor for FHE on the EVM*. Technical documentation, v0.4. <https://docs.fhenix.io/cofhe> (accessed May 2026). Cited in-body as [Fhenix CoFHE 2024].
- [5] Inco Network. 2024. *Inco: The Universal Confidentiality Layer*. Vendor whitepaper and documentation. <https://docs.inco.org> (accessed May 2026).
- [6] Penumbra Labs. 2023. *Penumbra: A Shielded, Cross-Chain Network for Private Trading on Cosmos*. Vendor specification, v0.62. <https://protocol.penumbra.zone> (accessed May 2026).
- [6a] Oded Regev. 2005. *On lattices, learning with errors, random linear codes, and cryptography*. In Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC 2005), 84–93. <https://doi.org/10.1145/1060590.1060603>
- [7] Zama. 2023. *fhEVM: Confidential EVM Smart Contracts using Fully Homomorphic Encryption*. Vendor whitepaper. <https://github.com/zama-ai/fhevm/blob/main/fhevm-whitepaper.pdf> (accessed May 2026).

A.2 Settlement, Escrow, and Insurance Primitives

- [8] Aave. 2020. *Aave Protocol Whitepaper V1 — Including the Safety Module Specification*. Vendor whitepaper. https://github.com/aave/aave-protocol/blob/master/docs/Aave_Protocol_Whitepaper_v1_0.pdf (accessed May 2026).
- [9] Cover Protocol. 2020. *Cover Protocol Documentation (legacy)*. Vendor documentation, archived. <https://docs.coverprotocol.com> (accessed May 2026).
- [10] David Bone (ed.) for the OpenZeppelin Project. 2017. *OpenZeppelin Contracts: Escrow and Conditional Escrow Reference*. Solidity contract library, v2.x and forward. <https://github.com/OpenZeppelin/openzeppelin-contracts> (accessed May 2026).
- [11] Hugh Karp, et al. 2019. *Nexus Mutual: A Peer-to-Peer Discretionary Mutual on the Ethereum Blockchain*. Vendor whitepaper. https://nexusmutual.io/assets/docs/nmx_white_paperv2_3.pdf (accessed May 2026).
- [12] Joseph Poon and Thaddeus Dryja. 2016. *The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments*. Whitepaper, draft 0.5.9.2. <https://lightning.network/lightning-network-paper.pdf> (accessed May 2026).
- [13] Risk Harbor. 2021. *Risk Harbor: A Parametric Risk Management Marketplace*. Vendor whitepaper. <https://docs.riskharbor.com> (accessed May 2026).
- [14] Sablier Labs. 2020. *Sablier: Money Streaming on Ethereum*. Vendor documentation. <https://docs.sablier.com> (accessed May 2026).
- [15] Sherlock. 2022. *Sherlock: Smart-Contract Coverage with Claim Adjudication and Watson Auditors*. Vendor whitepaper. <https://docs.sherlock.xyz> (accessed May 2026).
- [16] Superfluid Finance. 2021. *Superfluid: Continuous-Flow Money Streams with Off-Chain Net-Flow Accounting*. Vendor documentation. <https://docs.superfluid.finance> (accessed May 2026).
- [17] Reineira Labs (compiler). 2023. *Structural Conflicts in Role-Collapsed CeFi Lending and Custody: A Post-Mortem of the 2022 Episode*. Reineira Labs internal source dossier citing public bankruptcy filings of FTX/Alameda, Celsius, Voyager, and BlockFi (placeholder URL pending publication).

A.3 Restaking and Operator Networks

- [18] EigenLabs. 2023. *EigenLayer: The Restaking Collective*. Vendor whitepaper. https://docs.eigenlayer.xyz/assets/files/EigenLayer_WhitePaper-88c47923ca0319870c611decd6e562ad.pdf (accessed May 2026).
- [19] Karak Network. 2024. *Karak: Universal Restaking and Risk Management*. Vendor whitepaper. <https://docs.karak.network> (accessed May 2026).
- [20] Symbiotic. 2024. *Symbiotic: A Permissionless Shared Security Protocol*. Vendor whitepaper. <https://docs.symbiotic.fi> (accessed May 2026).

A.4 Cross-Chain Transports

- [21] Circle Internet Financial. 2024. *Cross-Chain Transfer Protocol V2 (CCTP V2) Technical Specification*. Vendor documentation. <https://developers.circle.com/stablecoins/docs/cctp-getting-started> (accessed May 2026).
- [22] Hyperlane (Abacus Works). 2023. *Hyperlane: A Permissionless Interoperability Protocol with Modular Interchain Security Modules*. Vendor documentation. <https://docs.hyperlane.xyz> (accessed May 2026).
- [23] Ryan Zarick, Bryan Pellegrino, and Caleb Banister. 2022. *LayerZero: Trustless Omnichain Interoperability Protocol*. Vendor whitepaper, v1.0. https://layerzero.network/publications/LayerZero_Whitepaper_V1.0.pdf
- [24] LayerZero Labs. 2024. *Omnichain Fungible Token (OFT) Standard, V2*. Developer documentation. <https://docs.layerzero.network/v2/developers/evm/oft/quickstart> (accessed May 2026).
- [25] Wormhole Foundation. 2022. *Wormhole: A Generic Cross-Chain Messaging Protocol Secured by a Guardian Set*. Vendor whitepaper. <https://wormhole.com/docs/> (accessed May 2026).

A.5 L2 / L3 Hosting Stacks

- [26] Offchain Labs. 2024. *Arbitrum Orbit: Permissionless L3 Chains Settling to Arbitrum One*. Vendor documentation. <https://docs.arbitrum.io/launch-orbit-chain/orbit-gentle-introduction> (accessed May 2026).
- [27] OP Labs. 2023. *The OP Stack and the Optimism Superchain: A Shared-Sequencing, Standardised-Bridge Layer for OP-Stack Rollups*. Vendor specification. <https://docs.optimism.io/stack/getting-started> (accessed May 2026).
- [28] Polygon Labs. 2024. *Polygon AggLayer: Aggregating ZK-Proven State across Heterogeneous Chains*. Vendor documentation. <https://docs.polygon.technology/agglayer> (accessed May 2026).

A.6 Standards Bodies and Regulation

- [29] European Parliament and Council of the European Union. 2023. *Regulation (EU) 2023/1114 on Markets in Crypto-Assets (MiCA) and Amending Regulations (EU) No 1093/2010 and (EU) No 1095/2010 and Directives 2013/36/EU and (EU) 2019/1937*. Official Journal of the European Union L 150 (June 2023), 40–205. <https://eur-lex.europa.eu/eli/reg/2023/1114/oj>
- [30] Financial Action Task Force. 2023. *FATF Recommendation 16 — Wire Transfers (Travel Rule), and Interpretive Note as Amended for Virtual Assets and Virtual Asset Service Providers*. FATF Standards. <https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Fatf-recommendations.html> (accessed May 2026).
- [31] MariaDB Corporation. 2017. *Business Source License 1.1 (BUSL-1.1) Specification*. Source-available license text and Change License clause. <https://mariadb.com/bsl11/> (accessed May 2026).

A.7 Public Standards (EIPs / ERCs / RFCs)

Entries are referenced in-body as [ERC-XXXX] or [EIP-XXXX] on first mention.

#	ID	Title	URL token
[32]	ERC-20	Token Standard	eip-20
[33]	ERC-165	Standard Interface Detection	eip-165
[34]	EIP-1822	Universal Upgradeable Proxy Standard (UUPS)	eip-1822
[35]	EIP-1967	Standard Proxy Storage Slots	eip-1967
[36]	ERC-2771	Secure Protocol for Native Meta Transactions	eip-2771
[37]	ERC-4337	Account Abstraction Using Alt Mempool	eip-4337
[38]	EIP-4844	Shard Blob Transactions (Proto-Danksharding)	eip-4844
[39]	ERC-7201	Namespaced Storage Layout	eip-7201
[40]	ERC-7984	Confidential Fungible Token (FHERC-20 lineage)	eip-7984
[41]	ERC-8004	Agent Identity Registry	eip-8004
[42]	ERC-2535	Diamond Standard (Multi-Facet Proxy)	eip-2535

Each row above is, formally: [N] ERC-XXXX. YEAR. Title. Ethereum Improvement Proposal. <https://eips.ethereum.org/EIPS/<token>>.

Appendix B — ERC / EIP Compliance Matrices

This appendix records per-contract evidence of ERC / EIP compliance for the v1.0 reference implementation. Each cell is verified by reading the contract source file and grepping for the relevant marker (inheritance, modifier, custom annotation, or imported library). Legend: **D** = implemented directly in the contract file; **inh** = inherited via the protocol base contract or the pausable base contract; **—** = not applicable; **IN** = the v1.0 specification expects this property but an Implementation Note flags a gap.

B.1 Per-contract compliance matrix

Five columns capture the standards each contract claims. The Upgrade-safety stack column collapses four orthogonal compliance facts (EIP-1822 / EIP-1967 proxy slots; **Initializable**; **Ownable**; **Reentrancy Guard**) into a single **inh** cell when all four are inherited together via the protocol base contract; the footnotes break down per-row exceptions.

#	Contract	ERC-165	ERC-7201	ERC-2771	Upgrade-safety stack
1	ConfidentialEscrow	—	inh ¹	inh	inh
2	Escrow	—	inh ¹	inh	inh
3	CCTPV2ConfidentialEscrowReceiver	—	—	inh	inh
4	CCTPV2EscrowReceiver	—	—	inh	inh
5	ConfidentialCoverageManager	—	—	inh	inh
6	ConfidentialRecoursePool	—	—	inh	inh
7	ConfidentialPolicyRegistry	D ²	—	inh	inh
8	ConfidentialPoolFactory	—	—	inh	inh
9	CoverageManager	—	—	inh	inh
10	RecoursePool	—	—	inh	inh
11	PolicyRegistry	D ²	—	inh	inh
12	PoolFactory	—	—	inh	inh
13	CCTPV2Forwarder	—	—	inh	inh
14	FeeManager	—	—	inh	inh
15	OperatorRegistry	—	—	inh ³	inh ³
16	OperatorSlashingManager	—	—	inh ³	inh ³
17	TaskExecutor	D ²	—	inh ³	inh ³
18	CCTPHandler	—	—	inh ³	inh ³
19	ConfidentialUSDC	—	—	—	D ⁴ (token-edge: non-upgradeable; Ownable only)

Footnotes: (1) inherited via the **EscrowCondition** extension, which carries the only contract-level ERC-7201 storage declaration in the reference implementation; see §B.3. The eighteen rows reading **inh** for ERC-7201 are not v1.0 conformance gaps — they inherit upgrade-safe storage from OpenZeppelin v5 upgradeable base contracts (**OwnableUpgradeable**, **ReentrancyGuardUpgradeable**, **UUPSUpgradeable**, **PausableUpgradeable**, **ERC2771ContextUpgradeable**), each of which uses ERC-7201 namespaced slots internally per the OpenZeppelin v5 storage convention. Per-contract ERC-7201 declarations are only required where a contract introduces struct-level storage outside the inherited bases, which the reference implementation deliberately avoids; the **EscrowCondition** row exists because the legacy **privara.storage.*** namespace predates the OZ v5 convention. (2) ERC-165 detection used at the call site via OpenZeppelin’s **ERC165Checker.supportsInterface** helper to validate registered plugins; the contract itself does not publish a **supportsInterface** selector. (3) inherited via the pausable base contract, which extends the protocol base contract with **PausableUpgradeable**. (4) **ConfidentialUSDC** is a non-upgradeable wrapper inheriting **FHERC20ERC20Wrapper**, **Ownable**, **IConfidentialUSDC**; it deliberately opts out of the UUPS / 2771 surface because it sits on the token edge and re-deployment is cheaper than upgrade governance for v1.0.

Inheritance chain. Every state-bearing contract derives from the protocol base contract, which composes **Initializable**, **UUPSUpgradeable**, **OwnableUpgradeable**, **ReentrancyGuardUpgradeable**, and **ERC2771ContextUpgradeable** in a single base. the pausable base contract extends that composition with **PausableUpgradeable** for the orchestration surface. Cells marked **inh** in §B.1 are satisfied by these base contracts and do not require per-contract reimplementations.

B.2 ERC-165 interface detection

Plugin interfaces inherit **IERC165** so that registries and the executor can introspect implementations before storage write. The matrix below lists the canonical extension and core interfaces that exist on disk.

Interface	Inherits IERC165			interfaceId source
IConditionResolver	Yes			type(IConditionResolver).interfaceId
IUnderwriterPolicy	Yes			type(IUnderwriterPolicy).interfaceId
IConfidentialUnderwriterPolicy	Yes			type(IConfidentialUnderwriterPolicy).interfaceId
IEscrow	No	(inherits ICore, Events)	IEscrow	not introspected
IConfidentialEscrow	No	(inherits ICore, Events)	IEscrow	not introspected
ICore	No (event-only base)			—

IConditionResolver inherits **IERC165** per the requirement that escrow plugins be ERC-165-introspectable before registration; the same pattern is applied to both underwriter-policy interfaces so that **PolicyRegistry.register(...)** and **ConfidentialPolicyRegistry.register(...)** can reject incompatible implementations at line 29 of each file.

B.3 ERC-7201 namespaced storage

The protocol declares one ERC-7201 namespaced-storage slot. All other contracts use the OpenZeppelin upgradeable storage layout, which has its own audited slots inside `@openzeppelin/contracts-upgradeable`.

Contract	Namespace string	Slot constant	Declaration
EscrowCondition (abstract extension)	<code>priv0xddbe0eff506238e35addb19b3c2d99c00660b3894c8f79a6042996EscrowCondition</code> <code>storage.ConfidentialEscrowCondition</code> (legacy)	<code>00660b3894c8f79a6042996EscrowCondition100</code>	Condition extension

Implementation Note (Storage Namespace, as of June 2026). The `@custom:storage-location` annotation in `EscrowCondition.sol:13` reads `erc7201:privara.storage.EscrowCondition`, but the slot constant on lines 22-23 is derived from the original namespace `privara.storage.ConfidentialEscrowCondition`, preserved across the `ConfidentialEscrowCondition → EscrowCondition` rename to keep UUPS upgrade compatibility with deployed proxies (the file comment on lines 18-21 documents this). The annotation string and the slot derivation therefore disagree. Convergence is tracked in §6 of this whitepaper and requires a storage-layout migration post-mainnet (a new immutable deployment).

B.4 ERC-2771 meta-transaction support

ERC-2771 trusted-forwarder support is provided once by the protocol base contract via **ERC2771ContextUpgradeable** (inherited verbatim from OpenZeppelin v5 upgradeable bases for storage-namespacing consistency, regardless of whether the deployed contract is upgradeable or immutable). Every contract in the reference implementation therefore exposes **isTrustedForwarder** and routes the sender accessor / the calldata accessor / the context-suffix accessor through the ERC-2771 path; the forwarder address is fixed at construction-time.

B.5 Deployment posture: immutable singletons (v1.0 mainnet)

v1.0 mainnet contracts are deployed as immutable singletons; no UUPS proxy, no upgrade authority, no admin upgrade path (§11.1). The OZ v5 upgradeable base contracts (**OwnableUpgradeable**, **ReentrancyGuardUpgradeable**, **ERC2771ContextUpgradeable**, **PausableUpgradeable**) are inherited for their storage-namespace discipline (ERC-7201) and for interface consistency with prior testnet deployments, but **UUPSUpgradeable** is *not* in the v1.0 mainnet inheritance chain and no `_authorizeUpgrade` hook is exposed.

Earlier testnet deployments retained the UUPS proxy pattern with an **owner** upgrade authority for iteration-speed reasons during the design-finalisation window; that posture is superseded at chaos-net mainnet. Where this whitepaper or earlier internal documentation references “the UUPS upgrade pattern,” “Safe + TimelockController on mainnet target,” or “owner-controlled timelock at mainnet,” those references describe the *testnet posture* and are superseded by the immutable mainnet posture of §11.1.

Functional evolution at mainnet ships as new immutable singleton deployments at new addresses with opt-in user migration. The canonical-deployment registry (a documentation artifact) tracks current v1.x deployment addresses per host chain; users migrate to v2.x by withdrawing from v1 and depositing into v2 at their own initiative. Cross-deployment-version interface compatibility is preserved via ERC-7201 storage namespacing and `uint256[50] private __gap` reserves on every contract.

Factory-created pool deployments (recourse pools created via **PoolFactory**) follow the same immutable pattern: each pool is deployed as an immutable singleton at a deterministic address; no factory-level upgrade authority can replace or modify a deployed pool.

B.6 Standards not used (and why)

ERC-4337 (Account Abstraction) is not used in the v1.0 reference implementation. Sponsored-transaction support is provided via ERC-2771 (§B.4) on the assumption that operator-relayed transactions are sufficient for the v1.0 settlement and recourse surfaces; the move to a bundler/entry-point model is deferred until cross-chain operator UX requires it. See Open Problem 3.1 (Account-Abstraction Convergence) and the selection criteria in §3.5.

ERC-2535 (Diamond Standard) is not used; each top-level contract is an immutable singleton (§B.5). The design rationale is per-primitive deployment boundaries (separate audit scope per contract, separate migration cadence per primitive) and a smaller per-deployment surface for audit and bytecode-diff review. A diamond layout would force shared facet storage and a single **diamondCut** authority — incompatible with the immutable posture of §11.1.

Appendix C — Formal Specification Stubs

This appendix records pre- and post-condition stubs for the canonical RSS interface functions of §5 and the load-bearing entry points of the escrow (§6), recourse (§7), operator (§8), and cross-chain bridge (§9) surfaces. Notation is informal set-builder and algebraic, with encrypted-type references following Definition 4.1 and decryption-authority references following Definition 4.2. The stubs are anchors for future mechanised verification work — Coq, Isabelle, K-framework, Certora, Foundry-invariant, or Hevm — and are not full Hoare-logic proofs.

Notation

- $\text{Enc}(x)$ denotes the TFHE ciphertext of plaintext x (Definition 4.1).
- $\text{authz}(c)$ denotes the decryption-authority set of ciphertext c (Definition 4.2).
- pi_spec denotes the public-skeleton projection (Definition 4.4).
- $\text{state}(t, E)$ denotes the on-chain state of escrow E at time t .
- $s' = T(s, \text{op})$ denotes the state-transition relation for operation op .
- $\text{dom}(\cdot)$ denotes the domain of a mapping; $\text{Dec}(\cdot)$ denotes substrate-mediated decryption.

C.1 IConditionResolver

Signatures from the **IConditionResolver** interface.

C.2 IUnderwriterPolicy / IConfidentialUnderwriterPolicy

Signatures from the `IUnderwriterPolicy` (plain variant) and `IConfidentialUnderwriterPolicy` (FHE variant) interfaces. Stubs target the FHE variant; the plain variant differs only in that `riskScore` is `uint256` and `valid` is `bool`.

C.3 IConfidentialEscrow

Signatures from the `IConfidentialEscrow` interface. The plain `IEscrow` has the same shape with plaintext `address` / `uint256` substituted for `eaddress` / `euint64`.

The interface does not expose a `cancel` operation; cancellation semantics are deferred to a future RSS revision.

C.4 IExecution / IAgenticJob (Implementation Note)

Implementation Note (Agentic surfaces, as of May 2026). The `IAgenticJob` and `IExecution` interfaces named by the v1.0 specification are not yet present in the shared or orchestration interface surfaces. Specification stubs for these interfaces are deferred to the chaos-net v1.0 release per the implementation note in §5.

C.5 Operator surfaces

Signatures from the `IOperatorRegistry` and `ISlashingManager` interfaces. The on-disk `IOperatorRegistry` exposes `requestUnbond` + `withdrawStake` rather than a single `unregisterOperator`.

C.6 Bridge handlers (CCTPHandler)

Signatures from the `CCTPHandler` contract. The handler implements `ITaskHandler`; the `executeTask` entry is the bridge-boundary settlement path.

C.7 Open invariants for mechanised verification

The following invariants are stated in the body chapters and are candidates for mechanised verification:

- **I1 (Encrypted Amount Conservation):** per Proposition 6.2.
- **I2 (LP Solvency):** per Proposition 7.1.
- **I3 (Slashing Safety):** per Proposition 8.1.
- **I4 (Cross-Chain Atomicity Bound):** per Proposition 9.1.
- **I5 (Silent-Failure Indistinguishability):** per Proposition 4.3.
- **I6 (Balance Conservation under FHE):** per Proposition 4.5.

Full mechanised proofs are deferred to post-chaos-net verification work and depend on the choice of framework and TFHE abstraction level.

Open Problem C.1 (Mechanised Proof of System Soundness). Whether the composition of Propositions 4.3, 4.5, 6.1, 6.2, 7.1, 8.1, and 9.1 can be discharged in a single proof framework is unresolved. Selection criteria: (a) verification framework — K-framework, Foundry-invariant, Certora, or Hevm; (b) TFHE abstraction level — opaque oracle returning ciphertext handles, or full TFHE model with IND-CPA simulation; (c) scope — per-primitive properties in isolation, or system-level composition across the four primitives; (d) handling of asynchronous resolvers and cross-chain finality (Proposition 9.1 covers a bounded-finality window no current EVM-only framework models). Decision target: post-chaos-net audit cycle.